```python
from __future__ import annotations

"""
2D boundary-integral / MoM RCS solver.

High-level workflow:
1) Parse geometry and material definitions into boundary primitives.
2) Build boundary-integral operators (single-layer plus normal-derivative terms).
3) Assemble and solve the coupled dielectric trace system (u, q-) with
   continuous linear Galerkin basis and testing functions.
4) Post-process the solved boundary unknowns into monostatic far-field RCS.

Notes:
- Uses e^{-j omega t} convention.
- Supports lossy media via complex wavenumber in the coupled formulation.
- Production discretization uses continuous two-node linear boundary elements.
"""

import cmath
import ctypes
import ctypes.util
import math
import os
import threading
from dataclasses import dataclass
from typing import Any, Callable, Dict, List, Set, Tuple

import numpy as np
try:
    from scipy import special as _SCIPY_SPECIAL
except Exception:
    _SCIPY_SPECIAL = None
try:
    from scipy import linalg as _SCIPY_LINALG
except Exception:
    _SCIPY_LINALG = None
try:
    from scipy.sparse import linalg as _SCIPY_SPARSE_LINALG
except Exception:
    _SCIPY_SPARSE_LINALG = None

try:
    import mpmath as _MPMATH
except Exception:
```

```
    _MPMATH = None

C0 = 299_792_458.0
ETA0 = 376.730313668
EPS = 1e-12
EULER_GAMMA = 0.5772156649015329
CFIE_ALPHA_DEFAULT = 0.2
MAX_PANELS_DEFAULT = 20_000
DEFAULT_PANELS_PER_WAVELENGTH = 20
GMRES_NODE_THRESHOLD = 3000
GMRES_RESTART = 50
GMRES_MAXITER = 200
GMRES_TOL = 1e-8
# Monostatic 2D RCS normalization controls.
#
# For the asymptotic convention used here,
#   G(r) = (j/4) H_0^(2)(k r),
# and for a far-field amplitude A defined such that
#   u_s(r,phi) ~ sqrt(1 / (8*pi*k*r)) * exp(-j(kr-pi/4)) * A(phi),
# the 2D scattering width per unit length is
#   sigma_2d(phi) = |A(phi)|^2 / (4 k).
#
# Use physical 2D scattering-width normalization by default.
#
RCS_NORM_NUMERATOR = 0.25
RCS_NORM_MODE_DEFAULT = "physical"
RCS_NORM_MODE_PHYSICAL = "physical"

@dataclass
class Panel:
    """Single discretized boundary element used by the solver mesh builder."""

    name: str
    seg_type: int
    ibc_flag: int
    ipn1: int
    ipn2: int
    p0: np.ndarray
    p1: np.ndarray
    center: np.ndarray
    tangent: np.ndarray
    normal: np.ndarray
    length: float

@dataclass
class LinearNode:
    """Unique mesh node for a continuous piecewise-linear boundary discretization
```

```
    xy: np.ndarray
    key: Tuple[int, int]

@dataclass
class LinearElement:
    """Two-node straight boundary element used by the Galerkin discretization."""

    name: str
    seg_type: int
    ibc_flag: int
    ipn1: int
    ipn2: int
    node_ids: Tuple[int, int]
    p0: np.ndarray
    p1: np.ndarray
    center: np.ndarray
    tangent: np.ndarray
    normal: np.ndarray
    length: float
    panel_index: int

@dataclass
class LinearMesh:
    """Continuous linear boundary mesh assembled from boundary primitives."""

    nodes: List[LinearNode]
    elements: List[LinearElement]

@dataclass
class PanelCoupledInfo:
    """
    Per-element material and interface bookkeeping for the coupled formulation.

    The unknown vector is [u_trace, q_minus]. This record maps each element's
    plus-side and minus-side constitutive data into the assembled system.
    """

    seg_type: int
    plus_region: int
    minus_region: int
    plus_has_incident: bool
    minus_has_incident: bool
    eps_plus: complex
    mu_plus: complex
    eps_minus: complex
    mu_minus: complex
```

```
    k_plus: complex
    k_minus: complex
    q_plus_beta: complex
    q_plus_gamma: complex
    bc_kind: str
    robin_impedance: complex

@dataclass
class ComplexTable:
    """Frequency-dependent complex scalar table with linear interpolation."""

    freqs_ghz: np.ndarray
    values: np.ndarray

    def sample(self, freq_ghz: float) -> complex:
        if len(self.freqs_ghz) == 1:
            return complex(self.values[0])
        real = np.interp(freq_ghz, self.freqs_ghz, self.values.real)
        imag = np.interp(freq_ghz, self.freqs_ghz, self.values.imag)
        return complex(real, imag)

@dataclass
class MediumTable:
    """Frequency-dependent (eps, mu) table with linear interpolation."""

    freqs_ghz: np.ndarray
    eps_values: np.ndarray
    mu_values: np.ndarray

    def sample(self, freq_ghz: float) -> Tuple[complex, complex]:
        if len(self.freqs_ghz) == 1:
            return complex(self.eps_values[0]), complex(self.mu_values[0])
        eps_r = np.interp(freq_ghz, self.freqs_ghz, self.eps_values.real)
        eps_i = np.interp(freq_ghz, self.freqs_ghz, self.eps_values.imag)
        mu_r = np.interp(freq_ghz, self.freqs_ghz, self.mu_values.real)
        mu_i = np.interp(freq_ghz, self.freqs_ghz, self.mu_values.imag)
        return complex(eps_r, eps_i), complex(mu_r, mu_i)

@dataclass
class PreparedLinearSolver:
    """Reusable linear-solve handle for repeated Ax=b with fixed A."""

    a_mat: np.ndarray
    method: str
    lu: np.ndarray | None = None
    piv: np.ndarray | None = None
    null_basis: np.ndarray | None = None
```

```
    reduced_mat: np.ndarray | None = None
    constraint_mat: np.ndarray | None = None
    preconditioner: Any | None = None
    gmres_restart: int = GMRES_RESTART
    gmres_maxiter: int = GMRES_MAXITER
    gmres_tol: float = GMRES_TOL

class MaterialLibrary:
    """Material lookup facade for constant values and fort.* frequency tables."""

    def __init__(
        self,
        impedance_models: Dict[int, complex | ComplexTable],
        dielectric_models: Dict[int, Tuple[complex, complex] | MediumTable],
    ):
        self.impedance_models = impedance_models
        self.dielectric_models = dielectric_models
        self.warnings: List[str] = []
        self._warning_seen: Set[str] = set()

    @classmethod
    def from_entries(
        cls,
        ibcs_entries: List[List[str]],
        dielectric_entries: List[List[str]],
        base_dir: str,
    ) -> "MaterialLibrary":
        impedance_models: Dict[int, complex | ComplexTable] = {}
        dielectric_models: Dict[int, Tuple[complex, complex] | MediumTable] = {}

        for row in ibcs_entries:
            if not row:
                continue
            flag = _parse_flag(row[0])
            if flag <= 0:
                continue
            if flag > 50:
                path = _resolve_fort_file(base_dir, flag)
                impedance_models[flag] = _load_impedance_table(path)
                continue
            z_real = _parse_float(row[1] if len(row) > 1 else 0.0, 0.0)
            z_imag = _parse_float(row[2] if len(row) > 2 else 0.0, 0.0)
            impedance_models[flag] = _ensure_finite_complex(
                complex(z_real, z_imag),
                f"IBC flag {flag} impedance",
            )
```

```
    for row in dielectric_entries:
        if not row:
            continue
        flag = _parse_flag(row[0])
        if flag <= 0:
            continue
        if flag > 50:
            path = _resolve_fort_file(base_dir, flag)
            dielectric_models[flag] = _load_dielectric_table(path)
            continue
        eps_real = _parse_float(row[1] if len(row) > 1 else 1.0, 1.0)
        eps_imag = _parse_float(row[2] if len(row) > 2 else 0.0, 0.0)
        mu_real = _parse_float(row[3] if len(row) > 3 else 1.0, 1.0)
        mu_imag = _parse_float(row[4] if len(row) > 4 else 0.0, 0.0)
        eps_raw = _ensure_finite_complex(
            complex(eps_real, -eps_imag),
            f"Dielectric flag {flag} epsilon",
        )
        mu_raw = _ensure_finite_complex(
            complex(mu_real, -mu_imag),
            f"Dielectric flag {flag} mu",
        )
        eps = _normalize_material_value(eps_raw, 1.0 + 0j)
        mu = _normalize_material_value(mu_raw, 1.0 + 0j)
        dielectric_models[flag] = (eps, mu)

    return cls(impedance_models=impedance_models, dielectric_models=dielectri

def get_impedance(self, flag: int, freq_ghz: float) -> complex:
    if flag <= 0:
        return 0.0 + 0.0j
    model = self.impedance_models.get(flag)
    if model is None:
        return 0.0 + 0.0j
    if isinstance(model, ComplexTable):
        fmin = float(np.min(model.freqs_ghz))
        fmax = float(np.max(model.freqs_ghz))
        if freq_ghz < fmin or freq_ghz > fmax:
            self._warn_once(
                f"Impedance flag {flag} sampled at {freq_ghz:g} GHz outside t
            )
        return _ensure_finite_complex(
            model.sample(freq_ghz),
            f"IBC flag {flag} impedance sampled at {freq_ghz:g} GHz",
        )
    return _ensure_finite_complex(model, f"IBC flag {flag} impedance")
```

```
    def get_medium(self, flag: int, freq_ghz: float) -> Tuple[complex, complex]:
        if flag <= 0:
            return 1.0 + 0.0j, 1.0 + 0.0j
        model = self.dielectric_models.get(flag)
        if model is None:
            return 1.0 + 0.0j, 1.0 + 0.0j
        if isinstance(model, MediumTable):
            fmin = float(np.min(model.freqs_ghz))
            fmax = float(np.max(model.freqs_ghz))
            if freq_ghz < fmin or freq_ghz > fmax:
                self._warn_once(
                    f"Dielectric flag {flag} sampled at {freq_ghz:g} GHz outside
                )
            eps, mu = model.sample(freq_ghz)
            return (
                _normalize_material_value(eps, 1.0 + 0.0j),
                _normalize_material_value(mu, 1.0 + 0.0j),
            )
        eps, mu = model
        return (
            _normalize_material_value(eps, 1.0 + 0.0j),
            _normalize_material_value(mu, 1.0 + 0.0j),
        )

    def _warn_once(self, message: str) -> None:
        if message in self._warning_seen:
            return
        self._warning_seen.add(message)
        self.warnings.append(message)

    def warn_once(self, message: str) -> None:
        self._warn_once(message)

class _BesselBackend:
    """
    Real-argument Bessel backend.

    Backend preference:
    1) libc/libm j0/y0/j1/y1
    2) scipy.special j0/y0/j1/y1
    3) local series/asymptotic approximations
    """

    def __init__(self):
        self._lib = None
        self._j0 = None
        self._y0 = None
```

```
    self._j1 = None
    self._y1 = None
    self._backend_name = "series-fallback"

    libname = ctypes.util.find_library("m")
    if libname:
        try:
            lib = ctypes.CDLL(libname)
            self._j0 = lib.j0
            self._j0.argtypes = [ctypes.c_double]
            self._j0.restype = ctypes.c_double
            self._y0 = lib.y0
            self._y0.argtypes = [ctypes.c_double]
            self._y0.restype = ctypes.c_double
            self._j1 = lib.j1
            self._j1.argtypes = [ctypes.c_double]
            self._j1.restype = ctypes.c_double
            self._y1 = lib.y1
            self._y1.argtypes = [ctypes.c_double]
            self._y1.restype = ctypes.c_double
            self._lib = lib
            self._backend_name = "libm"
            return
        except Exception:
            self._lib = None
            self._j0 = None
            self._y0 = None
            self._j1 = None
            self._y1 = None

    if _SCIPY_SPECIAL is not None:
        try:
            # Ensure required real-order functions are present/callable.
            float(_SCIPY_SPECIAL.j0(0.0))
            float(_SCIPY_SPECIAL.y0(1.0))
            float(_SCIPY_SPECIAL.j1(0.0))
            float(_SCIPY_SPECIAL.y1(1.0))
            self._backend_name = "scipy-special"
        except Exception:
            self._backend_name = "series-fallback"

@property
def available(self) -> bool:
    return self._backend_name != "series-fallback"

@property
def backend_name(self) -> str:
```

```
        return self._backend_name

    def j0(self, x: float) -> float:
        if self._j0 is not None:
            return float(self._j0(float(x)))
        if self._backend_name == "scipy-special" and _SCIPY_SPECIAL is not None:
            return float(_SCIPY_SPECIAL.j0(float(x)))
        return _j0_fallback(x)

    def y0(self, x: float) -> float:
        if self._y0 is not None:
            return float(self._y0(float(x)))
        if self._backend_name == "scipy-special" and _SCIPY_SPECIAL is not None:
            return float(_SCIPY_SPECIAL.y0(float(x)))
        return _y0_fallback(x)

    def j1(self, x: float) -> float:
        if self._j1 is not None:
            return float(self._j1(float(x)))
        if self._backend_name == "scipy-special" and _SCIPY_SPECIAL is not None:
            return float(_SCIPY_SPECIAL.j1(float(x)))
        return _j1_fallback(x)

    def y1(self, x: float) -> float:
        if self._y1 is not None:
            return float(self._y1(float(x)))
        if self._backend_name == "scipy-special" and _SCIPY_SPECIAL is not None:
            return float(_SCIPY_SPECIAL.y1(float(x)))
        return _y1_fallback(x)

_BESSEL = _BesselBackend()

# --- Special-function helpers ------------------------------------------------
# Real-argument helpers are used heavily for lossless/real-k paths.
# Complex-argument Hankel is needed for lossy media (complex-k kernels).
def _j0_fallback(x: float) -> float:
    ax = abs(float(x))
    if ax < 12.0:
        xsq = 0.25 * ax * ax
        term = 1.0
        acc = 1.0
        for m in range(1, 80):
            term *= -xsq / (m * m)
            acc += term
            if abs(term) < 1e-16:
                break
        return acc
```

```
    phase = ax - math.pi / 4.0
    amp = math.sqrt(2.0 / (math.pi * ax))
    return amp * math.cos(phase)

def _y0_fallback(x: float) -> float:
    ax = max(abs(float(x)), 1e-12)
    if ax < 12.0:
        j0 = _j0_fallback(ax)
        xsq = 0.25 * ax * ax
        term = 1.0
        harmonic = 0.0
        acc = 0.0
        for m in range(1, 80):
            harmonic += 1.0 / m
            term *= -xsq / (m * m)
            acc -= harmonic * term
            if abs(term * harmonic) < 1e-16:
                break
        return (2.0 / math.pi) * ((math.log(ax / 2.0) + EULER_GAMMA) * j0 + acc)

    phase = ax - math.pi / 4.0
    amp = math.sqrt(2.0 / (math.pi * ax))
    return amp * math.sin(phase)

def _j1_fallback(x: float) -> float:
    ax = abs(float(x))
    sign = -1.0 if x < 0.0 else 1.0
    if ax < 12.0:
        xhalf = 0.5 * ax
        term = xhalf
        acc = term
        for m in range(1, 80):
            term *= -(xhalf * xhalf) / (m * (m + 1.0))
            acc += term
            if abs(term) < 1e-16:
                break
        return sign * acc

    phase = ax - 3.0 * math.pi / 4.0
    amp = math.sqrt(2.0 / (math.pi * ax))
    return sign * (amp * math.cos(phase))

def _y1_fallback(x: float) -> float:
    ax = max(abs(float(x)), 1e-12)
    sign = -1.0 if x < 0.0 else 1.0
    if ax < 12.0:
```

```
        # Full series using harmonic numbers (Abramowitz & Stegun 9.1.56):
        # Y1(x) = (2/pi)[J1(x)(ln(x/2)+gamma) - 1/x]
        #        - (1/pi) Sum_{k=0}^inf (-1)^k (H_k+H_{k+1}) (x/2)^{2k+1} / (k!(k
        # where H_0=0, H_k = 1 + 1/2 + ... + 1/k.
        j1 = _j1_fallback(ax)
        xhalf = 0.5 * ax
        xhalf2 = xhalf * xhalf
        term = xhalf  # k=0: (x/2)^1 / (0! * 1!)
        h_k = 0.0     # H_0 = 0
        h_k1 = 1.0    # H_1 = 1
        acc = (h_k + h_k1) * term
        for k in range(1, 80):
            term *= -xhalf2 / (k * (k + 1.0))
            h_k += 1.0 / k
            h_k1 = h_k + 1.0 / (k + 1.0)
            contrib = (h_k + h_k1) * term
            acc += contrib
            if abs(contrib) < 1e-16 * max(1.0, abs(acc)):
                break
        return sign * (
            (2.0 / math.pi) * (math.log(ax / 2.0) + EULER_GAMMA) * j1
            - (2.0 / (math.pi * ax))
            - (1.0 / math.pi) * acc
        )

    phase = ax - 3.0 * math.pi / 4.0
    amp = math.sqrt(2.0 / (math.pi * ax))
    return sign * (amp * math.sin(phase))

def _complex_hankel_backend_name() -> str:
    """Report which complex Hankel implementation is active."""

    if _SCIPY_SPECIAL is not None:
        return "scipy-special"
    if _MPMATH is not None:
        return "mpmath"
    return "native-series-asymptotic"

def _raise_if_untrusted_math_backends() -> None:
    """Abort production solves when only approximation fallback math backends are

    if _BESSEL.backend_name == "series-fallback":
        raise RuntimeError(
            "Aborting solve: real-argument Bessel evaluation is using the native
            "fallback backend. Install SciPy or provide libm j0/y0/j1/y1 before r
        )
```

```
def _j0_complex_series(z: complex) -> complex:
    zz = 0.25 * z * z
    term = 1.0 + 0.0j
    acc = term
    for m in range(1, 160):
        term *= -zz / (m * m)
        acc += term
        if abs(term) <= 1e-16 * max(1.0, abs(acc)):
            break
    return acc

def _j1_complex_series(z: complex) -> complex:
    z_half = 0.5 * z
    term = z_half
    acc = term
    for m in range(1, 160):
        term *= -(z_half * z_half) / (m * (m + 1.0))
        acc += term
        if abs(term) <= 1e-16 * max(1.0, abs(acc)):
            break
    return acc

def _y0_complex_series(z: complex) -> complex:
    z_safe = z if abs(z) > 1e-14 else (1e-14 + 0.0j)
    j0 = _j0_complex_series(z_safe)
    zz = 0.25 * z_safe * z_safe
    term = 1.0 + 0.0j
    harmonic = 0.0
    acc = 0.0 + 0.0j
    for m in range(1, 160):
        harmonic += 1.0 / m
        term *= -zz / (m * m)
        acc -= harmonic * term
        if abs(harmonic * term) <= 1e-16 * max(1.0, abs(acc), abs(j0)):
            break
    return (2.0 / math.pi) * ((cmath.log(z_safe / 2.0) + EULER_GAMMA) * j0 + acc)

def _y1_complex_series(z: complex) -> complex:
    z_safe = z if abs(z) > 1e-14 else (1e-14 + 0.0j)
    j1 = _j1_complex_series(z_safe)
    z_half = 0.5 * z_safe
    term = z_half
    harmonic_k = 0.0
    harmonic_k1 = 1.0
    acc = (harmonic_k + harmonic_k1) * term
    for k in range(1, 160):
        term *= -(z_half * z_half) / (k * (k + 1.0))
```

```
        harmonic_k += 1.0 / k
        harmonic_k1 = harmonic_k + 1.0 / (k + 1.0)
        contrib = (harmonic_k + harmonic_k1) * term
        acc += contrib
        if abs(contrib) <= 1e-16 * max(1.0, abs(acc), abs(j1)):
            break
    return (
        (2.0 / math.pi) * (cmath.log(z_safe / 2.0) + EULER_GAMMA) * j1
        - (1.0 / math.pi) * acc
        - (2.0 / (math.pi * z_safe))
    )

def _hankel2_asymptotic(order: int, z: complex) -> complex:
    z_safe = z if abs(z) > 1e-14 else (1e-14 + 0.0j)
    phase = z_safe - ((0.5 * order) + 0.25) * math.pi
    amp = cmath.sqrt(2.0 / (math.pi * z_safe))
    return amp * cmath.exp(-1j * phase)

def _hankel2_complex_fallback(order: int, z: complex) -> complex:
    if abs(z) < 16.0:
        if order == 0:
            return _j0_complex_series(z) - 1j * _y0_complex_series(z)
        return _j1_complex_series(z) - 1j * _y1_complex_series(z)
    return _hankel2_asymptotic(order, z)

def _hankel2_0(x: complex | float) -> complex:
    """Hankel H_0^(2), with real fast path and no approximation fallback in produ

    z = complex(x)
    if abs(z.imag) <= 1e-14 and z.real >= 0.0:
        xx = max(float(z.real), 1e-12)
        return complex(_BESSEL.j0(xx), -_BESSEL.y0(xx))
    if _SCIPY_SPECIAL is not None:
        try:
            return complex(_SCIPY_SPECIAL.hankel2(0, z))
        except Exception:
            pass
    if _MPMATH is not None:
        try:
            return complex(_MPMATH.hankel2(0, z))
        except Exception:
            pass
    raise RuntimeError(
        "Aborting solve: complex Hankel H_0^(2) evaluation requires SciPy or mpma
        "Native complex series/asymptotic fallback is disabled for production run
    )
```

```
def _hankel2_1(x: complex | float) -> complex:
    """Hankel H_1^(2), with real fast path and no approximation fallback in produ

    z = complex(x)
    if abs(z.imag) <= 1e-14 and z.real >= 0.0:
        xx = max(float(z.real), 1e-12)
        return complex(_BESSEL.j1(xx), -_BESSEL.y1(xx))
    if _SCIPY_SPECIAL is not None:
        try:
            return complex(_SCIPY_SPECIAL.hankel2(1, z))
        except Exception:
            pass
    if _MPMATH is not None:
        try:
            return complex(_MPMATH.hankel2(1, z))
        except Exception:
            pass
    raise RuntimeError(
        "Aborting solve: complex Hankel H_1^(2) evaluation requires SciPy or mpma
        "Native complex series/asymptotic fallback is disabled for production run
    )

def _parse_flag(token: Any) -> int:
    text = str(token).strip().lower()
    if not text:
        return 0
    if text.startswith("fort."):
        text = text.split("fort.", 1)[1]
    try:
        return int(float(text))
    except ValueError:
        return 0

def _parse_float(token: Any, default: float = 0.0) -> float:
    try:
        return float(token)
    except (TypeError, ValueError):
        return default

def _parse_int(token: Any, default: int = 0) -> int:
    try:
        return int(round(float(token)))
    except (TypeError, ValueError):
        return default

def _ensure_finite_complex(value: complex, context: str) -> complex:
    z = complex(value)
```

```
    if not np.isfinite(z.real) or not np.isfinite(z.imag):
        raise ValueError(f"{context} contains non-finite value {z!r}.")
    return z

def _normalize_material_value(value: complex, fallback: complex) -> complex:
    if not np.isfinite(value.real) or not np.isfinite(value.imag) or abs(value) <
        return fallback
    return value

def _resolve_fort_file(base_dir: str, flag: int) -> str:
    """Resolve a fort.<flag> material file relative to geometry dir/current cwd."

    name = f"fort.{flag}"
    candidates = [os.path.join(base_dir, name), os.path.join(os.getcwd(), name)]
    for path in candidates:
        if os.path.isfile(path):
            return path
    raise FileNotFoundError(f"Could not locate material file {name} in {base_dir}

def _read_numeric_rows(path: str, min_columns: int) -> List[List[float]]:
    """Read numeric rows, drop comments/bad rows, sort by frequency, de-duplicate

    rows: List[List[float]] = []
    with open(path, "r") as f:
        for lineno, raw in enumerate(f, start=1):
            line = raw.split("#", 1)[0].strip()
            if not line:
                continue
            tokens = line.split()
            if len(tokens) < min_columns:
                continue
            try:
                parsed = [float(tokens[i]) for i in range(min_columns)]
            except ValueError:
                continue
            if not all(math.isfinite(v) for v in parsed):
                raise ValueError(
                    f"Material file '{path}' line {lineno} contains non-finite nu
                )
            rows.append(parsed)
    if not rows:
        raise ValueError(f"No valid numeric rows found in {path}")
    rows.sort(key=lambda row: row[0])
    dedup: Dict[float, List[float]] = {}
    for row in rows:
        dedup[row[0]] = row
    return [dedup[freq] for freq in sorted(dedup.keys())]
```

```
def _load_impedance_table(path: str) -> ComplexTable:
    """Load frequency -> complex impedance table: f(GHz) z_real z_imag."""

    rows = _read_numeric_rows(path, 3)
    freqs = np.asarray([r[0] for r in rows], dtype=float)
    vals = np.asarray([complex(r[1], r[2]) for r in rows], dtype=np.complex128)
    return ComplexTable(freqs_ghz=freqs, values=vals)

def _load_dielectric_table(path: str) -> MediumTable:
    """Load frequency -> (eps, mu) table: f eps_r eps_i mu_r mu_i."""

    rows = _read_numeric_rows(path, 5)
    freqs = np.asarray([r[0] for r in rows], dtype=float)
    eps_vals = np.asarray([complex(r[1], -r[2]) for r in rows], dtype=np.complex1
    mu_vals = np.asarray([complex(r[3], -r[4]) for r in rows], dtype=np.complex12
    return MediumTable(freqs_ghz=freqs, eps_values=eps_vals, mu_values=mu_vals)

def _canonical_user_polarization_label(label: str | None) -> str:
    text = str(label or '').strip().upper()
    if text in {'TE', 'VV', 'V', 'VERTICAL'}:
        return 'TE'
    if text in {'TM', 'HH', 'H', 'HORIZONTAL'}:
        return 'TM'
    raise ValueError(f"Unsupported polarization '{label}'. Use TE/TM or VV/HH.")

def _primary_alias_for_user_polarization(label: str) -> str:
    return 'VV' if _canonical_user_polarization_label(label) == 'TE' else 'HH'

def _normalize_polarization(polarization: str) -> str:
    """
    Normalize user-facing polarization labels without swapping TE and TM.

    Production convention in this file is now direct:
    - user/internal "TE" are the same branch
    - user/internal "TM" are the same branch

    Accepted aliases are retained for convenience:
    - TE, VV, V, VERTICAL -> TE
    - TM, HH, H, HORIZONTAL -> TM
    """

    pol = (polarization or "").strip().upper()
    if pol in {"TE", "VV", "V", "VERTICAL"}:
        return "TE"
    if pol in {"TM", "HH", "H", "HORIZONTAL"}:
        return "TM"
```

```
    raise ValueError(f"Unsupported polarization '{polarization}'. Use TE/TM or VV

def _unit_scale_to_meters(units: str) -> float:
    value = (units or "").strip().lower()
    if value in {"inch", "inches", "in"}:
        return 0.0254
    if value in {"meter", "meters", "m"}:
        return 1.0
    raise ValueError(f"Unsupported geometry units '{units}'. Use inches or meters

def _wrap_to_pi(angle: float) -> float:
    return (angle + math.pi) % (2.0 * math.pi) - math.pi

def _arc_center_from_endpoints(p0: np.ndarray, p1: np.ndarray, ang_rad: float) ->
    """Recover arc center/radius/start-angle from endpoints + subtended angle.

    Handles near-180° arcs where the two candidate centers nearly coincide,
    and angles in the range (-2*pi, 2*pi).
    """

    chord_vec = p1 - p0
    chord = float(np.linalg.norm(chord_vec))
    if chord <= EPS:
        raise ValueError("Arc endpoints are coincident.")

    abs_phi = abs(ang_rad)
    if abs_phi <= 1e-9:
        raise ValueError("Arc angle too small.")
    if abs_phi > 2.0 * math.pi - 1e-9:
        raise ValueError("Arc angle must be less than 2*pi.")

    sin_half = math.sin(abs_phi * 0.5)
    if abs(sin_half) < 1e-14:
        raise ValueError("Arc angle produces degenerate geometry.")

    radius = chord / (2.0 * abs(sin_half))

    # For angles near pi, tan(phi/2) -> infinity and h -> 0.
    # Use a numerically stable formula: h = radius * cos(phi/2) with sign.
    cos_half = math.cos(abs_phi * 0.5)
    h = radius * cos_half  # distance from midpoint to center along perp

    mid = 0.5 * (p0 + p1)
    perp = np.asarray([-chord_vec[1], chord_vec[0]], dtype=float) / chord

    # For arcs with |angle| < pi, center is on the side determined by the sign.
    # For arcs with |angle| > pi (reflex), center is on the opposite side.
```

```
    # The sign of ang_rad determines CW vs CCW sweep direction.
    if ang_rad > 0:
        center = mid + perp * h
    else:
        center = mid - perp * h

    a0 = math.atan2(p0[1] - center[1], p0[0] - center[0])

    # Verify endpoint recovery.
    p1_pred = center + radius * np.asarray([math.cos(a0 + ang_rad), math.sin(a0 +
    err = float(np.linalg.norm(p1_pred - p1))
    if err > 1e-6 * max(chord, radius):
        # Try the other candidate center.
        center_alt = mid - perp * h if ang_rad > 0 else mid + perp * h
        a0_alt = math.atan2(p0[1] - center_alt[1], p0[0] - center_alt[0])
        p1_alt = center_alt + radius * np.asarray([math.cos(a0_alt + ang_rad), ma
        err_alt = float(np.linalg.norm(p1_alt - p1))
        if err_alt < err:
            center, a0 = center_alt, a0_alt

    return center, radius, a0

def _discretize_primitive(p0: np.ndarray, p1: np.ndarray, ang_deg: float, count:
    """Generate panel endpoints for a line or circular-arc primitive."""

    count = max(1, int(count))
    if abs(ang_deg) < 1e-9:
        return [p0 + (p1 - p0) * (i / count) for i in range(count + 1)]

    ang_rad = math.radians(ang_deg)
    center, radius, a0 = _arc_center_from_endpoints(p0, p1, ang_rad)
    points: List[np.ndarray] = []
    for i in range(count + 1):
        t = i / count
        a = a0 + ang_rad * t
        points.append(center + radius * np.asarray([math.cos(a), math.sin(a)], dt
    return points

def _primitive_length(p0: np.ndarray, p1: np.ndarray, ang_deg: float) -> float:
    chord = float(np.linalg.norm(p1 - p0))
    if chord <= EPS:
        return 0.0
    if abs(ang_deg) < 1e-9:
        return chord
    phi = abs(math.radians(ang_deg))
    radius = chord / (2.0 * math.sin(phi * 0.5))
    return radius * phi
```

```
def _panel_count_from_n(n_prop: int, primitive_len: float, min_wavelength: float)
    """
    Convert geometry n property to panel count.

    n > 0: explicit panel count.
    n < 0: panels-per-wavelength style control.
    """

    if primitive_len <= EPS:
        return 1
    if n_prop > 0:
        return max(1, n_prop)
    if n_prop < 0:
        n_wave = max(1, abs(n_prop))
        target = max(min_wavelength / n_wave, primitive_len / 2000.0)
        return max(1, int(math.ceil(primitive_len / target)))
    # n_prop == 0: apply default panels-per-wavelength density.
    if min_wavelength > EPS:
        target = min_wavelength / float(DEFAULT_PANELS_PER_WAVELENGTH)
        return max(1, int(math.ceil(primitive_len / target)))
    return max(1, int(math.ceil(primitive_len / (primitive_len / 10.0 + EPS))))

def _segment_closed_area2(point_pairs: List[Dict[str, Any]], meters_scale: float)
    """Return (is_closed, signed_area2) for a multi-primitive segment chain."""

    if not point_pairs:
        return False, 0.0

    pts: List[tuple[float, float]] = []
    for idx, pair in enumerate(point_pairs):
        x1 = _parse_float(pair.get("x1", 0.0), 0.0) * meters_scale
        y1 = _parse_float(pair.get("y1", 0.0), 0.0) * meters_scale
        x2 = _parse_float(pair.get("x2", 0.0), 0.0) * meters_scale
        y2 = _parse_float(pair.get("y2", 0.0), 0.0) * meters_scale
        if idx == 0:
            pts.append((x1, y1))
        pts.append((x2, y2))

    if len(pts) < 3:
        return False, 0.0

    xs = [p[0] for p in pts]
    ys = [p[1] for p in pts]
    diag = max(float(math.hypot(max(xs) - min(xs), max(ys) - min(ys))), 1.0)
    tol = max(1e-12, 1e-9 * diag)
    closed = math.hypot(pts[0][0] - pts[-1][0], pts[0][1] - pts[-1][1]) <= tol
```

```
    if not closed:
        return False, 0.0

    area2 = 0.0
    for (x0, y0), (x1, y1) in zip(pts[:-1], pts[1:]):
        area2 += x0 * y1 - x1 * y0
    return True, float(area2)

def _normalize_segment_orientation(
    seg_type: int,
    ang_deg: float,
    point_pairs: List[Dict[str, Any]],
    meters_scale: float,
) -> tuple[List[Dict[str, Any]], float]:
    """
    Auto-flip closed contours to enforce CCW winding (outward normals).

    For closed segments (TYPE 3/4/5 with closed geometry), the normal direction
    determines which side is IPN1/plus vs IPN2/minus.  A CW-wound contour
    produces inward normals, silently swapping material assignments.  We detect
    this via the shoelace signed area and reverse the primitive chain when
    negative (CW).

    Open segments are left untouched — the user controls orientation explicitly.
    """

    if seg_type < 3:
        # TYPE 1 and 2 (PEC/IBC on exterior side) don't depend on winding.
        return point_pairs, ang_deg

    is_closed, area2 = _segment_closed_area2(point_pairs, meters_scale)
    if not is_closed:
        return point_pairs, ang_deg

    if area2 >= 0.0:
        # Already CCW — normals point outward.
        return point_pairs, ang_deg

    # CW winding detected — reverse the primitive chain.
    reversed_pairs: List[Dict[str, Any]] = []
    for pair in reversed(point_pairs):
        reversed_pairs.append({
            'x1': pair.get('x2', 0.0),
            'y1': pair.get('y2', 0.0),
            'x2': pair.get('x1', 0.0),
            'y2': pair.get('y1', 0.0),
        })
```

```
    # Flip arc angle sign when reversing direction.
    return reversed_pairs, -ang_deg

def _snapshot_segments(geometry_snapshot: Dict[str, Any]) -> List[Dict[str, Any]]
    return list(geometry_snapshot.get('segments', []) or [])

def _solver_point_key(x: float, y: float, tol: float) -> Tuple[int, int]:
    inv = 1.0 / max(tol, 1e-12)
    return int(round(float(x) * inv)), int(round(float(y) * inv))

def _points_close(a: Tuple[float, float], b: Tuple[float, float], tol: float) ->
    return ((float(a[0]) - float(b[0])) ** 2 + (float(a[1]) - float(b[1])) ** 2)

def _segment_intersects_strict(
    a1: Tuple[float, float],
    a2: Tuple[float, float],
    b1: Tuple[float, float],
    b2: Tuple[float, float],
    tol: float,
) -> bool:
    if _points_close(a1, b1, tol) or _points_close(a1, b2, tol) or _points_close(
        return False

    def orient(p, q, r):
        return (float(q[0]) - float(p[0])) * (float(r[1]) - float(p[1])) - (float

    def on_seg(p, q, r):
        return (
            min(float(p[0]), float(r[0])) - tol <= float(q[0]) <= max(float(p[0])
            and min(float(p[1]), float(r[1])) - tol <= float(q[1]) <= max(float(p
        )

    o1 = orient(a1, a2, b1)
    o2 = orient(a1, a2, b2)
    o3 = orient(b1, b2, a1)
    o4 = orient(b1, b2, a2)

    if ((o1 > tol and o2 < -tol) or (o1 < -tol and o2 > tol)) and ((o3 > tol and
        return True
    if abs(o1) <= tol and on_seg(a1, b1, a2):
        return True
    if abs(o2) <= tol and on_seg(a1, b2, a2):
        return True
    if abs(o3) <= tol and on_seg(b1, a1, b2):
        return True
    if abs(o4) <= tol and on_seg(b1, a2, b2):
        return True
```

```
    return False

def validate_geometry_snapshot_for_solver(
    geometry_snapshot: Dict[str, Any],
    base_dir: str,
) -> Dict[str, Any]:
    """
    Strict solver-side preflight for geometry/material consistency.

    This complements the GUI validator and protects headless solves / exports.
    Fatal problems raise before assembly begins.
    """

    segments = _snapshot_segments(geometry_snapshot)
    if not segments:
        raise ValueError('Geometry snapshot contains no segments.')

    ibc_rows = [list(row) for row in (geometry_snapshot.get('ibcs', []) or []) if
    diel_rows = [list(row) for row in (geometry_snapshot.get('dielectrics', []) o
    ibc_flags = {_parse_flag(row[0]) for row in ibc_rows if row}
    diel_flags = {_parse_flag(row[0]) for row in diel_rows if row}

    warnings: List[str] = []
    primitives: List[Tuple[int, int, str, Tuple[float, float], Tuple[float, float
    all_points: List[Tuple[float, float]] = []

    for seg_idx, seg in enumerate(segments):
        props = list(seg.get('properties', []) or [])
        if len(props) < 6:
            props.extend([''] * (6 - len(props)))
        seg_name = str(seg.get('name', f'segment_{seg_idx + 1}'))
        seg_type = _parse_flag(props[0] if props and str(props[0]).strip() else s
        ibc_flag = _parse_flag(props[3])
        ipn1 = _parse_flag(props[4])
        ipn2 = _parse_flag(props[5])
        point_pairs = list(seg.get('point_pairs', []) or [])

        if seg_type < 1 or seg_type > 5:
            raise ValueError(f"Segment '{seg_name}' has invalid TYPE '{props[0]}'
        if not point_pairs:
            raise ValueError(f"Segment '{seg_name}' has no primitives/point_pairs

        prev_end = None
        for prim_idx, pair in enumerate(point_pairs):
            x1 = _parse_float(pair.get('x1', 0.0), 0.0)
            y1 = _parse_float(pair.get('y1', 0.0), 0.0)
            x2 = _parse_float(pair.get('x2', 0.0), 0.0)
```

```
        y2 = _parse_float(pair.get('y2', 0.0), 0.0)
        vals = [x1, y1, x2, y2]
        if not all(math.isfinite(v) for v in vals):
            raise ValueError(f"Segment '{seg_name}' primitive {prim_idx + 1}
        if ((x2 - x1) ** 2 + (y2 - y1) ** 2) <= EPS * EPS:
            raise ValueError(f"Segment '{seg_name}' primitive {prim_idx + 1}
        p1 = (x1, y1)
        p2 = (x2, y2)
        primitives.append((seg_idx, prim_idx, seg_name, p1, p2))
        all_points.extend([p1, p2])
        if prev_end is not None and not _points_close(prev_end, p1, 1e-9):
            warnings.append(
                f"Segment '{seg_name}' has a disconnected primitive chain bet
            )
        prev_end = p2

    if ibc_flag > 0:
        if ibc_flag > 50:
            _resolve_fort_file(base_dir, ibc_flag)
        elif ibc_flag not in ibc_flags:
            raise ValueError(f"Segment '{seg_name}' references undefined IBC

    if seg_type == 3:
        if ipn1 <= 0:
            raise ValueError(f"TYPE 3 segment '{seg_name}' requires IPN1 > 0.
        if ipn1 > 50:
            _resolve_fort_file(base_dir, ipn1)
        elif ipn1 not in diel_flags:
            raise ValueError(f"TYPE 3 segment '{seg_name}' references undefin
    elif seg_type == 4:
        if ipn1 <= 0:
            raise ValueError(f"TYPE 4 segment '{seg_name}' requires IPN1 > 0.
        if ipn1 > 50:
            _resolve_fort_file(base_dir, ipn1)
        elif ipn1 not in diel_flags:
            raise ValueError(f"TYPE 4 segment '{seg_name}' references undefin
    elif seg_type == 5:
        if ipn1 <= 0 or ipn2 <= 0:
            raise ValueError(f"TYPE 5 segment '{seg_name}' requires IPN1 > 0
        for flag in (ipn1, ipn2):
            if flag > 50:
                _resolve_fort_file(base_dir, flag)
            elif flag not in diel_flags:
                raise ValueError(f"TYPE 5 segment '{seg_name}' references und

xs = [p[0] for p in all_points] if all_points else [0.0]
ys = [p[1] for p in all_points] if all_points else [0.0]
```

```
    diag = max(math.hypot(max(xs) - min(xs), max(ys) - min(ys)), 1.0)
    tol = max(1e-8, 1e-6 * diag)

    node_degree: Dict[Tuple[int, int], int] = {}
    for _, _, _, p1, p2 in primitives:
        key1 = _solver_point_key(p1[0], p1[1], tol)
        key2 = _solver_point_key(p2[0], p2[1], tol)
        node_degree[key1] = node_degree.get(key1, 0) + 1
        node_degree[key2] = node_degree.get(key2, 0) + 1

    dangling_nodes = sum(1 for v in node_degree.values() if v == 1)
    high_degree_nodes = sum(1 for v in node_degree.values() if v > 2)
    if dangling_nodes > 0:
        warnings.append(f'Geometry contains {dangling_nodes} dangling endpoint no
    if high_degree_nodes > 0:
        warnings.append(f'Geometry contains {high_degree_nodes} high-degree node(

    for i in range(len(primitives)):
        seg_i, prim_i, name_i, a1, a2 = primitives[i]
        for j in range(i + 1, len(primitives)):
            seg_j, prim_j, name_j, b1, b2 = primitives[j]
            if seg_i == seg_j and abs(prim_i - prim_j) <= 1:
                continue
            if _segment_intersects_strict(a1, a2, b1, b2, tol):
                raise ValueError(
                    f"Geometry contains an unsupported segment intersection betwe
                )

    return {
        'segment_count': int(len(segments)),
        'primitive_count': int(len(primitives)),
        'dangling_nodes': int(dangling_nodes),
        'high_degree_nodes': int(high_degree_nodes),
        'warning_count': int(len(warnings)),
        'warnings': warnings,
    }

def _build_panels(
    geometry_snapshot: Dict[str, Any],
    meters_scale: float,
    min_wavelength: float,
    max_panels: int = MAX_PANELS_DEFAULT,
) -> List[Panel]:
    """
    Discretize all geometry primitives into oriented boundary elements.

    Normal direction follows endpoint ordering of each primitive.
```

```
    """

    panels: List[Panel] = []
    segments = geometry_snapshot.get("segments", []) or []

    for seg in segments:
        props = list(seg.get("properties", []) or [])
        seg_type = _parse_flag(props[0] if len(props) > 0 else 2)
        n_prop = _parse_int(props[1] if len(props) > 1 else 1, 1)
        ang_deg = _parse_float(props[2] if len(props) > 2 else 0.0, 0.0)
        ibc_flag = _parse_flag(props[3] if len(props) > 3 else 0)
        ipn1 = _parse_flag(props[4] if len(props) > 4 else 0)
        ipn2 = _parse_flag(props[5] if len(props) > 5 else 0)
        name = str(seg.get("name", "segment"))

        point_pairs = list(seg.get("point_pairs", []) or [])
        point_pairs, ang_deg = _normalize_segment_orientation(seg_type, ang_deg,
        for pair in point_pairs:
            p0 = np.asarray([
                _parse_float(pair.get("x1", 0.0), 0.0) * meters_scale,
                _parse_float(pair.get("y1", 0.0), 0.0) * meters_scale,
            ], dtype=float)
            p1 = np.asarray([
                _parse_float(pair.get("x2", 0.0), 0.0) * meters_scale,
                _parse_float(pair.get("y2", 0.0), 0.0) * meters_scale,
            ], dtype=float)

            prim_len = _primitive_length(p0, p1, ang_deg)
            count = _panel_count_from_n(n_prop, prim_len, min_wavelength)
            pts = _discretize_primitive(p0, p1, ang_deg, count)

            for i in range(count):
                q0 = pts[i]
                q1 = pts[i + 1]
                vec = q1 - q0
                length = float(np.linalg.norm(vec))
                if length <= EPS:
                    continue
                tangent = vec / length
                # Project convention: a segment drawn left->right has an upward n
                # This makes IPN1 the medium on the GUI-indicated normal side.
                normal = np.asarray([-tangent[1], tangent[0]], dtype=float)
                center = 0.5 * (q0 + q1)
                panels.append(
                    Panel(
                        name=name,
                        seg_type=seg_type,
```

```
                        ibc_flag=ibc_flag,
                        ipn1=ipn1,
                        ipn2=ipn2,
                        p0=q0,
                        p1=q1,
                        center=center,
                        tangent=tangent,
                        normal=normal,
                        length=length,
                    )
                )

    if not panels:
        raise ValueError("Geometry does not contain any valid discretized panels.
    max_allowed = max(1, int(max_panels))
    if len(panels) > max_allowed:
        raise ValueError(
            f"Discretization produced {len(panels)} panels; limit is {max_allowed
            "Reduce n/frequency range or increase max_panels."
        )
    return panels

def _linear_node_snap_key(xy: np.ndarray, tol: float = 1.0e-9) -> Tuple[int, int]
    scale = 1.0 / max(float(tol), EPS)
    return (int(round(float(xy[0]) * scale)), int(round(float(xy[1]) * scale)))

def _linear_shape_values(xi: float) -> np.ndarray:
    x = float(xi)
    return np.asarray([1.0 - x, x], dtype=float)

def _build_linear_mesh(
    panels: List[Panel],
    node_snap_tol: float = 1.0e-9,
) -> LinearMesh:
    """
    Convert boundary elements into a continuous two-node linear boundary mesh.

    This is the stage-1 data-structure upgrade for the future linear Galerkin pat
    Each panel becomes one linear element, while shared endpoints are merged into
    unique global nodes by snapped coordinates.
    """

    node_index: Dict[Tuple[int, int], int] = {}
    nodes: List[LinearNode] = []
    elements: List[LinearElement] = []

    def get_node_id(xy: np.ndarray) -> int:
```

```
        key = _linear_node_snap_key(xy, tol=node_snap_tol)
        idx = node_index.get(key)
        if idx is not None:
            return idx
        idx = len(nodes)
        node_index[key] = idx
        nodes.append(LinearNode(xy=np.asarray(xy, dtype=float).copy(), key=key))
        return idx

    for pidx, panel in enumerate(panels):
        n0 = get_node_id(panel.p0)
        n1 = get_node_id(panel.p1)
        elements.append(
            LinearElement(
                name=panel.name,
                seg_type=panel.seg_type,
                ibc_flag=panel.ibc_flag,
                ipn1=panel.ipn1,
                ipn2=panel.ipn2,
                node_ids=(n0, n1),
                p0=np.asarray(panel.p0, dtype=float).copy(),
                p1=np.asarray(panel.p1, dtype=float).copy(),
                center=np.asarray(panel.center, dtype=float).copy(),
                tangent=np.asarray(panel.tangent, dtype=float).copy(),
                normal=np.asarray(panel.normal, dtype=float).copy(),
                length=float(panel.length),
                panel_index=int(pidx),
            )
        )

    if not elements:
        raise ValueError("Linear mesh construction requires at least one element.
    return LinearMesh(nodes=nodes, elements=elements)

def _linear_panel_signature_from_info(
    panel: Panel,
    info: PanelCoupledInfo,
) -> Tuple[Any, ...]:
    """Topology signature used to decide when linear nodes may be shared safely."

    return (
        int(panel.seg_type),
        int(panel.ibc_flag),
        int(panel.ipn1),
        int(panel.ipn2),
        int(info.minus_region),
        int(info.plus_region),
```

```
        str(info.bc_kind),
    )

def _build_linear_mesh_interface_aware(
    panels: List[Panel],
    infos: List[PanelCoupledInfo],
    node_snap_tol: float = 1.0e-9,
) -> Tuple[LinearMesh, Dict[str, int]]:
    """
    Build a linear boundary mesh that only shares nodes across the *same* interfa

    This hardens the linear/Galerkin path for ordinary corners where distinct int
    touch at the same geometric coordinate. Those cases should not be forced to s
    nodal DOF, because that incorrectly imposes trace continuity across different

    True branching nodes where more than two elements of the same interface signa
    still reported separately by `_linear_coupled_node_report` for diagnostics.
    solver in production runs.
    """

    if len(panels) != len(infos):
        raise ValueError("Interface-aware linear mesh requires matching panels an

    node_index: Dict[Tuple[Tuple[int, int], Tuple[Any, ...]], int] = {}
    nodes: List[LinearNode] = []
    elements: List[LinearElement] = []
    geometric_keys: Set[Tuple[int, int]] = set()

    def get_node_id(xy: np.ndarray, signature: Tuple[Any, ...]) -> int:
        geom_key = _linear_node_snap_key(xy, tol=node_snap_tol)
        geometric_keys.add(geom_key)
        full_key = (geom_key, signature)
        idx = node_index.get(full_key)
        if idx is not None:
            return idx
        idx = len(nodes)
        node_index[full_key] = idx
        nodes.append(LinearNode(xy=np.asarray(xy, dtype=float).copy(), key=geom_k
        return idx

    for pidx, (panel, info) in enumerate(zip(panels, infos)):
        sig = _linear_panel_signature_from_info(panel, info)
        n0 = get_node_id(panel.p0, sig)
        n1 = get_node_id(panel.p1, sig)
        elements.append(
            LinearElement(
                name=panel.name,
```

```
                seg_type=panel.seg_type,
                ibc_flag=panel.ibc_flag,
                ipn1=panel.ipn1,
                ipn2=panel.ipn2,
                node_ids=(n0, n1),
                p0=np.asarray(panel.p0, dtype=float).copy(),
                p1=np.asarray(panel.p1, dtype=float).copy(),
                center=np.asarray(panel.center, dtype=float).copy(),
                tangent=np.asarray(panel.tangent, dtype=float).copy(),
                normal=np.asarray(panel.normal, dtype=float).copy(),
                length=float(panel.length),
                panel_index=int(pidx),
            )
        )

    if not elements:
        raise ValueError("Interface-aware linear mesh construction requires at le

    mesh = LinearMesh(nodes=nodes, elements=elements)
    geometric_count = int(len(geometric_keys))
    total_nodes = int(len(nodes))
    split_nodes = max(0, total_nodes - geometric_count)

    # Count geometric locations where multiple interface signatures created separ
    geo_key_counts: Dict[Tuple[int, int], int] = {}
    for (gk, _sig), _nid in node_index.items():
        geo_key_counts[gk] = geo_key_counts.get(gk, 0) + 1
    multi_sig = sum(1 for c in geo_key_counts.values() if c > 1)

    stats = {
        "linear_geometric_node_count": geometric_count,
        "linear_interface_split_nodes": split_nodes,
        "shared_node_count": geometric_count,
        "split_node_count": split_nodes,
        "split_boundary_primitive_count": int(len(elements)),
        "multi_signature_node_count": multi_sig,
    }
    return mesh, stats

def _linear_param_to_point(elem: LinearElement, xi: float) -> np.ndarray:
    return elem.p0 + float(xi) * (elem.p1 - elem.p0)

def _linear_interval_point(elem: LinearElement, interval: Tuple[float, float], us
    a, b = float(interval[0]), float(interval[1])
    return _linear_param_to_point(elem, a if use_start else b)

def _linear_interval_length(elem: LinearElement, interval: Tuple[float, float]) -
```

```
    a, b = float(interval[0]), float(interval[1])
    return max(abs(b - a) * float(elem.length), 0.0)

def _linear_interval_midpoint(elem: LinearElement, interval: Tuple[float, float])
    a, b = float(interval[0]), float(interval[1])
    return _linear_param_to_point(elem, 0.5 * (a + b))

def _linear_map_local_to_parent(interval: Tuple[float, float], local_xi: float, s
    a, b = float(interval[0]), float(interval[1])
    h = b - a
    x = float(local_xi)
    return (a + h * x) if start_is_shared else (b - h * x)

def _linear_shared_interval_endpoint_info(
    obs_elem: LinearElement,
    obs_interval: Tuple[float, float],
    src_elem: LinearElement,
    src_interval: Tuple[float, float],
    tol: float = 1.0e-12,
) -> Tuple[bool, bool] | None:
    obs_pts = [
        _linear_interval_point(obs_elem, obs_interval, True),
        _linear_interval_point(obs_elem, obs_interval, False),
    ]
    src_pts = [
        _linear_interval_point(src_elem, src_interval, True),
        _linear_interval_point(src_elem, src_interval, False),
    ]
    for obs_is_start, op in enumerate(obs_pts):
        for src_is_start, sp in enumerate(src_pts):
            if float(np.linalg.norm(op - sp)) <= float(tol):
                return bool(obs_is_start == 0), bool(src_is_start == 0)
    return None

def _integrate_linear_pair_box(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
    obs_interval: Tuple[float, float],
    src_interval: Tuple[float, float],
    obs_order: int,
    src_order: int,
) -> np.ndarray:
    qt_obs, qw_obs = _get_quadrature(max(2, int(obs_order)))
    qt_src, qw_src = _get_quadrature(max(2, int(src_order)))
    obs_scale = max(float(obs_interval[1]) - float(obs_interval[0]), 0.0)
    src_scale = max(float(src_interval[1]) - float(src_interval[0]), 0.0)
```

```
    obs_len = float(obs_elem.length) * obs_scale
    src_len = float(src_elem.length) * src_scale
    block = np.zeros((2, 2), dtype=np.complex128)
    if obs_len <= 0.0 or src_len <= 0.0:
        return block

    for tobs, wobs in zip(qt_obs, qw_obs):
        xi_obs = float(obs_interval[0]) + obs_scale * float(tobs)
        phi_obs = _linear_shape_values(xi_obs)
        robs = _linear_param_to_point(obs_elem, xi_obs)
        for tsrc, wsrc in zip(qt_src, qw_src):
            xi_src = float(src_interval[0]) + src_scale * float(tsrc)
            phi_src = _linear_shape_values(xi_src)
            rsrc = _linear_param_to_point(src_elem, xi_src)
            kval = complex(kernel_eval(robs, rsrc))
            block += (float(wobs) * float(wsrc) * kval) * np.outer(phi_obs, phi_s

    return block * obs_len * src_len

def _integrate_linear_pair_box_sk_vectorized(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    k0: complex | float,
    obs_normal_deriv: bool,
    obs_interval: Tuple[float, float],
    src_interval: Tuple[float, float],
    obs_order: int,
    src_order: int,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Vectorized tensor-Gauss 2×2 S and K block assembly for one element pair.

    Evaluates all quadrature point pairs at once using array Hankel functions,
    avoiding per-point Python-loop overhead.  Returns (S_block, K_block).
    """

    qt_obs, qw_obs = _get_quadrature(max(2, int(obs_order)))
    qt_src, qw_src = _get_quadrature(max(2, int(src_order)))
    oa, ob = float(obs_interval[0]), float(obs_interval[1])
    sa, sb = float(src_interval[0]), float(src_interval[1])
    obs_scale = max(ob - oa, 0.0)
    src_scale = max(sb - sa, 0.0)
    obs_len = float(obs_elem.length) * obs_scale
    src_len = float(src_elem.length) * src_scale
    s_block = np.zeros((2, 2), dtype=np.complex128)
    k_block = np.zeros((2, 2), dtype=np.complex128)
    if obs_len <= 0.0 or src_len <= 0.0:
```

```
    return s_block, k_block

nobs = len(qt_obs)
nsrc = len(qt_src)

# Precompute all parametric coordinates and physical points.
xi_obs_all = oa + obs_scale * np.asarray(qt_obs, dtype=float)          # (nob
xi_src_all = sa + src_scale * np.asarray(qt_src, dtype=float)          # (nsr
phi_obs_all = np.column_stack([1.0 - xi_obs_all, xi_obs_all])          # (nob
phi_src_all = np.column_stack([1.0 - xi_src_all, xi_src_all])          # (nsr

obs_seg = obs_elem.p1 - obs_elem.p0
src_seg = src_elem.p1 - src_elem.p0
robs_all = obs_elem.p0[None, :] + xi_obs_all[:, None] * obs_seg[None, :]  # (
rsrc_all = src_elem.p0[None, :] + xi_src_all[:, None] * src_seg[None, :]  # (

# All pairwise differences: (nobs, nsrc, 2)
diff = robs_all[:, None, :] - rsrc_all[None, :, :]
dist = np.sqrt(np.sum(diff * diff, axis=2))   # (nobs, nsrc)
dist_safe = np.maximum(dist, EPS)

# Green's function: G = j/4 * H_0^(2)(k*r)
kr = np.asarray(complex(k0) * dist_safe, dtype=np.complex128)
kr[np.abs(kr) <= 1e-12] = 1e-12 + 0.0j
h0 = _hankel2_0_array(kr.ravel()).reshape(nobs, nsrc)
g_vals = 0.25j * h0   # (nobs, nsrc)

# Double-layer kernel
h1 = _hankel2_1_array(kr.ravel()).reshape(nobs, nsrc)
if obs_normal_deriv:
    # dG/dn_obs
    proj = np.sum(diff * obs_elem.normal[None, None, :], axis=2) / dist_safe
    dk_vals = (-0.25j * complex(k0)) * h1 * proj
else:
    # dG/dn_src (note: diff = robs - rsrc, projection with src normal flips s
    proj = np.sum(src_elem.normal[None, None, :] * diff, axis=2) / dist_safe
    dk_vals = (0.25j * complex(k0)) * h1 * proj
dk_vals[dist <= EPS] = 0.0

# Weight tensor: (nobs, nsrc)
w_outer = np.outer(np.asarray(qw_obs, dtype=float), np.asarray(qw_src, dtype=

# Accumulate 2×2 blocks using einsum.
# weighted_g = w * G, shape (nobs, nsrc)
weighted_g = w_outer * g_vals
weighted_k = w_outer * dk_vals
```

```
    # s_block[a,b] = sum_{i,j} weighted_g[i,j] * phi_obs[i,a] * phi_src[j,b]
    s_block = np.einsum('ij,ia,jb->ab', weighted_g, phi_obs_all, phi_src_all)
    k_block = np.einsum('ij,ia,jb->ab', weighted_k, phi_obs_all, phi_src_all)

    scale = obs_len * src_len
    return s_block * scale, k_block * scale

def _integrate_linear_self_duffy(
    elem: LinearElement,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
    interval: Tuple[float, float],
    order: int = 20,
) -> np.ndarray:
    qt, qw = _get_quadrature(max(4, int(order)))
    a, b = float(interval[0]), float(interval[1])
    h = max(b - a, 0.0)
    elem_len = float(elem.length) * h
    block = np.zeros((2, 2), dtype=np.complex128)
    if elem_len <= 0.0:
        return block

    for u, wu in zip(qt, qw):
        uu = float(u)
        jac_outer = float(wu) * uu
        t_major = a + h * uu
        s_major = t_major
        robs_major = _linear_param_to_point(elem, t_major)
        rsrc_major = _linear_param_to_point(elem, s_major)
        phi_t_major = _linear_shape_values(t_major)
        phi_s_major = _linear_shape_values(s_major)
        for v, wv in zip(qt, qw):
            vv = float(v)
            weight = jac_outer * float(wv)
            # Triangle: s <= t
            xi_t = a + h * uu
            xi_s = a + h * (uu * vv)
            phi_t = _linear_shape_values(xi_t)
            phi_s = _linear_shape_values(xi_s)
            robs = _linear_param_to_point(elem, xi_t)
            rsrc = _linear_param_to_point(elem, xi_s)
            block += weight * complex(kernel_eval(robs, rsrc)) * np.outer(phi_t,
            # Triangle: t <= s
            xi_t2 = a + h * (uu * vv)
            xi_s2 = a + h * uu
            phi_t2 = _linear_shape_values(xi_t2)
            phi_s2 = _linear_shape_values(xi_s2)
            robs2 = _linear_param_to_point(elem, xi_t2)
```

```
            rsrc2 = _linear_param_to_point(elem, xi_s2)
            block += weight * complex(kernel_eval(robs2, rsrc2)) * np.outer(phi_t

    return block * (elem_len * elem_len)

def _integrate_linear_touching_duffy(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
    obs_interval: Tuple[float, float],
    src_interval: Tuple[float, float],
    obs_start_is_shared: bool,
    src_start_is_shared: bool,
    order: int = 20,
) -> np.ndarray:
    qt, qw = _get_quadrature(max(4, int(order)))
    obs_len = _linear_interval_length(obs_elem, obs_interval)
    src_len = _linear_interval_length(src_elem, src_interval)
    block = np.zeros((2, 2), dtype=np.complex128)
    if obs_len <= 0.0 or src_len <= 0.0:
        return block

    for u, wu in zip(qt, qw):
        uu = float(u)
        jac_outer = float(wu) * uu
        for v, wv in zip(qt, qw):
            vv = float(v)
            weight = jac_outer * float(wv)
            # Triangle 1: source local distance <= observation local distance
            xi_obs = _linear_map_local_to_parent(obs_interval, uu, obs_start_is_s
            xi_src = _linear_map_local_to_parent(src_interval, uu * vv, src_start
            phi_obs = _linear_shape_values(xi_obs)
            phi_src = _linear_shape_values(xi_src)
            robs = _linear_param_to_point(obs_elem, xi_obs)
            rsrc = _linear_param_to_point(src_elem, xi_src)
            block += weight * complex(kernel_eval(robs, rsrc)) * np.outer(phi_obs
            # Triangle 2: observation local distance <= source local distance
            xi_obs2 = _linear_map_local_to_parent(obs_interval, uu * vv, obs_star
            xi_src2 = _linear_map_local_to_parent(src_interval, uu, src_start_is_
            phi_obs2 = _linear_shape_values(xi_obs2)
            phi_src2 = _linear_shape_values(xi_src2)
            robs2 = _linear_param_to_point(obs_elem, xi_obs2)
            rsrc2 = _linear_param_to_point(src_elem, xi_src2)
            block += weight * complex(kernel_eval(robs2, rsrc2)) * np.outer(phi_o

    return block * (obs_len * src_len)
```

```
def _integrate_linear_pair_recursive(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
    obs_interval: Tuple[float, float],
    src_interval: Tuple[float, float],
    obs_order: int,
    src_order: int,
    depth: int = 0,
    max_depth: int = 3,
) -> np.ndarray:
    obs_len = _linear_interval_length(obs_elem, obs_interval)
    src_len = _linear_interval_length(src_elem, src_interval)
    block = np.zeros((2, 2), dtype=np.complex128)
    if obs_len <= 0.0 or src_len <= 0.0:
        return block

    same_elem_same_interval = (
        obs_elem.panel_index == src_elem.panel_index
        and abs(float(obs_interval[0]) - float(src_interval[0])) <= 1.0e-15
        and abs(float(obs_interval[1]) - float(src_interval[1])) <= 1.0e-15
    )
    if same_elem_same_interval:
        order = max(6, int(max(obs_order, src_order)) + 1)
        return _integrate_linear_self_duffy(
            obs_elem,
            kernel_eval,
            interval=obs_interval,
            order=order,
        )

    shared = _linear_shared_interval_endpoint_info(obs_elem, obs_interval, src_el
    if shared is not None:
        order = max(6, int(max(obs_order, src_order)) + 1)
        return _integrate_linear_touching_duffy(
            obs_elem,
            src_elem,
            kernel_eval,
            obs_interval=obs_interval,
            src_interval=src_interval,
            obs_start_is_shared=bool(shared[0]),
            src_start_is_shared=bool(shared[1]),
            order=order,
        )

    obs_mid = _linear_interval_midpoint(obs_elem, obs_interval)
    src_mid = _linear_interval_midpoint(src_elem, src_interval)
```

```
distance = float(np.linalg.norm(obs_mid - src_mid))
scale = max(obs_len, src_len, EPS)
ratio = distance / scale

# Refine near-singular element pairs adaptively before falling back to tensor
if depth < max_depth and ratio < 0.95:
    oa, ob = float(obs_interval[0]), float(obs_interval[1])
    sa, sb = float(src_interval[0]), float(src_interval[1])
    if ratio < 0.16:
        om = 0.5 * (oa + ob)
        sm = 0.5 * (sa + sb)
        sub_obs = [(oa, om), (om, ob)]
        sub_src = [(sa, sm), (sm, sb)]
        for oi in sub_obs:
            for si in sub_src:
                block += _integrate_linear_pair_recursive(
                    obs_elem,
                    src_elem,
                    kernel_eval,
                    oi,
                    si,
                    obs_order=obs_order,
                    src_order=src_order,
                    depth=depth + 1,
                    max_depth=max_depth,
                )
        return block
    if obs_len >= src_len:
        om = 0.5 * (oa + ob)
        return (
            _integrate_linear_pair_recursive(
                obs_elem, src_elem, kernel_eval, (oa, om), src_interval,
                obs_order=obs_order, src_order=src_order, depth=depth + 1, ma
            )
            + _integrate_linear_pair_recursive(
                obs_elem, src_elem, kernel_eval, (om, ob), src_interval,
                obs_order=obs_order, src_order=src_order, depth=depth + 1, ma
            )
        )
    sm = 0.5 * (sa + sb)
    return (
        _integrate_linear_pair_recursive(
            obs_elem, src_elem, kernel_eval, obs_interval, (sa, sm),
            obs_order=obs_order, src_order=src_order, depth=depth + 1, max_de
        )
        + _integrate_linear_pair_recursive(
            obs_elem, src_elem, kernel_eval, obs_interval, (sm, sb),
```

```
                obs_order=obs_order, src_order=src_order, depth=depth + 1, max_de
            )
        )

    adapt_order, _ = _near_singular_scheme(distance, scale)
    tensor_order = max(int(max(obs_order, src_order)), min(16, int(max(5, adapt_o
    return _integrate_linear_pair_box(
        obs_elem,
        src_elem,
        kernel_eval,
        obs_interval=obs_interval,
        src_interval=src_interval,
        obs_order=tensor_order,
        src_order=tensor_order,
    )

def _integrate_linear_pair_generic(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
    obs_order: int = 6,
    src_order: int = 6,
) -> np.ndarray:
    """
    Assemble a 2x2 Galerkin block for one observation/source element pair.

    This upgraded implementation keeps the straight-element tensor-Gauss backbone
    adds two accuracy-critical improvements for the experimental linear/Galerkin
    - Duffy-type quadrature for same-element and endpoint-touching singular pairs
    - adaptive recursive interval subdivision for near-singular pairs
    """

    return _integrate_linear_pair_recursive(
        obs_elem,
        src_elem,
        kernel_eval,
        obs_interval=(0.0, 1.0),
        src_interval=(0.0, 1.0),
        obs_order=obs_order,
        src_order=src_order,
        depth=0,
        max_depth=6,
    )

def _stable_hankel2_array(order: int, x: np.ndarray) -> np.ndarray:
    """Robust array Hankel evaluator for real and complex arguments.
```

```
    Uses scaled SciPy Hankel for complex arguments when available, then repairs
    any remaining non-finite entries with the existing scalar helpers.
    """

    z = np.asarray(x, dtype=np.complex128)
    out: np.ndarray | None = None
    if _SCIPY_SPECIAL is not None:
        try:
            # Real fast path when possible.
            if np.all(np.abs(z.imag) <= 1e-14) and np.all(z.real >= 0.0):
                xr = np.maximum(z.real.astype(float, copy=False), 1e-12)
                if order == 0:
                    out = np.asarray(_SCIPY_SPECIAL.j0(xr) - 1j * _SCIPY_SPECIAL.
                else:
                    out = np.asarray(_SCIPY_SPECIAL.j1(xr) - 1j * _SCIPY_SPECIAL.
            elif hasattr(_SCIPY_SPECIAL, 'hankel2e'):
                scaled = np.asarray(_SCIPY_SPECIAL.hankel2e(order, z), dtype=np.c
                out = scaled * np.exp(-1j * z)
            else:
                out = np.asarray(_SCIPY_SPECIAL.hankel2(order, z), dtype=np.compl
        except Exception:
            out = None
    if out is None:
        vec = np.vectorize(_hankel2_0 if order == 0 else _hankel2_1, otypes=[np.c
        return np.asarray(vec(z), dtype=np.complex128)

    finite = np.isfinite(out.real) & np.isfinite(out.imag)
    if not np.all(finite):
        vec = np.vectorize(_hankel2_0 if order == 0 else _hankel2_1, otypes=[np.c
        repaired = np.asarray(vec(z[~finite]), dtype=np.complex128)
        out = np.asarray(out, dtype=np.complex128)
        out[~finite] = repaired
    return np.asarray(out, dtype=np.complex128)

def _hankel2_0_array(x: np.ndarray) -> np.ndarray:
    return _stable_hankel2_array(0, x)

def _hankel2_1_array(x: np.ndarray) -> np.ndarray:
    return _stable_hankel2_array(1, x)

def _green_2d_array(k0: complex | float, r: np.ndarray) -> np.ndarray:
    rr = np.maximum(np.asarray(r, dtype=float), EPS)
    x = np.asarray(complex(k0) * rr, dtype=np.complex128)
    x[np.abs(x) <= 1e-12] = 1e-12 + 0.0j
    return 0.25j * _hankel2_0_array(x)

def _dgreen_dn_obs_array(k0: complex | float, r_vec: np.ndarray, n_obs: np.ndarra
```

```
    rr = np.linalg.norm(r_vec, axis=1)
    out = np.zeros(rr.shape[0], dtype=np.complex128)
    mask = rr > EPS
    if not np.any(mask):
        return out
    rrm = rr[mask]
    x = np.asarray(complex(k0) * rrm, dtype=np.complex128)
    x[np.abs(x) <= 1e-12] = 1e-12 + 0.0j
    h1 = _hankel2_1_array(x)
    projection = (r_vec[mask] @ np.asarray(n_obs, dtype=float)) / rrm
    out[mask] = (-0.25j * complex(k0)) * h1 * projection
    return out

def _dgreen_dn_src_array(k0: complex | float, r_vec: np.ndarray, n_src: np.ndarra
    rr = np.linalg.norm(r_vec, axis=1)
    out = np.zeros(rr.shape[0], dtype=np.complex128)
    mask = rr > EPS
    if not np.any(mask):
        return out
    rrm = rr[mask]
    x = np.asarray(complex(k0) * rrm, dtype=np.complex128)
    x[np.abs(x) <= 1e-12] = 1e-12 + 0.0j
    h1 = _hankel2_1_array(x)
    projection = np.sum(np.asarray(n_src, dtype=float)[mask] * r_vec[mask], axis=
    out[mask] = (0.25j * complex(k0)) * h1 * projection
    return out

def _linear_pair_far_mask(
    elements: List[LinearElement],
    obs_index: int,
    centers: np.ndarray,
    lengths: np.ndarray,
    node_ids: np.ndarray,
    far_ratio: float,
) -> np.ndarray:
    obs_ids = node_ids[obs_index]
    shared = np.any(node_ids == obs_ids[0], axis=1) | np.any(node_ids == obs_ids[
    dist = np.linalg.norm(centers - centers[obs_index], axis=1)
    scale = np.maximum(np.maximum(lengths, lengths[obs_index]), EPS)
    far = (dist / scale) >= float(far_ratio)
    far[obs_index] = False
    far &= ~shared
    return far

def _assemble_linear_far_blocks_for_obs(
    obs_elem: LinearElement,
    src_elems: List[LinearElement],
```

```
    k0: complex | float,
    obs_normal_deriv: bool,
    obs_order: int,
    src_order: int,
) -> Tuple[np.ndarray, np.ndarray]:
    """Vectorised far-pair 2x2 block assembly for one observation element."""
    m = len(src_elems)
    if m == 0:
        return (
            np.zeros((0, 2, 2), dtype=np.complex128),
            np.zeros((0, 2, 2), dtype=np.complex128),
        )

    qt_obs, qw_obs = _get_quadrature(max(2, int(obs_order)))
    qt_src, qw_src = _get_quadrature(max(2, int(src_order)))
    obs_seg = obs_elem.p1 - obs_elem.p0
    src_p0 = np.stack([e.p0 for e in src_elems], axis=0)
    src_seg = np.stack([e.p1 - e.p0 for e in src_elems], axis=0)
    src_normals = np.stack([e.normal for e in src_elems], axis=0)
    src_lengths = np.asarray([e.length for e in src_elems], dtype=float)
    s_blocks = np.zeros((m, 2, 2), dtype=np.complex128)
    k_blocks = np.zeros((m, 2, 2), dtype=np.complex128)

    for tobs, wobs in zip(qt_obs, qw_obs):
        tobs_f = float(tobs)
        phi_obs = _linear_shape_values(tobs_f)
        robs = obs_elem.p0 + tobs_f * obs_seg
        for tsrc, wsrc in zip(qt_src, qw_src):
            tsrc_f = float(tsrc)
            phi_src = _linear_shape_values(tsrc_f)
            rsrc = src_p0 + tsrc_f * src_seg
            diff = robs[None, :] - rsrc
            kval_s = _green_2d_array(k0, np.linalg.norm(diff, axis=1))
            if obs_normal_deriv:
                kval_k = _dgreen_dn_obs_array(k0, diff, obs_elem.normal)
            else:
                kval_k = _dgreen_dn_src_array(k0, diff, src_normals)
            outer = np.outer(phi_obs, phi_src)[None, :, :]
            w = float(wobs) * float(wsrc)
            s_blocks += w * kval_s[:, None, None] * outer
            k_blocks += w * kval_k[:, None, None] * outer

    scale = float(obs_elem.length) * src_lengths[:, None, None]
    s_blocks *= scale
    k_blocks *= scale
    return s_blocks, k_blocks
```

```
def _single_layer_block_linear(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    k0: complex | float,
    obs_order: int = 8,
    src_order: int = 8,
) -> np.ndarray:
    return _integrate_linear_pair_generic(
        obs_elem,
        src_elem,
        lambda robs, rsrc: _green_2d(k0, max(float(np.linalg.norm(robs - rsrc)),
        obs_order=obs_order,
        src_order=src_order,
    )

def _double_layer_block_linear(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    k0: complex | float,
    obs_normal_deriv: bool,
    obs_order: int = 8,
    src_order: int = 8,
) -> np.ndarray:
    if obs_normal_deriv:
        return _integrate_linear_pair_generic(
            obs_elem,
            src_elem,
            lambda robs, rsrc: _dgreen_dn_obs(k0, robs - rsrc, obs_elem.normal),
            obs_order=obs_order,
            src_order=src_order,
        )
    return _integrate_linear_pair_generic(
        obs_elem,
        src_elem,
        lambda robs, rsrc: _dgreen_dn_src(k0, robs - rsrc, src_elem.normal),
        obs_order=obs_order,
        src_order=src_order,
    )

def _sk_blocks_near_linear(
    obs_elem: LinearElement,
    src_elem: LinearElement,
    k0: complex | float,
    obs_normal_deriv: bool,
    obs_order: int = 8,
    src_order: int = 8,
) -> Tuple[np.ndarray, np.ndarray]:
```

```
    """
    Compute S and K 2×2 blocks for a near element pair.

    Uses Duffy transforms for self and touching pairs (via the existing recursive
    path), and the vectorized tensor-Gauss path for separated-near pairs.
    """

    same_elem = obs_elem.panel_index == src_elem.panel_index
    node_ids_obs = set(obs_elem.node_ids)
    node_ids_src = set(src_elem.node_ids)
    is_touching = bool(node_ids_obs & node_ids_src) and not same_elem

    if same_elem or is_touching:
        # Self and touching pairs need Duffy — fall back to scalar recursive path
        s_blk = _single_layer_block_linear(obs_elem, src_elem, k0, obs_order, src
        k_blk = _double_layer_block_linear(obs_elem, src_elem, k0, obs_normal_der
        return s_blk, k_blk

    # Separated-near pairs: use vectorized box integrator with adaptive order.
    obs_mid = obs_elem.center
    src_mid = src_elem.center
    distance = float(np.linalg.norm(obs_mid - src_mid))
    scale = max(obs_elem.length, src_elem.length, EPS)
    adapt_order, _ = _near_singular_scheme(distance, scale)
    tensor_order = max(int(max(obs_order, src_order)), min(16, int(max(5, adapt_o

    return _integrate_linear_pair_box_sk_vectorized(
        obs_elem, src_elem, k0, obs_normal_deriv,
        obs_interval=(0.0, 1.0), src_interval=(0.0, 1.0),
        obs_order=tensor_order, src_order=tensor_order,
    )

_TANGENT_OUTER = np.array([[1.0, -1.0], [-1.0, 1.0]], dtype=np.complex128)

def _hypersingular_block_from_s_block(
    s_block: np.ndarray,
    k0: complex | float,
    n_obs: np.ndarray,
    n_src: np.ndarray,
    obs_length: float,
    src_length: float,
) -> np.ndarray:
    """
    Compute the 2x2 hypersingular D block from the single-layer S block via Maue

    The Maue regularisation recasts the hypersingular kernel integral as:
        D_ij = -k^2 (n_obs . n_src) S_ij
```

```
            + (1/(L_obs*L_src)) * tangent_outer_ij * sum(S_block)

    where tangent_outer = [[1,-1],[-1,1]] encodes the linear shape-function
    tangential derivatives.  This avoids all hypersingular quadrature.
    """

    k2 = complex(k0) ** 2
    n_dot_n = float(np.dot(n_obs, n_src))
    raw_integral = complex(np.sum(s_block))
    denom = max(float(obs_length) * float(src_length), EPS * EPS)
    return -k2 * n_dot_n * s_block + _TANGENT_OUTER * (raw_integral / denom)

def _assemble_linear_operator_matrices(
    mesh: LinearMesh,
    k0: complex | float,
    obs_normal_deriv: bool,
    obs_order: int = 8,
    src_order: int = 8,
    far_ratio: float = 3.0,
    source_element_mask: np.ndarray | None = None,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Assemble dense linear-Galerkin S and K/K' matrices on global nodal DOFs.

    Uses a two-pass approach:
    1. Far interactions: fully batched numpy operations (no Python element loop).
    2. Near interactions: per-element-pair recursive/Duffy quadrature.
    """

    nnodes = len(mesh.nodes)
    s_mat = np.zeros((nnodes, nnodes), dtype=np.complex128)
    k_mat = np.zeros((nnodes, nnodes), dtype=np.complex128)
    elements = list(mesh.elements)
    nelems = len(elements)
    if not elements:
        return s_mat, k_mat

    if source_element_mask is None:
        src_mask = np.ones(nelems, dtype=bool)
    else:
        src_mask = np.asarray(source_element_mask, dtype=bool).reshape(-1)
        if src_mask.size != nelems:
            raise ValueError("source_element_mask length must match mesh element
    if not np.any(src_mask):
        return s_mat, k_mat

    centers = np.stack([e.center for e in elements], axis=0)
```

```
lengths = np.asarray([e.length for e in elements], dtype=float)
node_ids = np.asarray([e.node_ids for e in elements], dtype=int)  # (nelems,
p0_arr = np.stack([e.p0 for e in elements], axis=0)               # (nelems,
seg_arr = np.stack([e.p1 - e.p0 for e in elements], axis=0)       # (nelems,
normals_arr = np.stack([e.normal for e in elements], axis=0)       # (nelems,

# --- Global far mask (nelems × nelems boolean) ---
dist_mat = np.linalg.norm(centers[:, None, :] - centers[None, :, :], axis=2)
scale_mat = np.maximum(lengths[:, None], lengths[None, :])
scale_mat = np.maximum(scale_mat, EPS)
far_mat = (dist_mat / scale_mat) >= float(far_ratio)

# Exclude self and shared-node pairs.
for i in range(nelems):
    far_mat[i, i] = False
shared = (node_ids[:, 0, None] == node_ids[None, :, 0]) | \
         (node_ids[:, 0, None] == node_ids[None, :, 1]) | \
         (node_ids[:, 1, None] == node_ids[None, :, 0]) | \
         (node_ids[:, 1, None] == node_ids[None, :, 1])
far_mat &= ~shared
far_mat &= src_mask[None, :]

# --- Pass 1: Fully batched far interactions ---
qt_obs, qw_obs = _get_quadrature(max(2, int(obs_order)))
qt_src, qw_src = _get_quadrature(max(2, int(src_order)))

phi_obs_arr = np.array([_linear_shape_values(float(t)) for t in qt_obs])  # (
phi_src_arr = np.array([_linear_shape_values(float(t)) for t in qt_src])  # (

# Precompute all quadrature points: (nelems, Q, 2).
t_obs_f = np.asarray(qt_obs, dtype=float)
t_src_f = np.asarray(qt_src, dtype=float)
all_obs_pts = p0_arr[:, None, :] + t_obs_f[None, :, None] * seg_arr[:, None,
all_src_pts = p0_arr[:, None, :] + t_src_f[None, :, None] * seg_arr[:, None,
len_scale = lengths[:, None] * lengths[None, :]

# 4 accumulators for each (obs_shape, src_shape) pair.
accum_s = [[np.zeros((nelems, nelems), dtype=np.complex128) for _ in range(2)
accum_k = [[np.zeros((nelems, nelems), dtype=np.complex128) for _ in range(2)

for qi in range(len(qt_obs)):
    r_obs = all_obs_pts[:, qi, :]   # (N, 2)
    w_obs_qi = float(qw_obs[qi])
    phi_o = phi_obs_arr[qi]          # (2,)

    for qj in range(len(qt_src)):
        r_src = all_src_pts[:, qj, :]
```

```
        w_src_qj = float(qw_src[qj])
        phi_s = phi_src_arr[qj]

        diff = r_obs[:, None, :] - r_src[None, :, :]  # (N, N, 2)
        dist = np.sqrt(diff[:, :, 0]**2 + diff[:, :, 1]**2)
        dist_safe = np.maximum(dist, EPS)
        x_flat = (complex(k0) * dist_safe).ravel()

        h0 = _hankel2_0_array(x_flat).reshape(nelems, nelems)
        h1 = _hankel2_1_array(x_flat).reshape(nelems, nelems)
        g = (0.25j * h0) * far_mat

        if obs_normal_deriv:
            proj = (diff[:, :, 0] * normals_arr[:, None, 0] +
                    diff[:, :, 1] * normals_arr[:, None, 1]) / dist_safe
            dk = ((-0.25j * complex(k0)) * h1 * proj) * far_mat
        else:
            proj = (normals_arr[None, :, 0] * diff[:, :, 0] +
                    normals_arr[None, :, 1] * diff[:, :, 1]) / dist_safe
            dk = ((0.25j * complex(k0)) * h1 * proj) * far_mat

        w = w_obs_qi * w_src_qj
        for a in range(2):
            coeff_a = w * float(phi_o[a])
            for b in range(2):
                c = coeff_a * float(phi_s[b])
                accum_s[a][b] += c * g
                accum_k[a][b] += c * dk

    # Distribute to global node matrix.
    for a in range(2):
        for b in range(2):
            scaled_s = accum_s[a][b] * len_scale
            scaled_k = accum_k[a][b] * len_scale
            np.add.at(s_mat, (node_ids[:, a, None], node_ids[None, :, b]), scaled
            np.add.at(k_mat, (node_ids[:, a, None], node_ids[None, :, b]), scaled

    # --- Pass 2: Near interactions (self, touching, close pairs) ---
    near_mat = (~far_mat) & src_mask[None, :]
    np.fill_diagonal(near_mat, src_mask)  # include self-interactions

    for obs_index in range(nelems):
        obs_elem = elements[obs_index]
        obs_ids = np.asarray(obs_elem.node_ids, dtype=int)
        near_idx = np.flatnonzero(near_mat[obs_index])
        for j in near_idx:
            src_elem = elements[int(j)]
```

```
            src_ids = src_elem.node_ids
            s_blk, k_blk = _sk_blocks_near_linear(
                obs_elem=obs_elem,
                src_elem=src_elem,
                k0=k0,
                obs_normal_deriv=obs_normal_deriv,
                obs_order=obs_order,
                src_order=src_order,
            )
            s_mat[np.ix_(obs_ids, src_ids)] += s_blk
            k_mat[np.ix_(obs_ids, src_ids)] += k_blk
    return s_mat, k_mat

def _assemble_linear_hypersingular_matrix(
    mesh: LinearMesh,
    k0: complex | float,
    obs_order: int = 8,
    src_order: int = 8,
    far_ratio: float = 3.0,
    source_element_mask: np.ndarray | None = None,
) -> np.ndarray:
    """
    Assemble the hypersingular D operator via the Maue identity.

    D is computed element-by-element from single-layer S blocks:
        D_block = -k^2 (n_obs . n_src) S_block
                + tangent_outer / (L_obs * L_src) * sum(S_block)

    This avoids all hypersingular quadrature; the log singularity in S is handled
    by the existing Duffy transforms.
    """

    nnodes = len(mesh.nodes)
    d_mat = np.zeros((nnodes, nnodes), dtype=np.complex128)
    elements = list(mesh.elements)
    if not elements:
        return d_mat

    if source_element_mask is None:
        src_mask = np.ones(len(elements), dtype=bool)
    else:
        src_mask = np.asarray(source_element_mask, dtype=bool).reshape(-1)
        if src_mask.size != len(elements):
            raise ValueError("source_element_mask length must match mesh element
    if not np.any(src_mask):
        return d_mat
```

```
    active_indices = np.flatnonzero(src_mask)
    for obs_elem in elements:
        obs_ids = np.asarray(obs_elem.node_ids, dtype=int)
        for j in active_indices:
            src_elem = elements[int(j)]
            src_ids = np.asarray(src_elem.node_ids, dtype=int)
            s_blk = _single_layer_block_linear(
                obs_elem=obs_elem,
                src_elem=src_elem,
                k0=k0,
                obs_order=obs_order,
                src_order=src_order,
            )
            d_blk = _hypersingular_block_from_s_block(
                s_blk, k0, obs_elem.normal, src_elem.normal,
                obs_elem.length, src_elem.length,
            )
            d_mat[np.ix_(obs_ids, src_ids)] += d_blk
    return d_mat

def _build_linear_coupled_infos(
    mesh: LinearMesh,
    materials: MaterialLibrary,
    freq_ghz: float,
    pol: str,
    k0: float,
) -> List[PanelCoupledInfo]:
    pseudo_panels = [
        Panel(
            name=e.name,
            seg_type=e.seg_type,
            ibc_flag=e.ibc_flag,
            ipn1=e.ipn1,
            ipn2=e.ipn2,
            p0=e.p0,
            p1=e.p1,
            center=e.center,
            tangent=e.tangent,
            normal=e.normal,
            length=e.length,
        )
        for e in mesh.elements
    ]
    return _build_coupled_panel_info(pseudo_panels, materials, freq_ghz, pol, k0)

def _linear_element_incident_load_many(
    elem: LinearElement,
```

```
    k_air: float,
    elevations_deg: np.ndarray,
    order: int = 8,
) -> np.ndarray:
    qt, qw = _get_quadrature(max(2, int(order)))
    seg = elem.p1 - elem.p0
    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)
    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    out = np.zeros((2, elev.size), dtype=np.complex128)
    for t, w in zip(qt, qw):
        shape = _linear_shape_values(float(t))[:, None]
        rp = elem.p0 + float(t) * seg
        phase = np.exp((1j * k_air) * (dirs @ rp))
        out += float(w) * shape * phase[None, :]
    return out * float(elem.length)

def _linear_element_incident_dn_load_many(
    elem: LinearElement,
    k_air: float,
    elevations_deg: np.ndarray,
    order: int = 8,
) -> np.ndarray:
    """
    Galerkin-tested normal derivative of the incident plane wave on one element.

    du_inc/dn = j*k*(d_inc . n) * exp(j*k*d_inc . r)

    Used by the Burton-Miller CFIE RHS correction.
    """

    qt, qw = _get_quadrature(max(2, int(order)))
    seg = elem.p1 - elem.p0
    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)
    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    # d_inc . n for each elevation angle
    d_dot_n = dirs @ np.asarray(elem.normal, dtype=float)  # shape (nelevations,)
    out = np.zeros((2, elev.size), dtype=np.complex128)
    for t, w in zip(qt, qw):
        shape = _linear_shape_values(float(t))[:, None]
        rp = elem.p0 + float(t) * seg
        phase = np.exp((1j * k_air) * (dirs @ rp))
        out += float(w) * shape * (1j * k_air * d_dot_n * phase)[None, :]
    return out * float(elem.length)

def _build_coupled_rhs_many_linear(
```

```
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    k_air: float,
    elevations_deg: np.ndarray,
    cfie_alpha: float = 0.0,
) -> np.ndarray:
    """
    Build tested incident-field load vectors on linear nodal DOFs.

    Returns an array of shape (2 * nnodes, E) corresponding to the future nodal
    unknown ordering [U_trace_nodes, Q_minus_nodes].

    When ``cfie_alpha > 0`` the Burton-Miller RHS correction is added:
        rhs += (j * cfie_alpha / k_air) * <phi, du_inc/dn>
    to the representation-formula rows.
    """

    nnodes = len(mesh.nodes)
    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)
    rhs = np.zeros((2 * nnodes, elev.size), dtype=np.complex128)
    use_cfie = float(cfie_alpha) > 0.0 and float(k_air) > EPS
    bm_eta = (1j * float(cfie_alpha) / float(k_air)) if use_cfie else 0.0

    for elem, info in zip(mesh.elements, infos):
        local = _linear_element_incident_load_many(elem, k_air=k_air, elevations_
        if use_cfie:
            local_dn = _linear_element_incident_dn_load_many(elem, k_air=k_air, e
        ids = elem.node_ids
        active_is_minus = info.minus_region >= 0
        if info.minus_has_incident if active_is_minus else info.plus_has_incident
            rhs[np.asarray(ids, dtype=int), :] += local
            if use_cfie:
                rhs[np.asarray(ids, dtype=int), :] += bm_eta * local_dn
        if info.bc_kind == "transmission":
            passive_has_inc = info.plus_has_incident if active_is_minus else info
            if passive_has_inc:
                rhs[nnodes + np.asarray(ids, dtype=int), :] += local
                if use_cfie:
                    rhs[nnodes + np.asarray(ids, dtype=int), :] += bm_eta * local
    return rhs

def _backscatter_rcs_coupled_many_linear(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    u_trace_nodes_mat: np.ndarray,
    q_minus_nodes_mat: np.ndarray,
    k_air: float,
```

```
    elevations_deg: np.ndarray,
    order: int = 8,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Linear-element far-field projector for the future coupled Galerkin solve.

    This helper already evaluates the backscatter integral from nodal traces/flux
    it is intended to be used once the linear coupled system assembly is wired in
    """

    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)
    u_eval = np.asarray(u_trace_nodes_mat, dtype=np.complex128)
    q_eval = np.asarray(q_minus_nodes_mat, dtype=np.complex128)
    if u_eval.ndim == 1:
        u_eval = u_eval.reshape(-1, 1)
    if q_eval.ndim == 1:
        q_eval = q_eval.reshape(-1, 1)
    nnodes = len(mesh.nodes)
    if u_eval.shape != q_eval.shape or u_eval.shape[0] != nnodes:
        raise ValueError("Linear nodal trace/flux arrays must have shape (nnodes,
    if u_eval.shape[1] != elev.size:
        raise ValueError("Linear nodal solution columns must match elevation coun

    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, int(order)))
    amp = np.zeros(elev.size, dtype=np.complex128)

    for elem, info in zip(mesh.elements, infos):
        ids = np.asarray(elem.node_ids, dtype=int)
        beta = complex(info.q_plus_beta)
        gamma = complex(info.q_plus_gamma)
        u_local = u_eval[ids, :]
        q_minus_local = q_eval[ids, :]
        q_plus_local = beta * q_minus_local + gamma * u_local
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))[:, None]
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp((1j * k_air) * (dirs @ rp))
            dot_scatter = dirs @ elem.normal
            u_t = np.sum(shape * u_local, axis=0)
            q_minus_t = np.sum(shape * q_minus_local, axis=0)
            q_plus_t = np.sum(shape * q_plus_local, axis=0)
            if info.minus_has_incident:
                amp += float(w) * float(elem.length) * phase * (
                    -q_minus_t + 1j * k_air * dot_scatter * u_t
                )
```

```
            if info.plus_has_incident:
                amp += float(w) * float(elem.length) * phase * (
                    q_plus_t - 1j * k_air * dot_scatter * u_t
                )

    sigma_lin = _rcs_sigma_from_amp(amp, k_air)
    return np.asarray(sigma_lin, dtype=float), np.asarray(amp, dtype=np.complex12

def _linear_mass_block(elem: LinearElement) -> np.ndarray:
    """Consistent 2-node boundary mass matrix on one straight element."""

    l = float(elem.length)
    return l * np.asarray([[1.0 / 3.0, 1.0 / 6.0], [1.0 / 6.0, 1.0 / 3.0]], dtype

def _linear_coupled_interface_signature(elem: LinearElement, info: PanelCoupledIn
    return (
        int(elem.seg_type),
        int(elem.ibc_flag),
        int(elem.ipn1),
        int(elem.ipn2),
        int(info.minus_region),
        int(info.plus_region),
        str(info.bc_kind),
    )

def _linear_coupled_node_report(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
) -> Dict[str, int]:
    """
    Summarize node configurations for the nodal coupled solve.

    The production-hardened linear/Galerkin path now handles shared geometric jun
    augmenting the nodal system with trace-continuity and region-wise flux-balanc
    We still report branching and mixed-interface node counts for diagnostics, bu
    no longer treated as automatic blockers by themselves.
    """

    incident: Dict[int, List[int]] = {}
    for eidx, elem in enumerate(mesh.elements):
        for nid in elem.node_ids:
            incident.setdefault(int(nid), []).append(int(eidx))

    branching_nodes = 0
    mixed_interface_nodes = 0
    for nid, elem_ids in incident.items():
        unique = sorted(set(int(v) for v in elem_ids))
```

```
        if len(unique) <= 1:
            continue
        sigs = {
            _linear_coupled_interface_signature(mesh.elements[eidx], infos[eidx])
            for eidx in unique
        }
        if len(unique) > 2:
            branching_nodes += 1
        if len(sigs) > 1:
            mixed_interface_nodes += 1

    return {
        "linear_node_count": int(len(mesh.nodes)),
        "linear_element_count": int(len(mesh.elements)),
        "linear_branching_nodes": int(branching_nodes),
        "linear_mixed_interface_nodes": int(mixed_interface_nodes),
        "linear_unsupported_nodes": 0,
    }

def _build_linear_junction_constraints(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
) -> Tuple[np.ndarray, Dict[str, int]]:
    """
    Build nodal junction constraints for the linear/Galerkin coupled solve.

    The linear trace unknown is continuous only across explicitly shared nodes. W
    interface-aware mesh intentionally splits nodes at the same geometric coordin
    restore pointwise continuity at true shared geometric junctions with explicit
    constraints. We also add region-wise flux-balance constraints using the endpo
    convention from the previous junction treatment.
    """

    nnodes = len(mesh.nodes)
    grouped: Dict[Tuple[int, int], List[Tuple[int, int, int]]] = {}
    for eidx, elem in enumerate(mesh.elements):
        n0, n1 = (int(v) for v in elem.node_ids)
        grouped.setdefault(mesh.nodes[n0].key, []).append((int(eidx), 0, n0))
        grouped.setdefault(mesh.nodes[n1].key, []).append((int(eidx), 1, n1))

    rows: List[np.ndarray] = []
    trace_count = 0
    flux_count = 0
    junction_nodes = 0
    orientation_conflict_nodes = 0
    constrained_nodes: Set[int] = set()
    constrained_elems: Set[int] = set()
```

```
for entries in grouped.values():
    unique_elems = sorted({int(eidx) for eidx, _, _ in entries})
    unique_nodes = sorted({int(nid) for _, _, nid in entries})
    if len(unique_elems) < 2 and len(unique_nodes) < 2:
        continue

    by_elem_sign: Dict[int, int] = {}
    seg_names: Set[str] = set()
    region_set: Set[int] = set()
    for eidx, local_end, nid in entries:
        endpoint_sign = +1 if int(local_end) == 0 else -1
        by_elem_sign[int(eidx)] = by_elem_sign.get(int(eidx), 0) + endpoint_s
        seg_names.add(mesh.elements[int(eidx)].name)
        info = infos[int(eidx)]
        if info.minus_region >= 0:
            region_set.add(int(info.minus_region))
        if info.plus_region >= 0:
            region_set.add(int(info.plus_region))

    if len(seg_names) >= 2:
        signs = [int(np.sign(by_elem_sign.get(eidx, 0))) for eidx in unique_e
        has_pos = any(s > 0 for s in signs)
        has_neg = any(s < 0 for s in signs)
        if not (has_pos and has_neg):
            orientation_conflict_nodes += 1

    if len(unique_nodes) > 1:
        ref_nid = unique_nodes[0]
        for other_nid in unique_nodes[1:]:
            row = np.zeros(2 * nnodes, dtype=np.complex128)
            row[ref_nid] = 1.0 + 0.0j
            row[other_nid] = -1.0 + 0.0j
            rows.append(row)
            trace_count += 1
            constrained_nodes.add(ref_nid)
            constrained_nodes.add(other_nid)

    for region in sorted(region_set):
        row = np.zeros(2 * nnodes, dtype=np.complex128)
        terms = 0
        for eidx, local_end, nid in entries:
            endpoint_sign = +1 if int(local_end) == 0 else -1
            info = infos[int(eidx)]
            coeff_u = 0.0 + 0.0j
            coeff_q = 0.0 + 0.0j
            participates = False
```

```
                if info.minus_region == region:
                    coeff_q += 1.0 + 0.0j
                    participates = True
                if info.plus_region == region:
                    coeff_u += complex(info.q_plus_gamma)
                    coeff_q += complex(info.q_plus_beta)
                    participates = True
                if not participates:
                    continue

                w = complex(float(endpoint_sign), 0.0)
                nid_i = int(nid)
                row[nid_i] += w * coeff_u
                row[nnodes + nid_i] += w * coeff_q
                terms += 1
                constrained_nodes.add(nid_i)
                constrained_elems.add(int(eidx))

            if terms >= 2 and np.linalg.norm(row) > 0.0:
                rows.append(row)
                flux_count += 1

        junction_nodes += 1

    if not rows:
        return np.zeros((0, 2 * nnodes), dtype=np.complex128), {
            "junction_nodes": 0,
            "junction_constraints": 0,
            "junction_panels": 0,
            "junction_trace_constraints": 0,
            "junction_flux_constraints": 0,
            "junction_orientation_conflict_nodes": int(orientation_conflict_nodes
        }

    c_mat = np.vstack(rows)
    return c_mat, {
        "junction_nodes": int(junction_nodes),
        "junction_constraints": int(c_mat.shape[0]),
        "junction_panels": int(len(constrained_elems)),
        "junction_trace_constraints": int(trace_count),
        "junction_flux_constraints": int(flux_count),
        "junction_orientation_conflict_nodes": int(orientation_conflict_nodes),
    }

def _ensure_finite_linear_system(a_mat: np.ndarray, rhs: np.ndarray | None = None
    """Raise a clear error before calling LAPACK if the assembled system contains
```

```
    a_eval = np.asarray(a_mat)
    if not np.all(np.isfinite(a_eval)):
        bad = np.argwhere(~np.isfinite(a_eval))
        first = tuple(int(v) for v in bad[0]) if bad.size else None
        raise ValueError(f"{label}: system matrix contains NaN/Inf at index {firs
    if rhs is None:
        return
    b_eval = np.asarray(rhs)
    if not np.all(np.isfinite(b_eval)):
        bad = np.argwhere(~np.isfinite(b_eval))
        first = tuple(int(v) for v in bad[0]) if bad.size else None
        raise ValueError(f"{label}: RHS contains NaN/Inf at index {first}.")

def _assemble_linear_mass_matrix(mesh: LinearMesh) -> np.ndarray:
    """Assemble the global consistent mass matrix for the linear boundary mesh."""

    nnodes = len(mesh.nodes)
    m_mat = np.zeros((nnodes, nnodes), dtype=np.complex128)
    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        m_mat[np.ix_(ids, ids)] += _linear_mass_block(elem)
    return m_mat

@dataclass
class LinearCoupledNodeInfo:
    """Per-node coupled metadata for the global linear/Galerkin assembly."""

    active_region: int
    passive_region: int
    bc_kind: str
    robin_impedance: complex
    coeff_u_active: complex
    coeff_q_active: complex
    eps_phys: complex
    mu_phys: complex
    k_phys: complex
    q_plus_beta: complex
    q_plus_gamma: complex
    plus_region: int

def _build_linear_coupled_node_infos(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
) -> List[LinearCoupledNodeInfo]:
    """
    Derive one consistent coupled-interface record per nodal test/unknown DOF.
```

```
The interface-aware linear mesh is expected to share a node only across eleme
the same physical interface signature. We still verify that the incident elem
on the metadata needed by the global nodal assembly.
"""

incident: Dict[int, List[int]] = {}
for eidx, elem in enumerate(mesh.elements):
    for nid in elem.node_ids:
        incident.setdefault(int(nid), []).append(int(eidx))

def _complex_close(a: complex, b: complex, tol: float = 1.0e-10) -> bool:
    return abs(complex(a) - complex(b)) <= tol * max(1.0, abs(complex(a)), ab

node_infos: List[LinearCoupledNodeInfo | None] = [None] * len(mesh.nodes)
for nid in range(len(mesh.nodes)):
    elem_ids = incident.get(int(nid), [])
    if not elem_ids:
        raise ValueError(f"Linear coupled node {nid} is not attached to any e
    ref = infos[int(elem_ids[0])]
    active_region = int(ref.minus_region if ref.minus_region >= 0 else ref.pl
    passive_region = int(ref.plus_region if active_region == ref.minus_region
    coeff_u_active, coeff_q_active = _region_side_trace_coefficients(ref, act
    eps_phys = ref.eps_minus if active_region == ref.minus_region else ref.ep
    mu_phys = ref.mu_minus if active_region == ref.minus_region else ref.mu_p
    k_phys = ref.k_minus if active_region == ref.minus_region else ref.k_plus
    expected = {
        'active_region': active_region,
        'passive_region': passive_region,
        'bc_kind': str(ref.bc_kind),
        'robin_impedance': complex(ref.robin_impedance),
        'coeff_u_active': complex(coeff_u_active),
        'coeff_q_active': complex(coeff_q_active),
        'eps_phys': complex(eps_phys),
        'mu_phys': complex(mu_phys),
        'k_phys': complex(k_phys),
        'q_plus_beta': complex(ref.q_plus_beta),
        'q_plus_gamma': complex(ref.q_plus_gamma),
        'plus_region': int(ref.plus_region),
    }
    for eidx in elem_ids[1:]:
        info = infos[int(eidx)]
        active_chk = int(info.minus_region if info.minus_region >= 0 else inf
        passive_chk = int(info.plus_region if active_chk == info.minus_region
        coeff_u_chk, coeff_q_chk = _region_side_trace_coefficients(info, acti
        eps_chk = info.eps_minus if active_chk == info.minus_region else info
        mu_chk = info.mu_minus if active_chk == info.minus_region else info.m
        k_chk = info.k_minus if active_chk == info.minus_region else info.k_p
```

```
            actual = {
                'active_region': active_chk,
                'passive_region': passive_chk,
                'bc_kind': str(info.bc_kind),
                'robin_impedance': complex(info.robin_impedance),
                'coeff_u_active': complex(coeff_u_chk),
                'coeff_q_active': complex(coeff_q_chk),
                'eps_phys': complex(eps_chk),
                'mu_phys': complex(mu_chk),
                'k_phys': complex(k_chk),
                'q_plus_beta': complex(info.q_plus_beta),
                'q_plus_gamma': complex(info.q_plus_gamma),
                'plus_region': int(info.plus_region),
            }
            if actual['active_region'] != expected['active_region'] or actual['pa
                raise ValueError(
                    "Linear/Galerkin nodal assembly encountered incompatible inci
                    f"at node {nid}."
                )
            for key in ('robin_impedance', 'coeff_u_active', 'coeff_q_active', 'e
                if not _complex_close(expected[key], actual[key]):
                    raise ValueError(
                        "Linear/Galerkin nodal assembly encountered inconsistent
                        f"at node {nid}."
                    )

        node_infos[nid] = LinearCoupledNodeInfo(**expected)

    return [ni for ni in node_infos if ni is not None]

def _build_linear_coupled_region_operators(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    obs_order: int = 5,
    src_order: int = 5,
    far_ratio: float = 3.0,
    compute_cfie: bool = False,
) -> Dict[int, Dict[str, Any]]:
    """
    Assemble reusable nodal S/K operators for each region and interface side.

    Returns `region_ops[region]['minus'|'plus'] = (S, K)` where the matrices alre
    include only source elements whose minus/plus side belongs to the requested r

    When `compute_cfie` is True, also assembles K' (adjoint double layer) and D
    (hypersingular via Maue identity) for Burton-Miller CFIE.
    """
```

```
region_to_k: Dict[int, complex] = {}
for info in infos:
    if info.minus_region >= 0:
        region_to_k[int(info.minus_region)] = complex(info.k_minus)
    if info.plus_region >= 0:
        region_to_k[int(info.plus_region)] = complex(info.k_plus)

nelems = len(mesh.elements)
region_ops: Dict[int, Dict[str, Any]] = {}
for region, k_region in region_to_k.items():
    k_eval = k_region if abs(k_region) > EPS else (EPS + 0.0j)
    minus_mask = np.fromiter((info.minus_region == region for info in infos),
    plus_mask = np.fromiter((info.plus_region == region for info in infos), d
    entry: Dict[str, Any] = {
        'minus': _assemble_linear_operator_matrices(
            mesh=mesh,
            k0=k_eval,
            obs_normal_deriv=False,
            obs_order=obs_order,
            src_order=src_order,
            far_ratio=far_ratio,
            source_element_mask=minus_mask,
        ),
        'plus': _assemble_linear_operator_matrices(
            mesh=mesh,
            k0=k_eval,
            obs_normal_deriv=False,
            obs_order=obs_order,
            src_order=src_order,
            far_ratio=far_ratio,
            source_element_mask=plus_mask,
        ),
    }
    if compute_cfie:
        # K' (adjoint double layer): obs_normal_deriv=True
        entry['kp_minus'] = _assemble_linear_operator_matrices(
            mesh=mesh,
            k0=k_eval,
            obs_normal_deriv=True,
            obs_order=obs_order,
            src_order=src_order,
            far_ratio=far_ratio,
            source_element_mask=minus_mask,
        )
        entry['kp_plus'] = _assemble_linear_operator_matrices(
            mesh=mesh,
```

```
                k0=k_eval,
                obs_normal_deriv=True,
                obs_order=obs_order,
                src_order=src_order,
                far_ratio=far_ratio,
                source_element_mask=plus_mask,
            )
            # D (hypersingular via Maue identity)
            entry['d_minus'] = _assemble_linear_hypersingular_matrix(
                mesh=mesh,
                k0=k_eval,
                obs_order=obs_order,
                src_order=src_order,
                far_ratio=far_ratio,
                source_element_mask=minus_mask,
            )
            entry['d_plus'] = _assemble_linear_hypersingular_matrix(
                mesh=mesh,
                k0=k_eval,
                obs_order=obs_order,
                src_order=src_order,
                far_ratio=far_ratio,
                source_element_mask=plus_mask,
            )
        region_ops[int(region)] = entry
    return region_ops

def _build_coupled_matrix_linear(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    pol: str,
    obs_order: int = 5,
    src_order: int = 5,
    cfie_alpha: float = 0.0,
    k_air: float = 0.0,
) -> np.ndarray:
    """
    Assemble the nodal linear/Galerkin coupled matrix with optional Burton-Miller

    Unknown ordering is [U_trace_nodes, Q_minus_nodes].  When ``cfie_alpha > 0``,
    the Burton-Miller regularisation is applied to the representation-formula row
    suppressing interior-resonance artefacts.  The coupling parameter is

        eta_bm = j * cfie_alpha / k_air

    which follows the Kress (1985) convention for 2-D Helmholtz.
    """
```

```
nnodes = len(mesh.nodes)
a_mat = np.zeros((2 * nnodes, 2 * nnodes), dtype=np.complex128)
if nnodes == 0:
    return a_mat

use_cfie = float(cfie_alpha) > 0.0 and float(k_air) > EPS
bm_eta = (1j * float(cfie_alpha) / float(k_air)) if use_cfie else 0.0

node_infos = _build_linear_coupled_node_infos(mesh, infos)
mass_mat = _assemble_linear_mass_matrix(mesh)
region_ops = _build_linear_coupled_region_operators(
    mesh=mesh,
    infos=infos,
    obs_order=obs_order,
    src_order=src_order,
    compute_cfie=use_cfie,
)

node_ids = np.arange(nnodes, dtype=int)
q_plus_beta = np.asarray([ni.q_plus_beta for ni in node_infos], dtype=np.comp
q_plus_gamma = np.asarray([ni.q_plus_gamma for ni in node_infos], dtype=np.co
active_regions = np.asarray([ni.active_region for ni in node_infos], dtype=in
passive_regions = np.asarray([ni.passive_region for ni in node_infos], dtype=
bc_kinds = np.asarray([ni.bc_kind for ni in node_infos], dtype=object)
robin_impedance = np.asarray([ni.robin_impedance for ni in node_infos], dtype
coeff_u_active = np.asarray([ni.coeff_u_active for ni in node_infos], dtype=n
coeff_q_active = np.asarray([ni.coeff_q_active for ni in node_infos], dtype=n
eps_phys = np.asarray([ni.eps_phys for ni in node_infos], dtype=np.complex128
mu_phys = np.asarray([ni.mu_phys for ni in node_infos], dtype=np.complex128)
k_phys = np.asarray([ni.k_phys for ni in node_infos], dtype=np.complex128)

def _apply_region_rows(rows: np.ndarray, region: int, row_offset: int) -> Non
    if rows.size == 0:
        return
    ops = region_ops.get(int(region))
    if ops is None:
        raise ValueError(f"Missing linear/Galerkin region operators for regio
    s_minus, k_minus = ops['minus']
    s_plus, k_plus = ops['plus']
    # --- Standard EFIE rows ---
    a_mat[np.ix_(row_offset + rows, node_ids)] += (
        0.5 * mass_mat[np.ix_(rows, node_ids)]
        + k_minus[np.ix_(rows, node_ids)]
        - k_plus[np.ix_(rows, node_ids)]
        + s_plus[np.ix_(rows, node_ids)] * q_plus_gamma[None, :]
    )
```

```
    a_mat[np.ix_(row_offset + rows, nnodes + node_ids)] += (
        -s_minus[np.ix_(rows, node_ids)]
        + s_plus[np.ix_(rows, node_ids)] * q_plus_beta[None, :]
    )

    # --- Burton-Miller CFIE correction ---
    if use_cfie and 'd_minus' in ops:
        d_minus = ops['d_minus']
        d_plus = ops['d_plus']
        _s_kp_minus, kp_minus = ops['kp_minus']
        _s_kp_plus, kp_plus = ops['kp_plus']

        # u-block BM: eta*(D_minus - D_plus + K'_plus*gamma - 0.5*M*gamma)
        a_mat[np.ix_(row_offset + rows, node_ids)] += bm_eta * (
            d_minus[np.ix_(rows, node_ids)]
            - d_plus[np.ix_(rows, node_ids)]
            + kp_plus[np.ix_(rows, node_ids)] * q_plus_gamma[None, :]
            - 0.5 * mass_mat[np.ix_(rows, node_ids)] * q_plus_gamma[rows][:,
        )
        # q-block BM: eta*(M*(1-0.5*beta) - K'_minus + K'_plus*beta)
        a_mat[np.ix_(row_offset + rows, nnodes + node_ids)] += bm_eta * (
            mass_mat[np.ix_(rows, node_ids)] * (1.0 - 0.5 * q_plus_beta[rows]
            - kp_minus[np.ix_(rows, node_ids)]
            + kp_plus[np.ix_(rows, node_ids)] * q_plus_beta[None, :]
        )

for region in sorted(set(int(v) for v in active_regions)):
    rows = node_ids[active_regions == int(region)]
    _apply_region_rows(rows, int(region), row_offset=0)

transmission_nodes = node_ids[bc_kinds == 'transmission']
if transmission_nodes.size > 0:
    transmission_passive = passive_regions[transmission_nodes]
    for region in sorted(set(int(v) for v in transmission_passive if int(v) >
        rows = transmission_nodes[transmission_passive == int(region)]
        _apply_region_rows(rows, int(region), row_offset=nnodes)

bc_nodes = node_ids[bc_kinds != 'transmission']
if bc_nodes.size > 0:
    zero_z = np.abs(robin_impedance[bc_nodes]) <= EPS
    pec_nodes = bc_nodes[zero_z]
    if pec_nodes.size > 0:
        if pol == 'TM':
            a_mat[np.ix_(nnodes + pec_nodes, node_ids)] += mass_mat[np.ix_(pe
        else:
            a_mat[np.ix_(nnodes + pec_nodes, node_ids)] += (
                mass_mat[np.ix_(pec_nodes, node_ids)] * coeff_u_active[pec_no
```

```
                )
                a_mat[np.ix_(nnodes + pec_nodes, nnodes + node_ids)] += (
                    mass_mat[np.ix_(pec_nodes, node_ids)] * coeff_q_active[pec_no
                )

        robin_nodes = bc_nodes[~zero_z]
        if robin_nodes.size > 0:
            alpha = np.asarray([
                _surface_robin_alpha(pol, eps_phys[i], mu_phys[i], k_phys[i], rob
                for i in robin_nodes
            ], dtype=np.complex128)
            a_mat[np.ix_(nnodes + robin_nodes, node_ids)] += (
                mass_mat[np.ix_(robin_nodes, node_ids)] * (coeff_u_active[robin_n
            )
            a_mat[np.ix_(nnodes + robin_nodes, nnodes + node_ids)] += (
                mass_mat[np.ix_(robin_nodes, node_ids)] * coeff_q_active[robin_no
            )

    return a_mat

def prepare_linear_galerkin_system(
    geometry_snapshot: Dict[str, Any],
    frequency_ghz: float,
    polarization: str,
    geometry_units: str = "inches",
    material_base_dir: str | None = None,
    max_panels: int = MAX_PANELS_DEFAULT,
    mesh_reference_ghz: float | None = None,
    node_snap_tol: float = 1.0e-9,
    obs_order: int = 8,
    src_order: int = 8,
) -> Dict[str, Any]:
    """
    Build the reusable linear-Galerkin coupled system for one frequency.

    The helper validates the geometry, builds boundary primitives, promotes them
    continuous two-node linear mesh, derives per-element coupled material data, a
    assembles dense nodal S/K region operators.

    It returns reusable nodal operators and metadata for external scripts.
    """

    freq_ghz = float(frequency_ghz)
    if (not math.isfinite(freq_ghz)) or freq_ghz <= 0.0:
        raise ValueError("frequency_ghz must be a positive finite value.")
    pol = _normalize_polarization(polarization)
    unit_scale = _unit_scale_to_meters(geometry_units)
```

```
base_dir = material_base_dir or os.getcwd()
mesh_freq_ghz = float(mesh_reference_ghz) if mesh_reference_ghz is not None e
if (not math.isfinite(mesh_freq_ghz)) or mesh_freq_ghz <= 0.0:
    raise ValueError("mesh_reference_ghz must be a positive finite GHz value
lambda_min = C0 / (mesh_freq_ghz * 1e9)
preflight = validate_geometry_snapshot_for_solver(geometry_snapshot, base_dir
panels = _build_panels(
    geometry_snapshot=geometry_snapshot,
    meters_scale=unit_scale,
    min_wavelength=lambda_min,
    max_panels=max_panels,
)
materials = MaterialLibrary.from_entries(
    geometry_snapshot.get("ibcs", []) or [],
    geometry_snapshot.get("dielectrics", []) or [],
    base_dir=base_dir,
)
for _msg in list(preflight.get('warnings', []) or []):
    materials.warn_once(str(_msg))

mesh = _build_linear_mesh(panels, node_snap_tol=node_snap_tol)
k0 = 2.0 * math.pi * (freq_ghz * 1e9) / C0
infos = _build_linear_coupled_infos(mesh, materials, freq_ghz=freq_ghz, pol=p

region_to_k: Dict[int, complex] = {}
for info in infos:
    if info.minus_region >= 0:
        region_to_k[info.minus_region] = complex(info.k_minus)
    if info.plus_region >= 0:
        region_to_k[info.plus_region] = complex(info.k_plus)

region_ops: Dict[int, Tuple[np.ndarray, np.ndarray]] = {}
cache: Dict[Tuple[float, float, bool], Tuple[np.ndarray, np.ndarray]] = {}
for region, k_region in region_to_k.items():
    key = (round(float(np.real(k_region)), 12), round(float(np.imag(k_region)
    if key not in cache:
        cache[key] = _assemble_linear_operator_matrices(
            mesh=mesh,
            k0=k_region if abs(k_region) > EPS else (EPS + 0.0j),
            obs_normal_deriv=False,
            obs_order=obs_order,
            src_order=src_order,
        )
    region_ops[region] = cache[key]

return {
    "panels": panels,
```

```
        "mesh": mesh,
        "materials": materials,
        "infos": infos,
        "region_ops": region_ops,
        "metadata": {
            "frequency_ghz": float(freq_ghz),
            "mesh_reference_ghz": float(mesh_freq_ghz),
            "polarization_internal": pol,
            "panel_count": len(panels),
            "linear_element_count": len(mesh.elements),
            "linear_node_count": len(mesh.nodes),
            "node_snap_tol_m": float(node_snap_tol),
            "obs_order": int(obs_order),
            "src_order": int(src_order),
            "warnings": list(materials.warnings),
            "preflight": dict(preflight),
            "status": "stage1-system",
        },
    }


def _medium_eta(eps: complex, mu: complex) -> complex:
    eps = _normalize_material_value(eps, 1.0 + 0.0j)
    mu = _normalize_material_value(mu, 1.0 + 0.0j)
    return ETA0 * cmath.sqrt(mu / eps)

def _medium_n(eps: complex, mu: complex) -> complex:
    eps = _normalize_material_value(eps, 1.0 + 0.0j)
    mu = _normalize_material_value(mu, 1.0 + 0.0j)
    return cmath.sqrt(eps * mu)

def _safe_complex_div(num: complex, den: complex, fallback: complex) -> complex:
    if abs(den) <= EPS:
        return fallback
    return num / den

def _snell_cos_t(eps1: complex, mu1: complex, eps2: complex, mu2: complex, cos_i:
    c_i = max(0.0, min(1.0, float(abs(cos_i))))
    s_i2 = max(0.0, 1.0 - c_i * c_i)
    n1 = _medium_n(eps1, mu1)
    n2 = _medium_n(eps2, mu2)
    if abs(n2) <= EPS:
        n2 = 1.0 + 0.0j
    s_t2 = (n1 / n2) ** 2 * s_i2
    return cmath.sqrt(1.0 - s_t2)

def _projected_impedance(eps: complex, mu: complex, cos_theta: complex, pol: str)
```

```
    eta = _medium_eta(eps, mu)
    if pol == "TE":
        return _safe_complex_div(eta, cos_theta, eta)
    return eta * cos_theta

def _parallel_impedance(z1: complex, z2: complex) -> complex:
    if abs(z1) <= EPS:
        return z2
    if abs(z2) <= EPS:
        return z1
    return _safe_complex_div(z1 * z2, z1 + z2, z1)

def _region_medium(materials: MaterialLibrary, region_flag: int, freq_ghz: float)
    if region_flag <= 0:
        return 1.0 + 0.0j, 1.0 + 0.0j
    return materials.get_medium(region_flag, freq_ghz)

def _causal_medium_index(eps: complex, mu: complex) -> complex:
    """
    Choose refractive-index branch consistent with passive media in e^{-jwt}.

    Enforces a consistent sign choice so attenuation is physical.
    """

    n = _medium_n(eps, mu)
    if n.real < 0.0:
        n = -n
    # e^{-j omega t} convention prefers Im(n) <= 0 for passive attenuation.
    if n.imag > 0.0:
        n = -n
    if abs(n) <= EPS:
        return 1.0 + 0.0j
    return n

def _medium_wavenumber(
    k0: float,
    eps: complex,
    mu: complex,
) -> complex:
    """Complex medium wavenumber used directly inside integral kernels."""

    return complex(k0) * _causal_medium_index(eps, mu)

def _impedance_to_admittance(z_value: complex) -> complex:
    z_eval = _ensure_finite_complex(z_value, "Surface impedance")
    if abs(z_eval) <= EPS:
        return 0.0 + 0.0j
```

```
    return 1.0 / z_eval

def _surface_robin_alpha(
    pol: str,
    eps_medium: complex,
    mu_medium: complex,
    k_medium: complex,
    z_surface: complex,
) -> complex:
    """
    Return the scalar Robin coefficient alpha for q + alpha*u = 0.

    Physical SIBC boundary conditions for 2D scalar wave equation:

    TE (E_z, Dirichlet-like for PEC):
      E_z + Z_s * H_phi = 0
      → du/dn + j*k*eta/Z_s * u = 0
      → alpha = j * k * eta / Z_s
      Limits: Z_s→0 → alpha→∞ (u=0, PEC TE)
              Z_s→∞ → alpha→0 (q=0, PMC TE)

    TM (H_z, Neumann-like for PEC):
      H_z - (1/Z_s) * E_phi = 0
      → du/dn - j*k*Z_s/eta * u = 0
      → alpha = -j * k * Z_s / eta
      Limits: Z_s→0 → alpha→0 (q=0, PEC TM)
              Z_s→∞ → alpha→-∞ (u=0, PMC TM)
    """

    if abs(z_surface) <= EPS:
        return 0.0 + 0.0j
    eta_medium = _medium_eta(eps_medium, mu_medium)
    if pol == "TE":
        return 1j * complex(k_medium) * _safe_complex_div(eta_medium, z_surface,
    return -1j * complex(k_medium) * _safe_complex_div(z_surface, eta_medium, 0.0

def _region_side_trace_coefficients(info: PanelCoupledInfo, region_flag: int) ->
    """
    Map a region-side normal derivative to [u_trace, q_minus] coefficients.

    Returns (coeff_u, coeff_q) such that:
        q_region = coeff_u * u_trace + coeff_q * q_minus
    """

    if info.minus_region == region_flag:
        return 0.0 + 0.0j, 1.0 + 0.0j
    if info.plus_region == region_flag:
```

```
        return complex(info.q_plus_gamma), complex(info.q_plus_beta)
    raise ValueError("Requested region does not participate in this panel.")

def _q_plus_beta(
    pol: str,
    eps_minus: complex,
    mu_minus: complex,
    eps_plus: complex,
    mu_plus: complex,
) -> complex:
    """
    Scaling between minus-side and plus-side raw normal derivatives across interf

    Branch semantics with direct TE/TM labeling:
    - TE uses the (1/eps) * du/dn continuity branch.
    - TM uses the (1/mu) * du/dn continuity branch.
    """

    if pol == "TM":
        return _safe_complex_div(mu_plus, mu_minus, 1.0 + 0.0j)
    return _safe_complex_div(eps_plus, eps_minus, 1.0 + 0.0j)

def _panel_effective_impedance(
    panel: Panel,
    materials: MaterialLibrary,
    freq_ghz: float,
    pol: str,
    cos_inc: float,
) -> complex:
    """
    Return the effective local impedance associated with one boundary primitive.

    This helper is retained for local surface-impedance calculations used by the
    current formulation and related post-processing utilities.
    """

    if panel.seg_type == 1:
        z_card = materials.get_impedance(panel.ibc_flag, freq_ghz)
        return z_card

    if panel.seg_type == 2:
        if panel.ibc_flag > 0:
            return materials.get_impedance(panel.ibc_flag, freq_ghz)
        return 0.0 + 0.0j

    if panel.seg_type == 3:
        eps2, mu2 = materials.get_medium(panel.ipn1, freq_ghz)
```

```
        cos_t = _snell_cos_t(1.0 + 0.0j, 1.0 + 0.0j, eps2, mu2, cos_inc)
        z_int = _projected_impedance(eps2, mu2, cos_t, pol)
        if panel.ibc_flag > 0:
            z_card = materials.get_impedance(panel.ibc_flag, freq_ghz)
            return z_int + z_card
        return z_int

    if panel.seg_type == 4:
        if panel.ibc_flag > 0:
            return materials.get_impedance(panel.ibc_flag, freq_ghz)
        return 0.0 + 0.0j

    if panel.seg_type == 5:
        eps1, mu1 = materials.get_medium(panel.ipn1, freq_ghz)
        eps2, mu2 = materials.get_medium(panel.ipn2, freq_ghz)
        cos_i = complex(max(1e-6, min(1.0, abs(cos_inc))), 0.0)
        cos_t = _snell_cos_t(eps1, mu1, eps2, mu2, float(abs(cos_inc)))
        z1 = _projected_impedance(eps1, mu1, cos_i, pol)
        z2 = _projected_impedance(eps2, mu2, cos_t, pol)
        z_if = _parallel_impedance(z1, z2)
        if panel.ibc_flag > 0:
            z_card = materials.get_impedance(panel.ibc_flag, freq_ghz)
            return z_if + z_card
        return z_if

    if panel.ibc_flag > 0:
        return materials.get_impedance(panel.ibc_flag, freq_ghz)
    return 0.0 + 0.0j

def _build_coupled_panel_info(
    panels: List[Panel],
    materials: MaterialLibrary,
    freq_ghz: float,
    pol: str,
    k0: float,
) -> List[PanelCoupledInfo]:
    """
    Translate geometry TYPE/IBC/IPN flags into coupled interface algebra per pane

    Project convention:
    - the drawn panel normal points toward the IPN1 side,
    - TYPE 3: plus/IPN1 = dielectric, minus = air,
    - TYPE 5: plus/IPN1, minus/IPN2,
    - TYPE 4: plus/IPN1 = dielectric, minus = PEC/IBC side.

    The coupled assembly is allowed to use whichever side is the valid non-PEC si
    so TYPE 4 remains solvable even though the PEC side is the minus side.
```

```
    """

    infos: List[PanelCoupledInfo] = []
    sheet_region_by_name: Dict[str, int] = {}
    next_sheet_region = 900_000

    for panel in panels:
        seg_type = panel.seg_type
        if seg_type == 3:
            if panel.ipn1 <= 0:
                raise ValueError(f"TYPE 3 panel '{panel.name}' requires IPN1 > 0.
            plus_region = panel.ipn1
            minus_region = 0
            bc_kind = "transmission"
            plus_has_incident = False
            minus_has_incident = True
        elif seg_type == 5:
            if panel.ipn1 <= 0 or panel.ipn2 <= 0:
                raise ValueError(f"TYPE 5 panel '{panel.name}' requires IPN1 > 0
            plus_region = panel.ipn1
            minus_region = panel.ipn2
            bc_kind = "transmission"
            plus_has_incident = False
            minus_has_incident = False
        elif seg_type == 4:
            if panel.ipn1 <= 0:
                raise ValueError(f"TYPE 4 panel '{panel.name}' requires IPN1 > 0.
            plus_region = panel.ipn1
            minus_region = -1
            bc_kind = "robin"
            plus_has_incident = False
            minus_has_incident = False
        elif seg_type == 2:
            minus_region = 0
            plus_region = -1
            bc_kind = "robin"
            minus_has_incident = True
            plus_has_incident = False
        elif seg_type == 1:
            if panel.ibc_flag <= 0:
                raise ValueError(
                    f"TYPE 1 panel '{panel.name}' requires IBC > 0 in coupled die
                )
            sheet_name = panel.name.strip() or "__type1_sheet__"
            sheet_region = sheet_region_by_name.get(sheet_name)
            if sheet_region is None:
                sheet_region = next_sheet_region
```

```
        sheet_region_by_name[sheet_name] = sheet_region
        next_sheet_region += 1
    minus_region = 0
    plus_region = sheet_region
    bc_kind = "transmission"
    minus_has_incident = True
    plus_has_incident = False
else:
    minus_region = 0
    plus_region = -1
    bc_kind = "robin"
    minus_has_incident = True
    plus_has_incident = False

eps_minus, mu_minus = _region_medium(materials, minus_region, freq_ghz)
eps_plus, mu_plus = _region_medium(materials, plus_region, freq_ghz)
k_minus = _medium_wavenumber(k0, eps_minus, mu_minus)
k_plus = _medium_wavenumber(k0, eps_plus, mu_plus)
if (
    abs(k_minus.imag) > 1e-10 or abs(k_plus.imag) > 1e-10
) and _complex_hankel_backend_name() == "native-series-asymptotic":
    raise RuntimeError(
        "Lossy dielectric media require SciPy or mpmath for trustworthy c
        "Install one of those backends before running production dielectr
    )

z_card = materials.get_impedance(panel.ibc_flag, freq_ghz) if panel.ibc_f
if bc_kind == "transmission":
    if seg_type == 1:
        if abs(z_card) <= EPS:
            raise ValueError(
                f"TYPE 1 panel '{panel.name}' has zero impedance; provide
            )
        q_plus_beta = -1.0 + 0.0j
        q_plus_gamma = _impedance_to_admittance(z_card)
    else:
        q_plus_beta = _q_plus_beta(pol, eps_minus, mu_minus, eps_plus, mu
        q_plus_gamma = _impedance_to_admittance(z_card)
else:
    q_plus_beta = _q_plus_beta(pol, eps_minus, mu_minus, eps_plus, mu_plu
    q_plus_gamma = 0.0 + 0.0j

infos.append(
    PanelCoupledInfo(
        seg_type=seg_type,
        plus_region=plus_region,
        minus_region=minus_region,
```

```
                plus_has_incident=plus_has_incident,
                minus_has_incident=minus_has_incident,
                eps_plus=eps_plus,
                mu_plus=mu_plus,
                eps_minus=eps_minus,
                mu_minus=mu_minus,
                k_plus=k_plus,
                k_minus=k_minus,
                q_plus_beta=q_plus_beta,
                q_plus_gamma=q_plus_gamma,
                bc_kind=bc_kind,
                robin_impedance=z_card if bc_kind == "robin" else 0.0 + 0.0j,
            )
        )

    return infos

def _green_2d(k0: complex | float, r: float) -> complex:
    """2D scalar Green's function G = j/4 * H0^(2)(k r)."""

    x = complex(k0) * max(r, EPS)
    if abs(x) <= 1e-12:
        x = 1e-12 + 0.0j
    return 0.25j * _hankel2_0(x)

def _dgreen_dn_obs(k0: complex | float, r_vec: np.ndarray, n_obs: np.ndarray) ->
    """Normal derivative of Green's function w.r.t. observation point normal."""

    r = float(np.linalg.norm(r_vec))
    if r <= EPS:
        return 0.0 + 0.0j
    x = complex(k0) * r
    if abs(x) <= 1e-12:
        x = 1e-12 + 0.0j
    h1 = _hankel2_1(x)
    projection = float(np.dot(n_obs, r_vec) / r)
    return (-0.25j * complex(k0)) * h1 * projection

def _dgreen_dn_src(k0: complex | float, r_vec: np.ndarray, n_src: np.ndarray) ->
    """Normal derivative of Green's function w.r.t. source panel normal."""

    r = float(np.linalg.norm(r_vec))
    if r <= EPS:
        return 0.0 + 0.0j
    x = complex(k0) * r
    if abs(x) <= 1e-12:
        x = 1e-12 + 0.0j
```

```
    h1 = _hankel2_1(x)
    projection = float(np.dot(n_src, r_vec) / r)
    return (0.25j * complex(k0)) * h1 * projection

def _quadrature_nodes(order: int = 10) -> Tuple[np.ndarray, np.ndarray]:
    qx, qw = np.polynomial.legendre.leggauss(order)
    t = 0.5 * (qx + 1.0)
    w = 0.5 * qw
    return t, w

_QUAD_CACHE: Dict[int, Tuple[np.ndarray, np.ndarray]] = {}
_QUAD_LOCK = threading.Lock()

def _get_quadrature(order: int) -> Tuple[np.ndarray, np.ndarray]:
    o = int(order)
    result = _QUAD_CACHE.get(o)
    if result is not None:
        return result
    with _QUAD_LOCK:
        # Double-check after acquiring lock.
        if o not in _QUAD_CACHE:
            _QUAD_CACHE[o] = _quadrature_nodes(o)
        return _QUAD_CACHE[o]

def _near_singular_scheme(distance: float, panel_length: float) -> Tuple[int, int
    """
    Choose quadrature order and source-panel subdivision count.

    This improves near-singular accuracy when observation points approach a panel
    """

    ratio = float(distance) / max(float(panel_length), EPS)
    if ratio < 0.25:
        return 64, 16
    if ratio < 0.60:
        return 56, 10
    if ratio < 1.50:
        return 40, 6
    if ratio < 3.00:
        return 28, 3
    return 16, 1

def _integrate_panel_generic(
    obs: np.ndarray,
    src: Panel,
    kernel_eval: Callable[[np.ndarray, np.ndarray], complex],
) -> complex:
```

```
    seg = src.p1 - src.p0
    distance = float(np.linalg.norm(obs - src.center))
    order, splits = _near_singular_scheme(distance, src.length)
    qt, qw = _get_quadrature(order)

    acc = 0.0 + 0.0j
    inv_splits = 1.0 / float(splits)
    for sidx in range(splits):
        t0 = float(sidx) * inv_splits
        dt = inv_splits
        for t, w in zip(qt, qw):
            u = t0 + dt * float(t)
            rp = src.p0 + u * seg
            acc += (dt * float(w)) * kernel_eval(obs, rp)
    return acc * src.length

def _single_layer_self_term(k0: complex | float, panel_length: float) -> complex:
    """
    Self-term using singularity subtraction + correction.

    Base asymptotic piece is analytic; remainder is integrated numerically so thi
    remains accurate beyond the small-argument regime.
    """

    l = max(float(panel_length), EPS)
    kz = complex(k0)
    x = kz * l / 4.0
    if abs(x) <= 1e-12:
        x = 1e-12 + 0.0j
    asym = (l / (2.0 * math.pi)) * (cmath.log(x) + EULER_GAMMA - 1.0) + 0.25j * l

    # Correction integral for finite kL effects:
    # 2 * ∫_0^{L/2} [G(r) - G_asym(r)] dr
    a = 0.5 * l
    kl = abs(kz) * l
    if kl < 0.5:
        order, splits = 24, 6
    elif kl < 3.0:
        order, splits = 36, 8
    elif kl < 10.0:
        order, splits = 48, 12
    else:
        order, splits = 64, 16

    qt, qw = _get_quadrature(order)
    corr_pos = 0.0 + 0.0j
    inv_splits = 1.0 / float(splits)
```

```
    for sidx in range(splits):
        r0 = a * float(sidx) * inv_splits
        dr = a * inv_splits
        for t, w in zip(qt, qw):
            r = r0 + dr * float(t)
            g = _green_2d(k0, r)
            z = kz * max(r, EPS) / 2.0
            if abs(z) <= 1e-12:
                z = 1e-12 + 0.0j
            g_asym = (1.0 / (2.0 * math.pi)) * (cmath.log(z) + EULER_GAMMA) + 0.2
            corr_pos += dr * float(w) * (g - g_asym)

    return asym + 2.0 * corr_pos

def _build_bem_matrices(
    panels: List[Panel],
    k0: complex | float,
    obs_normal_deriv: bool = False,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Assemble dense S and K/K' operators on the linear boundary mesh built from pr

    This helper is retained because a few internal and external call sites still
    historical `_build_bem_matrices(...)` name. Its behavior is now the active li
    assembly path only.
    """

    mesh = _build_linear_mesh(list(panels))
    return _assemble_linear_operator_matrices(
        mesh=mesh,
        k0=complex(k0),
        obs_normal_deriv=bool(obs_normal_deriv),
    )


def _build_operator_matrices_coupled(panels: List[Panel], k0: complex | float) ->
    """Build coupled-formulation S and K dense operators at one medium k."""
    return _build_bem_matrices(panels, complex(k0), obs_normal_deriv=False)

def _propagation_direction_from_user_angle(elev_deg: float) -> np.ndarray:
    """
    Convert user "coming-from" angle convention to propagation direction.

    Convention:
    - 0 deg: coming from +x (right), propagating toward -x.
    - +90 deg: coming from +y (top), propagating toward -y.
    - -90 deg: coming from -y (bottom), propagating toward +y.
```

```
    """

    phi = math.radians(elev_deg)
    return np.asarray([-math.cos(phi), -math.sin(phi)], dtype=float)

def _constraint_null_space(c_mat: np.ndarray) -> np.ndarray:
    """Compute a complex null-space basis Z such that C @ Z ~= 0."""

    c_eval = np.asarray(c_mat, dtype=np.complex128)
    if c_eval.ndim != 2:
        raise ValueError("Constraint matrix must be two-dimensional.")
    ncols = int(c_eval.shape[1])
    if ncols <= 0:
        raise ValueError("Constraint matrix must have at least one primal column.
    if c_eval.shape[0] == 0 or c_eval.size == 0:
        return np.eye(ncols, dtype=np.complex128)

    if _SCIPY_LINALG is not None:
        try:
            _, svals, vh = _SCIPY_LINALG.svd(c_eval, full_matrices=True, check_fi
        except Exception:
            _, svals, vh = np.linalg.svd(c_eval, full_matrices=True)
    else:
        _, svals, vh = np.linalg.svd(c_eval, full_matrices=True)

    svals = np.asarray(svals, dtype=float)
    sigma_max = float(np.max(svals)) if svals.size else 0.0
    tol = max(c_eval.shape) * max(sigma_max, 1.0) * np.finfo(float).eps * 16.0
    rank = int(np.sum(svals > tol))
    z_basis = np.asarray(vh[rank:, :].conj().T, dtype=np.complex128)
    if z_basis.ndim != 2 or z_basis.shape[0] != ncols:
        raise RuntimeError("Internal error: invalid null-space basis shape for ex
    if z_basis.shape[1] == 0:
        raise RuntimeError(
            "Aborting solve: exact junction constraints eliminate all primal degr
        )
    return z_basis

def _prepare_linear_solver(
    a_mat: np.ndarray,
    constraint_mat: np.ndarray | None = None,
    solver_method: str = "auto",
) -> PreparedLinearSolver:
    """
    Prepare reusable factorization for repeated solves with identical matrix.

    ``solver_method`` controls the linear algebra strategy:
```

```
- ``'auto'``: LU for small systems, GMRES for large (>GMRES_NODE_THRESHOLD DO
- ``'direct'``: always dense LU
- ``'gmres'``: always GMRES with block-diagonal preconditioner

Unconstrained systems remain on the direct square-solve path. When
`constraint_mat` is provided, the solver computes an exact null-space basis
and solves the reduced least-squares problem over the constrained subspace,
so junction constraints are enforced exactly rather than by weighted rows.
"""

a_eval = np.asarray(a_mat, dtype=np.complex128)
c_eval = None if constraint_mat is None else np.asarray(constraint_mat, dtype
if c_eval is not None and c_eval.size > 0:
    if c_eval.ndim != 2:
        raise ValueError("Constraint matrix must be two-dimensional.")
    if c_eval.shape[1] != a_eval.shape[1]:
        raise ValueError("Constraint matrix width does not match the primal s
    z_basis = _constraint_null_space(c_eval)
    reduced_mat = np.asarray(a_eval @ z_basis, dtype=np.complex128)
    return PreparedLinearSolver(
        a_mat=a_eval,
        method="constrained_null_lstsq",
        null_basis=z_basis,
        reduced_mat=reduced_mat,
        constraint_mat=c_eval,
    )

is_square = a_eval.shape[0] == a_eval.shape[1]
if not is_square:
    raise RuntimeError(
        "Aborting solve: reusable prepared solver requires a square primal sy
        "Use exact constraints through constraint_mat instead of a rectangula
    )

n = a_eval.shape[0]
method = solver_method.strip().lower()
use_gmres = (
    (method == "gmres")
    or (method == "auto" and n > GMRES_NODE_THRESHOLD and _SCIPY_SPARSE_LINAL
)

if use_gmres and _SCIPY_SPARSE_LINALG is not None:
    # Block-diagonal preconditioner: LU-factor the 2x2 block-diagonal
    # consisting of the (u,u) and (q,q) sub-blocks.
    half = n // 2
    precond = None
    if half > 0 and _SCIPY_LINALG is not None:
```

```
            try:
                uu_block = a_eval[:half, :half]
                qq_block = a_eval[half:, half:]
                lu_uu, piv_uu = _SCIPY_LINALG.lu_factor(uu_block)
                lu_qq, piv_qq = _SCIPY_LINALG.lu_factor(qq_block)

                def precond_matvec(x):
                    x = np.asarray(x, dtype=np.complex128)
                    out = np.empty_like(x)
                    out[:half] = _SCIPY_LINALG.lu_solve((lu_uu, piv_uu), x[:half]
                    out[half:] = _SCIPY_LINALG.lu_solve((lu_qq, piv_qq), x[half:]
                    return out

                precond = _SCIPY_SPARSE_LINALG.LinearOperator(
                    shape=(n, n), matvec=precond_matvec, dtype=np.complex128,
                )
            except Exception:
                precond = None

        return PreparedLinearSolver(
            a_mat=a_eval, method="gmres", preconditioner=precond,
        )

    if _SCIPY_LINALG is not None:
        try:
            lu, piv = _SCIPY_LINALG.lu_factor(a_eval)
            return PreparedLinearSolver(a_mat=a_eval, method="scipy_lu", lu=lu, p
        except Exception:
            pass

    return PreparedLinearSolver(a_mat=a_eval, method="numpy_solve")

def _solve_with_prepared_solver(prepared: PreparedLinearSolver, rhs: np.ndarray)
    """Solve with a prepared linear-solver handle."""

    rhs_eval = np.asarray(rhs, dtype=np.complex128)
    if prepared.method == "scipy_lu" and _SCIPY_LINALG is not None and prepared.l
        return _SCIPY_LINALG.lu_solve((prepared.lu, prepared.piv), rhs_eval)
    if prepared.method == "numpy_solve":
        return np.linalg.solve(prepared.a_mat, rhs_eval)
    if prepared.method == "gmres" and _SCIPY_SPARSE_LINALG is not None:
        a = prepared.a_mat
        if rhs_eval.ndim == 1:
            sol, info = _SCIPY_SPARSE_LINALG.gmres(
                a, rhs_eval, M=prepared.preconditioner,
                restart=prepared.gmres_restart,
                maxiter=prepared.gmres_maxiter,
```

```
                atol=prepared.gmres_tol,
            )
            if info != 0:
                # Fallback to direct solve if GMRES didn't converge.
                sol = np.linalg.solve(a, rhs_eval)
            return sol
        # Multi-RHS: solve each column.
        sols = []
        for i in range(rhs_eval.shape[1]):
            sol_i, info = _SCIPY_SPARSE_LINALG.gmres(
                a, rhs_eval[:, i], M=prepared.preconditioner,
                restart=prepared.gmres_restart,
                maxiter=prepared.gmres_maxiter,
                atol=prepared.gmres_tol,
            )
            if info != 0:
                sol_i = np.linalg.solve(a, rhs_eval[:, i])
            sols.append(sol_i)
        return np.column_stack(sols)
    if prepared.method == "constrained_null_lstsq":
        if prepared.null_basis is None or prepared.reduced_mat is None:
            raise RuntimeError("Aborting solve: constrained solver is missing its
        reduced_sol, *_ = np.linalg.lstsq(prepared.reduced_mat, rhs_eval, rcond=N
        return np.asarray(prepared.null_basis @ reduced_sol, dtype=np.complex128)
    raise RuntimeError(
        f"Aborting solve: unsupported prepared solver method '{prepared.method}'.
    )

def _solve_many_with_prepared_solver(prepared: PreparedLinearSolver, rhs_list: Li
    """Solve A x_k = b_k for many right-hand-sides using one prepared handle."""

    if not rhs_list:
        return []
    rhs_mat = np.column_stack(rhs_list)
    sol_mat = _solve_with_prepared_solver(prepared, rhs_mat)
    if sol_mat.ndim == 1:
        sol_mat = sol_mat.reshape(-1, 1)
    return [np.asarray(sol_mat[:, i], dtype=np.complex128) for i in range(sol_mat

def _residual_norm(a_mat: np.ndarray, x: np.ndarray, b: np.ndarray) -> float:
    denom = float(np.linalg.norm(b))
    if denom <= EPS:
        denom = 1.0
    return float(np.linalg.norm(a_mat @ x - b) / denom)

def _residual_norm_many(a_mat: np.ndarray, x_mat: np.ndarray, b_mat: np.ndarray)
    """Vectorized residual norms for matrix right-hand-sides."""
```

```
    x_eval = np.asarray(x_mat)
    b_eval = np.asarray(b_mat)
    if x_eval.ndim == 1:
        return np.asarray([_residual_norm(a_mat, x_eval, b_eval)], dtype=float)

    residual = a_mat @ x_eval - b_eval
    num = np.linalg.norm(residual, axis=0)
    den = np.linalg.norm(b_eval, axis=0)
    den = np.where(den <= EPS, 1.0, den)
    return np.asarray(num / den, dtype=float)

def _constraint_residual_norm_many(c_mat: np.ndarray | None, x_mat: np.ndarray) -
    """Absolute residual norms for exact linear constraints C x = 0."""

    if c_mat is None:
        x_eval = np.asarray(x_mat)
        cols = 1 if x_eval.ndim == 1 else int(x_eval.shape[1])
        return np.zeros(cols, dtype=float)
    c_eval = np.asarray(c_mat, dtype=np.complex128)
    if c_eval.size == 0:
        x_eval = np.asarray(x_mat)
        cols = 1 if x_eval.ndim == 1 else int(x_eval.shape[1])
        return np.zeros(cols, dtype=float)

    x_eval = np.asarray(x_mat, dtype=np.complex128)
    if x_eval.ndim == 1:
        return np.asarray([float(np.linalg.norm(c_eval @ x_eval))], dtype=float)
    return np.asarray(np.linalg.norm(c_eval @ x_eval, axis=0), dtype=float)

def _cond_estimate(a_mat: np.ndarray) -> float:
    try:
        return float(np.linalg.cond(a_mat))
    except np.linalg.LinAlgError:
        return float("inf")

def _normalize_rcs_normalization_mode(mode: str | None) -> str:
    """Accept only physical sigma_2d normalization aliases."""

    text = str(mode or "").strip().lower().replace("-", "_")
    if text in {"", "physical", "divide_by_k", "with_k", "k", "derived", "width",
        return RCS_NORM_MODE_PHYSICAL
    raise ValueError(
        f"Unsupported rcs_normalization_mode '{mode}'. This solver now supports o
        "sigma_2d = |A|^2 / (4k)."
    )
```

```
def _rcs_sigma_from_amp(
    amp_vec: np.ndarray,
    k_value: float,
) -> np.ndarray:
    """Apply physical 2D scattering-width normalization to the far-field amplitud

    amp_eval = np.asarray(amp_vec, dtype=np.complex128)
    scale = float(RCS_NORM_NUMERATOR) / max(float(k_value), EPS)
    sigma_lin = scale * (np.abs(amp_eval) ** 2)
    sigma_lin = np.where(np.isfinite(sigma_lin) & (sigma_lin >= EPS), sigma_lin,
    return np.asarray(sigma_lin, dtype=float)

def _resolve_worker_count(enabled: bool, requested: int, jobs: int) -> int:
    """
    Resolve thread-pool worker count for per-elevation parallel execution.

    Returns 1 when parallel execution is disabled or not useful.
    """

    count = int(max(0, jobs))
    if not enabled or count <= 1:
        return 1
    if int(requested) > 0:
        return max(1, min(int(requested), count))
    cpu = int(os.cpu_count() or 1)
    return max(1, min(cpu, count))

def evaluate_quality_gate(
    metadata: Dict[str, Any],
    thresholds: Dict[str, float | int] | None = None,
) -> Dict[str, Any]:
    """
    Evaluate a lightweight numeric quality gate from solver metadata.

    This does not prove correctness; it catches obvious numerical-risk runs.
    """

    defaults: Dict[str, float | int] = {
        "residual_norm_max": 1.0e-2,
        "constraint_residual_norm_max": 1.0e-8,
        "condition_est_max": 1.0e6,
        "warnings_max": 10,
    }
    merged = dict(defaults)
    if thresholds:
        merged.update(dict(thresholds))
```

```
residual_limit = float(merged.get("residual_norm_max", defaults["residual_nor
constraint_limit = float(merged.get("constraint_residual_norm_max", defaults[
condition_limit = float(merged.get("condition_est_max", defaults["condition_e
warnings_limit = int(merged.get("warnings_max", defaults["warnings_max"]))

residual_value = float(metadata.get("residual_norm_max", 0.0) or 0.0)
constraint_value = float(metadata.get("constraint_residual_norm_max", 0.0) or
condition_value = float(metadata.get("condition_est_max", 0.0) or 0.0)
condition_computed = bool(metadata.get("condition_est_computed", True))
warnings_count = len(list(metadata.get("warnings", []) or []))

violations: List[str] = []
if not math.isfinite(residual_value) or residual_value > residual_limit:
    violations.append(
        f"residual_norm_max={residual_value:.6g} exceeds limit {residual_limi
    )
if int(metadata.get("junction_constraints", 0) or 0) > 0 and (
    (not math.isfinite(constraint_value)) or constraint_value > constraint_li
):
    violations.append(
        f"constraint_residual_norm_max={constraint_value:.6g} exceeds limit {
    )
if condition_computed and (not math.isfinite(condition_value) or condition_va
    violations.append(
        f"condition_est_max={condition_value:.6g} exceeds limit {condition_li
    )
if warnings_count > warnings_limit:
    violations.append(
        f"warnings_count={warnings_count} exceeds limit {warnings_limit}"
    )

return {
    "passed": len(violations) == 0,
    "thresholds": {
        "residual_norm_max": residual_limit,
        "constraint_residual_norm_max": constraint_limit,
        "condition_est_max": condition_limit,
        "warnings_max": warnings_limit,
    },
    "values": {
        "residual_norm_max": residual_value,
        "constraint_residual_norm_max": constraint_value,
        "condition_est_max": condition_value,
        "condition_est_computed": condition_computed,
        "warnings_count": warnings_count,
    },
    "violations": violations,
```

```
    }

def _is_all_pec(infos: List[PanelCoupledInfo]) -> bool:
    """Return True if every element is a PEC surface (robin BC with zero impedanc
    return all(
        info.bc_kind == 'robin' and abs(info.robin_impedance) <= EPS
        for info in infos
    )

def _is_all_robin(infos: List[PanelCoupledInfo]) -> bool:
    """Return True if every element uses a Robin BC (PEC or IBC, no dielectric)."
    return all(info.bc_kind == 'robin' for info in infos)

def _estimate_memory_gb(
    nnodes: int,
    use_cfie: bool,
    n_regions: int = 1,
) -> float:
    """
    Estimate peak memory for the dense BIE/MoM solve in GB.

    Accounts for: system matrix, region operators, RHS, solution, factorization.
    """

    bytes_per_complex = 16  # complex128
    # 2N×2N system matrix + factorization copy
    sys_size = 2 * nnodes
    sys_bytes = 2 * sys_size * sys_size * bytes_per_complex
    # Region operators: 2 matrices (S, K) per side per region, plus CFIE extras
    ops_per_region = 4 if not use_cfie else 8
    region_bytes = n_regions * ops_per_region * nnodes * nnodes * bytes_per_compl
    # RHS + solution
    misc_bytes = 4 * sys_size * bytes_per_complex * 1000  # generous estimate
    total = sys_bytes + region_bytes + misc_bytes
    return total / (1024 ** 3)

def _solve_tm_robin_mfie(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    pol: str,
    k0: float,
    elevations_deg: np.ndarray,
    obs_order: int = 8,
    src_order: int = 8,
) -> Tuple[np.ndarray, np.ndarray, float]:
    """
    Solve TM Robin (PEC or IBC) problems via a generalized MFIE.
```

```
Uses the single-layer potential representation u_scat = SLP(sigma).
The exterior-limit Robin BC gives:

    (-½ M + K' + α·S) σ = -(∂u_inc/∂n + α·u_inc)

where α is the per-node Robin coefficient (0 for PEC, nonzero for IBC).
K' is the adjoint double-layer operator (obs_normal_deriv=True).

Returns (rcs_linear, amplitude, residual_norm) arrays over elevations.
"""

nnodes = len(mesh.nodes)
elev = np.asarray(elevations_deg, dtype=float).reshape(-1)

# Assemble operators.
s_mat, kp_mat = _assemble_linear_operator_matrices(
    mesh, k0, obs_normal_deriv=True,
    obs_order=obs_order, src_order=src_order,
)
mass_mat = _assemble_linear_mass_matrix(mesh)

# Per-node Robin alpha for TM.
# The coupled formulation's alpha is defined for the DLP representation
# (exterior +1/2 jump): q + alpha*u = 0  →  (1/2 M + K + alpha*S)*u = b.
# The MFIE uses the SLP representation (exterior -1/2 jump):
#   du_total/dn + alpha*u_total = 0
#   (-1/2 M + K')*sigma + alpha*(S*sigma) = -(b_dn + alpha*b_u)
# The SLP normal derivative has a SIGN-FLIPPED jump (-1/2 vs +1/2), so
# the Robin coupling in the MFIE uses -alpha:
#   (-1/2 M + K' - alpha*S)*sigma = -(b_dn - alpha*b_u)
alpha_nodes = np.zeros(nnodes, dtype=np.complex128)
incident: Dict[int, List[int]] = {}
for eidx, elem in enumerate(mesh.elements):
    for nid in elem.node_ids:
        incident.setdefault(int(nid), []).append(int(eidx))
for nid in range(nnodes):
    elem_ids = incident.get(int(nid), [])
    if elem_ids:
        info = infos[int(elem_ids[0])]
        z_surf = complex(info.robin_impedance)
        if abs(z_surf) > EPS:
            eps_m = info.eps_minus if info.minus_region >= 0 else info.eps_pl
            mu_m = info.mu_minus if info.minus_region >= 0 else info.mu_plus
            k_m = info.k_minus if info.minus_region >= 0 else info.k_plus
            # The MFIE Robin BC derivation gives:
            #   (-1/2 M + K' + alpha_solver * S) sigma = -(b_dn + alpha_solve
```

```
            # where alpha_solver is the same coefficient used in the coupled
            alpha_nodes[nid] = _surface_robin_alpha(pol, eps_m, mu_m, k_m, z_

# System matrix: (-1/2 M + K' + diag(alpha_solver) * S)
# where S here is the SLP operator assembled with obs_normal_deriv=False.
s_standard, _ = _assemble_linear_operator_matrices(
    mesh, k0, obs_normal_deriv=False,
    obs_order=obs_order, src_order=src_order,
)
a_mfie = -0.5 * mass_mat + kp_mat
has_ibc = np.any(np.abs(alpha_nodes) > EPS)
if has_ibc:
    a_mfie += np.diag(alpha_nodes) @ s_standard

# RHS: -(du_inc/dn + alpha * u_inc)
rhs_mfie = np.zeros((nnodes, elev.size), dtype=np.complex128)
for elem in mesh.elements:
    ids = np.asarray(elem.node_ids, dtype=int)
    load_dn = _linear_element_incident_dn_load_many(elem, k_air=k0, elevation
    rhs_mfie[ids, :] -= load_dn
    if has_ibc:
        load_u = _linear_element_incident_load_many(elem, k_air=k0, elevation
        rhs_mfie[ids, :] -= alpha_nodes[ids, None] * load_u

_ensure_finite_linear_system(a_mfie, rhs_mfie, label="TM Robin MFIE system")
sigma_mat = np.linalg.solve(a_mfie, rhs_mfie)

# Residual.
residual = np.linalg.norm(a_mfie @ sigma_mat - rhs_mfie, axis=0)
rhs_norm = np.linalg.norm(rhs_mfie, axis=0)
rhs_norm = np.where(rhs_norm <= EPS, 1.0, rhs_norm)
residual_vec = residual / rhs_norm

# Far-field: SLP far-field projector  A = ∫ σ * exp(jk d·r') ds'.
phi = np.deg2rad(elev)
dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
qt, qw = _get_quadrature(max(2, obs_order))
amp = np.zeros(elev.size, dtype=np.complex128)

for elem in mesh.elements:
    ids = np.asarray(elem.node_ids, dtype=int)
    sigma_local = sigma_mat[ids, :]
    for t, w in zip(qt, qw):
        shape = _linear_shape_values(float(t))[:, None]
        rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
        phase = np.exp(1j * k0 * (dirs @ rp))
        sigma_t = np.sum(shape * sigma_local, axis=0)
```

```
            amp += float(w) * float(elem.length) * phase * sigma_t

    rcs_lin = _rcs_sigma_from_amp(amp, k0)
    return rcs_lin, amp, float(np.max(residual_vec))

def _is_single_dielectric_body(infos: List[PanelCoupledInfo]) -> bool:
    """Return True if every element is a transmission interface (TYPE 3 dielectri
    return all(info.bc_kind == 'transmission' for info in infos)

def _solve_dielectric_indirect(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    pol: str,
    k0: float,
    elevations_deg: np.ndarray,
    obs_order: int = 8,
    src_order: int = 8,
) -> Tuple[np.ndarray, np.ndarray, float]:
    """
    Solve dielectric scattering via the indirect two-density formulation.

    The coupled trace formulation degenerates for dielectrics because the exterio
    BIE alone determines the far-field amplitude regardless of the flux continuit
    parameter beta.  This indirect formulation uses separate densities:

        u_scat(r) = DL_0(mu)   (exterior, double-layer at k0)
        u_int(r)  = SL_1(sigma) (interior, single-layer at k1)

    Trace continuity:  u_inc + (mu/2 + K0*mu) = S1*sigma
    Flux continuity:   D0*mu + factor*(sigma/2 + K'1*sigma) = -du_inc/dn

    where factor = mu_ext/mu_int for E_z, eps_ext/eps_int for H_z.

    Far-field: A = integral jk0*(d.n)*mu * exp(jk0 d.r') ds'
    """

    nnodes = len(mesh.nodes)
    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)

    # Determine interior wavenumber from coupled infos.
    k1_vals = {complex(info.k_plus) for info in infos if info.plus_region > 0}
    if not k1_vals:
        k1_vals = {complex(info.k_minus) for info in infos if info.minus_region >
    if not k1_vals:
        raise ValueError("Dielectric indirect solver requires at least one dielec
    k1 = k1_vals.pop()
```

```
# Determine flux scaling factor.
info0 = infos[0]
if pol == 'TE':
    # E_z: flux uses 1/mu → factor = mu_ext/mu_int
    factor = complex(info0.mu_minus / info0.mu_plus) if abs(info0.mu_plus) >
else:
    # H_z: flux uses 1/eps → factor = eps_ext/eps_int
    factor = complex(info0.eps_minus / info0.eps_plus) if abs(info0.eps_plus)

# Assemble operators.
S0, K0 = _assemble_linear_operator_matrices(mesh, k0, obs_normal_deriv=False,
    obs_order=obs_order, src_order=src_order)
_, Kp1 = _assemble_linear_operator_matrices(mesh, k1, obs_normal_deriv=True,
    obs_order=obs_order, src_order=src_order)
S1, _ = _assemble_linear_operator_matrices(mesh, k1, obs_normal_deriv=False,
    obs_order=obs_order, src_order=src_order)
D0 = _assemble_linear_hypersingular_matrix(mesh, k0, obs_order=obs_order, src
M = _assemble_linear_mass_matrix(mesh)

# Build system: 2N x 2N.
a_sys = np.zeros((2 * nnodes, 2 * nnodes), dtype=np.complex128)
# Row 1 (trace): 0.5*M*mu + K0*mu - S1*sigma = bu
a_sys[:nnodes, :nnodes] = 0.5 * M + K0
a_sys[:nnodes, nnodes:] = -S1
# Row 2 (flux): D0*mu + factor*(0.5*M + K'1)*sigma = -bdn
a_sys[nnodes:, :nnodes] = D0
a_sys[nnodes:, nnodes:] = factor * (0.5 * M + Kp1)

# Build RHS for all elevations.
rhs_sys = np.zeros((2 * nnodes, elev.size), dtype=np.complex128)
for elem in mesh.elements:
    ids = np.asarray(elem.node_ids, dtype=int)
    load_u = _linear_element_incident_load_many(elem, k_air=k0, elevations_de
    load_dn = _linear_element_incident_dn_load_many(elem, k_air=k0, elevation
    rhs_sys[ids, :] += load_u
    rhs_sys[nnodes + ids, :] -= load_dn

_ensure_finite_linear_system(a_sys, rhs_sys, label="dielectric indirect syste
sol = np.linalg.solve(a_sys, rhs_sys)
if sol.ndim == 1:
    sol = sol.reshape(-1, 1)

mu_mat = sol[:nnodes, :]  # DL density

# Residual.
residual = np.linalg.norm(a_sys @ sol - rhs_sys, axis=0)
rhs_norm = np.linalg.norm(rhs_sys, axis=0)
```

```
    rhs_norm = np.where(rhs_norm <= EPS, 1.0, rhs_norm)
    residual_vec = residual / rhs_norm

    # Far-field from DL density: A = integral jk0*(d.n)*mu * phase ds'.
    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, obs_order))
    amp = np.zeros(elev.size, dtype=np.complex128)

    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        mu_local = mu_mat[ids, :]
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))[:, None]
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k0 * (dirs @ rp))
            dot_n = dirs @ elem.normal
            mu_t = np.sum(shape * mu_local, axis=0)
            amp += float(w) * float(elem.length) * phase * 1j * k0 * dot_n * mu_t

    rcs_lin = _rcs_sigma_from_amp(amp, k0)
    return rcs_lin, amp, float(np.max(residual_vec))

def _is_all_ibc(infos: List[PanelCoupledInfo]) -> bool:
    """Return True if every element is a Robin BC surface with nonzero impedance.
    return all(
        info.bc_kind == 'robin' and abs(info.robin_impedance) > EPS
        for info in infos
    )

def _solve_robin_bie(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    pol: str,
    k0: float,
    elevations_deg: np.ndarray,
    obs_order: int = 8,
    src_order: int = 8,
) -> Tuple[np.ndarray, np.ndarray, float]:
    """
    Solve IBC scattering via the Robin boundary integral equation.

    Uses the single-layer potential representation u_scat = SLP(sigma) with
    the Robin BC du/dn + alpha*u = 0 applied at the exterior limit:

        (-1/2 M + K' + alpha*S) sigma = -(du_inc/dn + alpha*u_inc)
```

```
The Robin coefficient alpha comes from _surface_robin_alpha which encodes
the physical SIBC for each polarization.

This formulation unifies PEC, PMC, and IBC in a single solver:
- alpha → ∞ (TE PEC): degenerates to EFIE  S*sigma = -u_inc
- alpha → 0 (TM PEC): degenerates to MFIE  (-1/2 M + K')*sigma = -du_inc/dn
- finite alpha: general impedance surface

Far-field: SLP projector  A = integral sigma * exp(jk d.r') ds'
"""

nnodes = len(mesh.nodes)
elev = np.asarray(elevations_deg, dtype=float).reshape(-1)

# Determine alpha from the first element's impedance and material data.
info0 = infos[0]
alpha = _surface_robin_alpha(pol, info0.eps_minus, info0.mu_minus, complex(k0

# Assemble operators.
S_mat, _ = _assemble_linear_operator_matrices(
    mesh, k0, obs_normal_deriv=False, obs_order=obs_order, src_order=src_orde
_, Kp_mat = _assemble_linear_operator_matrices(
    mesh, k0, obs_normal_deriv=True, obs_order=obs_order, src_order=src_order
M_mat = _assemble_linear_mass_matrix(mesh)

# System: (-1/2 M + K' + alpha*S) sigma = -(bdn + alpha*bu)
a_sys = -0.5 * M_mat + Kp_mat + alpha * S_mat

# RHS for all elevations.
rhs_sys = np.zeros((nnodes, elev.size), dtype=np.complex128)
for elem in mesh.elements:
    ids = np.asarray(elem.node_ids, dtype=int)
    load_u = _linear_element_incident_load_many(elem, k_air=k0, elevations_de
    load_dn = _linear_element_incident_dn_load_many(elem, k_air=k0, elevation
    rhs_sys[ids, :] -= (load_dn + alpha * load_u)

_ensure_finite_linear_system(a_sys, rhs_sys, label="Robin-BIE IBC system")
sigma_mat = np.linalg.solve(a_sys, rhs_sys)
if sigma_mat.ndim == 1:
    sigma_mat = sigma_mat.reshape(-1, 1)

# Residual.
residual = np.linalg.norm(a_sys @ sigma_mat - rhs_sys, axis=0)
rhs_norm = np.linalg.norm(rhs_sys, axis=0)
rhs_norm = np.where(rhs_norm <= EPS, 1.0, rhs_norm)
residual_vec = residual / rhs_norm
```

```
    # Far-field: SLP projector.
    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, obs_order))
    amp = np.zeros(elev.size, dtype=np.complex128)

    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        sigma_local = sigma_mat[ids, :]
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))[:, None]
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k0 * (dirs @ rp))
            sigma_t = np.sum(shape * sigma_local, axis=0)
            amp += float(w) * float(elem.length) * phase * sigma_t

    rcs_lin = _rcs_sigma_from_amp(amp, k0)
    return rcs_lin, amp, float(np.max(residual_vec))


def _solve_mixed_pec_dielectric(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    pol: str,
    k0: float,
    elevations_deg: np.ndarray,
    obs_order: int = 8,
    src_order: int = 8,
) -> Tuple[np.ndarray, np.ndarray, float]:
    """
    Solve mixed PEC/IBC + dielectric scattering via unified SLP formulation.

    Exterior: u_scat = SLP_k0(sigma) over ALL segments.
    Interior: u_int = SLP_k_int(tau) over dielectric segments only.
    """

    nnodes = len(mesh.nodes)
    elev = np.asarray(elevations_deg, dtype=float).reshape(-1)
    elements = list(mesh.elements)
    nelems = len(elements)

    # Classify nodes.
    node_is_diel = np.zeros(nnodes, dtype=bool)
    elem_is_diel = np.zeros(nelems, dtype=bool)
    node_robin_alpha = np.zeros(nnodes, dtype=np.complex128)
    node_is_pec = np.zeros(nnodes, dtype=bool)
```

```
for eidx, (elem, info) in enumerate(zip(elements, infos)):
    if info.bc_kind == 'transmission':
        node_is_diel[elem.node_ids[0]] = True
        node_is_diel[elem.node_ids[1]] = True
        elem_is_diel[eidx] = True
    else:
        for nid in elem.node_ids:
            z_s = complex(info.robin_impedance)
            if abs(z_s) <= EPS:
                node_is_pec[nid] = True
            else:
                eps_m = info.eps_minus if info.minus_region >= 0 else info.ep
                mu_m = info.mu_minus if info.minus_region >= 0 else info.mu_p
                k_m = info.k_minus if info.minus_region >= 0 else info.k_plus
                node_robin_alpha[nid] = _surface_robin_alpha(pol, eps_m, mu_m

diel_nodes = np.flatnonzero(node_is_diel)
n_diel = int(diel_nodes.size)
n_sys = nnodes + n_diel

# Interior wavenumber and flux ratio.
diel_info = next(info for info in infos if info.bc_kind == 'transmission')
k_int_vals = {complex(i.k_plus) for i in infos if i.bc_kind == 'transmission'
if not k_int_vals:
    k_int_vals = {complex(i.k_minus) for i in infos if i.bc_kind == 'transmis
k_int = k_int_vals.pop() if k_int_vals else complex(k0)

if pol == 'TE':
    beta = complex(diel_info.mu_plus / diel_info.mu_minus) if diel_info.minus
else:
    beta = complex(diel_info.eps_plus / diel_info.eps_minus) if diel_info.min
if abs(beta) <= EPS:
    beta = 1.0 + 0.0j
inv_beta = 1.0 / beta

# Operators.
S_0, _ = _assemble_linear_operator_matrices(mesh, k0, False, obs_order, src_o
_, Kp_0 = _assemble_linear_operator_matrices(mesh, k0, True, obs_order, src_o
S_k, _ = _assemble_linear_operator_matrices(mesh, k_int, False, obs_order, sr
    source_element_mask=elem_is_diel)
_, Kp_k = _assemble_linear_operator_matrices(mesh, k_int, True, obs_order, sr
    source_element_mask=elem_is_diel)
M = _assemble_linear_mass_matrix(mesh)

a_sys = np.zeros((n_sys, n_sys), dtype=np.complex128)

# Robin rows: BC on sigma.
```

```
for nid in np.flatnonzero(~node_is_diel):
    row = int(nid)
    is_pec = bool(node_is_pec[nid])
    alpha = complex(node_robin_alpha[nid])
    if pol == 'TE' and is_pec:
        a_sys[row, :nnodes] = S_0[nid, :]
    elif pol == 'TM' and is_pec:
        a_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :]
    else:
        a_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :] + alpha * S_0[n

# Dielectric flux rows (reuse sigma rows for diel nodes).
for di, nid in enumerate(diel_nodes):
    row = int(nid)
    a_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :]
    for dj, nid_j in enumerate(diel_nodes):
        a_sys[row, nnodes + dj] = -inv_beta * (0.5 * M[nid, nid_j] + Kp_k[nid

# Dielectric trace rows.
for di, nid in enumerate(diel_nodes):
    row = nnodes + di
    a_sys[row, :nnodes] = S_0[nid, :]
    for dj, nid_j in enumerate(diel_nodes):
        a_sys[row, nnodes + dj] = -S_k[nid, nid_j]

# RHS.
rhs_sys = np.zeros((n_sys, elev.size), dtype=np.complex128)
for elem in mesh.elements:
    ids = list(elem.node_ids)
    load_u = _linear_element_incident_load_many(elem, k_air=k0, elevations_de
    load_dn = _linear_element_incident_dn_load_many(elem, k_air=k0, elevation
    for li, nid in enumerate(ids):
        if node_is_diel[nid]:
            di = int(np.flatnonzero(diel_nodes == nid)[0])
            rhs_sys[nid, :] -= load_dn[li, :]
            rhs_sys[nnodes + di, :] -= load_u[li, :]
        else:
            is_pec = bool(node_is_pec[nid])
            alpha = complex(node_robin_alpha[nid])
            if pol == 'TE' and is_pec:
                rhs_sys[nid, :] -= load_u[li, :]
            elif pol == 'TM' and is_pec:
                rhs_sys[nid, :] -= load_dn[li, :]
            else:
                rhs_sys[nid, :] -= (load_dn[li, :] + alpha * load_u[li, :])

_ensure_finite_linear_system(a_sys, rhs_sys, label='mixed PEC+dielectric syst
```

```
    sol = np.linalg.solve(a_sys, rhs_sys)
    if sol.ndim == 1:
        sol = sol.reshape(-1, 1)
    sigma_mat = sol[:nnodes, :]

    residual = np.linalg.norm(a_sys @ sol - rhs_sys, axis=0)
    rhs_norm = np.linalg.norm(rhs_sys, axis=0)
    rhs_norm = np.where(rhs_norm <= EPS, 1.0, rhs_norm)
    residual_vec = residual / rhs_norm

    phi = np.deg2rad(elev)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, obs_order))
    amp = np.zeros(elev.size, dtype=np.complex128)
    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        sigma_local = sigma_mat[ids, :]
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))[:, None]
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k0 * (dirs @ rp))
            sigma_t = np.sum(shape * sigma_local, axis=0)
            amp += float(w) * float(elem.length) * phase * sigma_t

    rcs_lin = _rcs_sigma_from_amp(amp, k0)
    return rcs_lin, amp, float(np.max(residual_vec))


def _is_mixed_pec_dielectric(infos: List[PanelCoupledInfo]) -> bool:
    has_robin = any(info.bc_kind == 'robin' for info in infos)
    has_diel = any(info.bc_kind == 'transmission' for info in infos)
    return has_robin and has_diel



def solve_monostatic_rcs_2d(
    geometry_snapshot: Dict[str, Any],
    frequencies_ghz: List[float],
    elevations_deg: List[float],
    polarization: str,
    geometry_units: str = "inches",
    material_base_dir: str | None = None,
    progress_callback: Callable[[int, int, str], None] | None = None,
    quality_thresholds: Dict[str, float | int] | None = None,
    strict_quality_gate: bool = False,
    max_panels: int = MAX_PANELS_DEFAULT,
    compute_condition_number: bool = False,
```

```
    mesh_reference_ghz: float | None = None,
    rcs_normalization_mode: str = RCS_NORM_MODE_DEFAULT,
    cfie_alpha: float = CFIE_ALPHA_DEFAULT,
    abort_event: threading.Event | None = None,
    solver_method: str = "auto",
) -> Dict[str, Any]:
    """
    Main entry point for monostatic 2D RCS using the linear/Galerkin coupled trac

    Per frequency:
    - build the boundary discretization,
    - assemble the linear/Galerkin coupled system,
    - solve all requested elevations,
    - compute monostatic backscatter RCS.

    Angle convention (coming-from):
    - 0 deg: from right to left
    - +90 deg: from top to bottom
    - -90 deg: from bottom to top
    """

    if not frequencies_ghz:
        raise ValueError("At least one frequency is required.")
    if not elevations_deg:
        raise ValueError("At least one elevation angle is required.")

    frequencies = [float(f) for f in frequencies_ghz]
    elevations = [float(e) for e in elevations_deg]
    if any(f <= 0.0 for f in frequencies):
        raise ValueError("Frequencies must be positive GHz values.")

    mesh_ref_ghz: float | None = None
    if mesh_reference_ghz is not None:
        mesh_ref_ghz = float(mesh_reference_ghz)
        if (not math.isfinite(mesh_ref_ghz)) or mesh_ref_ghz <= 0.0:
            raise ValueError("mesh_reference_ghz must be a positive finite GHz va

    rcs_norm_mode = _normalize_rcs_normalization_mode(rcs_normalization_mode)
    _raise_if_untrusted_math_backends()

    pol = _normalize_polarization(polarization)
    unit_scale = _unit_scale_to_meters(geometry_units)

    base_dir = material_base_dir or os.getcwd()
    preflight_report = validate_geometry_snapshot_for_solver(geometry_snapshot, b
    materials = MaterialLibrary.from_entries(
        geometry_snapshot.get("ibcs", []) or [],
```

```
    geometry_snapshot.get("dielectrics", []) or [],
    base_dir=base_dir,
)
for _msg in list(preflight_report.get('warnings', []) or []):
    materials.warn_once(str(_msg))

samples: List[Dict[str, Any]] = []
total_steps = len(frequencies) * (len(elevations) + 1)
done_steps = 0

residual_values: List[float] = []
constraint_residual_values: List[float] = []
cond_values: List[float] = []
mesh_reference_values: List[float] = []
panel_count_values: List[int] = []
panel_length_min_values: List[float] = []
panel_length_max_values: List[float] = []
elevations_arr = np.asarray(elevations, dtype=float)
reused_matrix_solve_count = 0
max_parallel_workers_used = 1
formulation_label = "2D BIE/MoM coupled dielectric trace formulation (linear
junction_stats = {
    "junction_nodes": 0,
    "junction_constraints": 0,
    "junction_panels": 0,
    "junction_trace_constraints": 0,
    "junction_flux_constraints": 0,
    "junction_orientation_conflict_nodes": 0,
}

def emit_progress(message: str) -> None:
    if progress_callback is None:
        return
    try:
        progress_callback(done_steps, total_steps, message)
    except Exception:
        pass

def check_abort() -> None:
    if abort_event is not None and abort_event.is_set():
        raise InterruptedError("Solve cancelled by user.")

check_abort()
emit_progress("Initializing solver")

# --- Mesh caching: when mesh_reference_ghz is set, the mesh topology is
# frequency-independent and can be built once before the frequency loop. ---
```

```
    cached_panels: List[Any] | None = None
    cached_mesh: Any = None
    cached_mesh_stats: Dict[str, Any] | None = None
    cached_junction_constraints: np.ndarray | None = None
    cached_junction_stats: Dict[str, Any] | None = None

    if mesh_ref_ghz is not None and len(frequencies) > 1:
        ref_lambda = C0 / (mesh_ref_ghz * 1e9)
        ref_k0 = 2.0 * math.pi * mesh_ref_ghz * 1e9 / C0
        cached_panels = _build_panels(
            geometry_snapshot, unit_scale, ref_lambda, max_panels=max_panels,
        )
        # Build preview infos at reference frequency for interface-aware splittin
        ref_infos = _build_coupled_panel_info(cached_panels, materials, mesh_ref_
        cached_mesh, cached_mesh_stats = _build_linear_mesh_interface_aware(
            cached_panels, ref_infos,
        )
        cached_mesh_stats = dict(cached_mesh_stats)
        cached_mesh_stats.update(_linear_coupled_node_report(
            cached_mesh,
            _build_linear_coupled_infos(cached_mesh, materials, mesh_ref_ghz, pol
        ))
        # Junction constraints depend on coupled_infos which may be freq-dependen
        # Build once at reference freq; topology-based constraints are stable.
        ref_coupled = _build_linear_coupled_infos(cached_mesh, materials, mesh_re
        cached_junction_constraints, cached_junction_stats = _build_linear_juncti
            cached_mesh, ref_coupled,
        )
        materials.warn_once(
            f"Mesh cached at {mesh_ref_ghz:g} GHz reference frequency "
            f"({len(cached_panels)} panels, {len(cached_mesh.nodes)} nodes). "
            f"Reusing for {len(frequencies)} frequencies."
        )

    for freq_ghz in frequencies:
        check_abort()
        freq_hz = freq_ghz * 1e9
        k0 = 2.0 * math.pi * freq_hz / C0
        mesh_freq_ghz = mesh_ref_ghz if mesh_ref_ghz is not None else float(freq_
        lambda_min = C0 / (mesh_freq_ghz * 1e9)

        if cached_panels is not None and cached_mesh is not None:
            panels = cached_panels
            mesh = cached_mesh
            linear_mesh_stats_local = dict(cached_mesh_stats or {})
        else:
            panels = _build_panels(
```

```
        geometry_snapshot, unit_scale, lambda_min, max_panels=max_panels,
    )
    preview_infos = _build_coupled_panel_info(panels, materials, freq_ghz
    mesh, linear_mesh_stats_local = _build_linear_mesh_interface_aware(pa
    linear_mesh_stats_local = dict(linear_mesh_stats_local)

panel_lengths = np.asarray([p.length for p in panels], dtype=float)
mesh_reference_values.append(float(mesh_freq_ghz))
panel_count_values.append(int(len(panels)))
panel_length_min_values.append(float(np.min(panel_lengths)) if len(panel_
panel_length_max_values.append(float(np.max(panel_lengths)) if len(panel_

coupled_infos = _build_linear_coupled_infos(mesh, materials, freq_ghz, po
if cached_panels is None:
    linear_mesh_stats_local.update(_linear_coupled_node_report(mesh, coup
done_steps += 1
emit_progress(f"Assembled linear/Galerkin coupled operators at {freq_ghz:

nnodes = len(mesh.nodes)

# Memory estimation — refuse before allocating multi-GB matrices.
n_regions = len({info.minus_region for info in coupled_infos if info.minu
                | {info.plus_region for info in coupled_infos if info.plu
est_gb = _estimate_memory_gb(nnodes, use_cfie=float(cfie_alpha) > 0, n_re
if est_gb > 32.0:
    raise MemoryError(
        f"Estimated peak memory {est_gb:.1f} GB exceeds 32 GB safety limi
        f"({nnodes} nodes, {n_regions} region(s)). "
        f"Reduce panel count, frequency, or use mesh_reference_ghz."
    )
if est_gb > 8.0:
    materials.warn_once(
        f"Estimated peak memory {est_gb:.1f} GB for {nnodes} nodes. "
        "Large problems may cause slowdowns or out-of-memory errors."
    )

if cached_junction_constraints is not None and cached_junction_stats is n
    linear_junction_constraints = cached_junction_constraints
    linear_junction_stats = dict(cached_junction_stats)
else:
    linear_junction_constraints, linear_junction_stats = _build_linear_ju
        mesh, coupled_infos,
    )
junction_stats.update(linear_mesh_stats_local)
junction_stats.update(linear_junction_stats)
orientation_conflicts = int(linear_junction_stats.get("junction_orientati
if orientation_conflicts > 0:
```

```
        materials.warn_once(
            f"Detected {orientation_conflicts} cross-segment junction node(s)
            "inconsistent segment orientation. The solver will proceed, but r
            "may be inaccurate at these junctions. Consider fixing the geomet
            "shared junctions have a physically consistent plus/minus side as
        )
    if linear_junction_constraints.size > 0:
        formulation_label = "2D BIE/MoM coupled dielectric trace formulation
        materials.warn_once(
            (
                "Applied "
                f"{int(linear_junction_stats.get('junction_constraints', 0))}
                f"(trace={int(linear_junction_stats.get('junction_trace_const
                f"flux={int(linear_junction_stats.get('junction_flux_constrai
                f"across {int(linear_junction_stats.get('junction_nodes', 0))
            )
        )

    check_abort()

    # --- TM Robin path: MFIE for PEC and IBC surfaces ---
    use_tm_robin_mfie = (pol == 'TM' and _is_all_robin(coupled_infos))

    if use_tm_robin_mfie:
        formulation_label = "2D MFIE TM Robin (SLP representation)"
        rcs_lin_vec, amp_vec, mfie_residual = _solve_tm_robin_mfie(
            mesh=mesh,
            infos=coupled_infos,
            pol=pol,
            k0=k0,
            elevations_deg=elevations_arr,
        )
        rcs_db_vec = 10.0 * np.log10(rcs_lin_vec)
        residual_vec = np.full(len(elevations), mfie_residual, dtype=float)
        constraint_residual_vec = np.zeros(len(elevations), dtype=float)
        if compute_condition_number:
            cond_values.append(float('nan'))
        reused_matrix_solve_count += len(elevations)

        for idx, elev_deg in enumerate(elevations):
            amp_val = complex(amp_vec[idx])
            residual_local = float(residual_vec[idx])
            samples.append(
                {
                    "frequency_ghz": float(freq_ghz),
                    "theta_inc_deg": float(elev_deg),
                    "theta_scat_deg": float(elev_deg),
```

```
                "rcs_linear": float(rcs_lin_vec[idx]),
                "rcs_db": float(rcs_db_vec[idx]),
                "rcs_amp_real": float(np.real(amp_val)),
                "rcs_amp_imag": float(np.imag(amp_val)),
                "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_v
                "linear_residual": residual_local,
            }
        )
        residual_values.append(residual_local)
        constraint_residual_values.append(0.0)
        done_steps += 1
        emit_progress(f"MFIE solved {freq_ghz:g} GHz at {elev_deg:g} deg"
    continue

# --- Dielectric indirect formulation ---
use_dielectric_indirect = _is_single_dielectric_body(coupled_infos)

if use_dielectric_indirect:
    formulation_label = "2D indirect two-density dielectric formulation"
    rcs_lin_vec, amp_vec, diel_residual = _solve_dielectric_indirect(
        mesh=mesh,
        infos=coupled_infos,
        pol=pol,
        k0=k0,
        elevations_deg=elevations_arr,
    )
    rcs_db_vec = 10.0 * np.log10(rcs_lin_vec)
    residual_vec = np.full(len(elevations), diel_residual, dtype=float)
    constraint_residual_vec = np.zeros(len(elevations), dtype=float)
    if compute_condition_number:
        cond_values.append(float('nan'))
    reused_matrix_solve_count += len(elevations)

    for idx, elev_deg in enumerate(elevations):
        amp_val = complex(amp_vec[idx])
        residual_local = float(residual_vec[idx])
        samples.append(
            {
                "frequency_ghz": float(freq_ghz),
                "theta_inc_deg": float(elev_deg),
                "theta_scat_deg": float(elev_deg),
                "rcs_linear": float(rcs_lin_vec[idx]),
                "rcs_db": float(rcs_db_vec[idx]),
                "rcs_amp_real": float(np.real(amp_val)),
                "rcs_amp_imag": float(np.imag(amp_val)),
                "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_v
                "linear_residual": residual_local,
```

```
            }
        )
        residual_values.append(residual_local)
        constraint_residual_values.append(0.0)
        done_steps += 1
        emit_progress(f"Dielectric solved {freq_ghz:g} GHz at {elev_deg:g
    continue

# --- IBC Robin-BIE formulation ---
use_robin_bie = _is_all_ibc(coupled_infos)

if use_robin_bie:
    formulation_label = "2D Robin-BIE IBC formulation (SLP representation
    rcs_lin_vec, amp_vec, robin_residual = _solve_robin_bie(
        mesh=mesh,
        infos=coupled_infos,
        pol=pol,
        k0=k0,
        elevations_deg=elevations_arr,
    )
    rcs_db_vec = 10.0 * np.log10(rcs_lin_vec)
    residual_vec = np.full(len(elevations), robin_residual, dtype=float)
    constraint_residual_vec = np.zeros(len(elevations), dtype=float)
    if compute_condition_number:
        cond_values.append(float('nan'))
    reused_matrix_solve_count += len(elevations)

    for idx, elev_deg in enumerate(elevations):
        amp_val = complex(amp_vec[idx])
        residual_local = float(residual_vec[idx])
        samples.append(
            {
                "frequency_ghz": float(freq_ghz),
                "theta_inc_deg": float(elev_deg),
                "theta_scat_deg": float(elev_deg),
                "rcs_linear": float(rcs_lin_vec[idx]),
                "rcs_db": float(rcs_db_vec[idx]),
                "rcs_amp_real": float(np.real(amp_val)),
                "rcs_amp_imag": float(np.imag(amp_val)),
                "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_v
                "linear_residual": residual_local,
            }
        )
        residual_values.append(residual_local)
        constraint_residual_values.append(0.0)
        done_steps += 1
        emit_progress(f"Robin-BIE solved {freq_ghz:g} GHz at {elev_deg:g}
```

```
        continue

    # --- Standard coupled formulation (TE PEC, or mixed PEC+IBC) ---
    # The coupled trace formulation is validated ONLY for TE PEC and TE PEC+I
    # For mixed PEC+dielectric or mixed IBC+dielectric, the formulation
    # cannot produce correct far-field amplitudes for either polarization
    # because the exterior BIE alone determines the amplitude regardless
    # of the flux continuity parameter.  Raise instead of silently giving wro
    has_pec = any(info.bc_kind == 'robin' and abs(info.robin_impedance) <= EP
    has_diel = any(info.bc_kind == 'transmission' for info in coupled_infos)
    has_ibc = any(info.bc_kind == 'robin' and abs(info.robin_impedance) > EPS

    if has_diel and (has_pec or has_ibc):
        formulation_label = "2D unified SLP mixed PEC/IBC + dielectric formul
        rcs_lin_vec, amp_vec, mixed_residual = _solve_mixed_pec_dielectric(
            mesh=mesh,
            infos=coupled_infos,
            pol=pol,
            k0=k0,
            elevations_deg=elevations_arr,
        )
        rcs_db_vec = 10.0 * np.log10(rcs_lin_vec)
        residual_vec = np.full(len(elevations), mixed_residual, dtype=float)
        constraint_residual_vec = np.zeros(len(elevations), dtype=float)
        if compute_condition_number:
            cond_values.append(float('nan'))
        reused_matrix_solve_count += len(elevations)

        for idx, elev_deg in enumerate(elevations):
            amp_val = complex(amp_vec[idx])
            residual_local = float(residual_vec[idx])
            samples.append(
                {
                    "frequency_ghz": float(freq_ghz),
                    "theta_inc_deg": float(elev_deg),
                    "theta_scat_deg": float(elev_deg),
                    "rcs_linear": float(rcs_lin_vec[idx]),
                    "rcs_db": float(rcs_db_vec[idx]),
                    "rcs_amp_real": float(np.real(amp_val)),
                    "rcs_amp_imag": float(np.imag(amp_val)),
                    "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_v
                    "linear_residual": residual_local,
                }
            )
            residual_values.append(residual_local)
            constraint_residual_values.append(0.0)
            done_steps += 1
```

```
        emit_progress(f"Mixed solved {freq_ghz:g} GHz at {elev_deg:g} deg
    continue
a_mat = _build_coupled_matrix_linear(
    mesh=mesh,
    infos=coupled_infos,
    pol=pol,
    cfie_alpha=float(cfie_alpha),
    k_air=float(k0),
)
rhs_mat = _build_coupled_rhs_many_linear(
    mesh=mesh,
    infos=coupled_infos,
    k_air=k0,
    elevations_deg=elevations_arr,
    cfie_alpha=float(cfie_alpha),
)
constraint_mat = linear_junction_constraints if linear_junction_constrain
_ensure_finite_linear_system(a_mat, rhs_mat, label="linear/Galerkin coupl
prepared = _prepare_linear_solver(a_mat, constraint_mat=constraint_mat, s
if compute_condition_number:
    cond_eval_mat = prepared.reduced_mat if prepared.reduced_mat is not N
    cond_values.append(_cond_estimate(cond_eval_mat))
sol_mat = _solve_with_prepared_solver(prepared, rhs_mat)
if sol_mat.ndim == 1:
    sol_mat = sol_mat.reshape(-1, 1)

primal_sol_mat = sol_mat[: 2 * nnodes, :]
reused_matrix_solve_count += len(elevations)
residual_vec = _residual_norm_many(a_mat, primal_sol_mat, rhs_mat)
constraint_residual_vec = _constraint_residual_norm_many(constraint_mat,
rcs_lin_vec, amp_vec = _backscatter_rcs_coupled_many_linear(
    mesh=mesh,
    infos=coupled_infos,
    u_trace_nodes_mat=primal_sol_mat[:nnodes, :],
    q_minus_nodes_mat=primal_sol_mat[nnodes:, :],
    k_air=k0,
    elevations_deg=elevations_arr,
)
rcs_db_vec = 10.0 * np.log10(rcs_lin_vec)

for idx, elev_deg in enumerate(elevations):
    amp_val = complex(amp_vec[idx])
    residual_local = float(residual_vec[idx])
    samples.append(
        {
            "frequency_ghz": float(freq_ghz),
            "theta_inc_deg": float(elev_deg),
```

```
                "theta_scat_deg": float(elev_deg),
                "rcs_linear": float(rcs_lin_vec[idx]),
                "rcs_db": float(rcs_db_vec[idx]),
                "rcs_amp_real": float(np.real(amp_val)),
                "rcs_amp_imag": float(np.imag(amp_val)),
                "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_val))
                "linear_residual": residual_local,
            }
        )
        residual_values.append(residual_local)
        constraint_residual_values.append(float(constraint_residual_vec[idx])
        done_steps += 1
        emit_progress(f"Solved {freq_ghz:g} GHz at {elev_deg:g} deg")

    metadata: Dict[str, Any] = {
        "source_path": str(geometry_snapshot.get("source_path", "") or ""),
        "segment_count": int(len(geometry_snapshot.get("segments", []) or [])),
        "panel_count": int(np.max(panel_count_values)) if panel_count_values else
        "panel_count_min": int(np.min(panel_count_values)) if panel_count_values
        "panel_count_max": int(np.max(panel_count_values)) if panel_count_values
        "panel_length_min_m": float(np.min(panel_length_min_values)) if panel_len
        "panel_length_max_m": float(np.max(panel_length_max_values)) if panel_len
        "mesh_reference_ghz": float(mesh_reference_values[0]) if len(set(round(v,
        "mesh_reference_ghz_min": float(np.min(mesh_reference_values)) if mesh_re
        "mesh_reference_ghz_max": float(np.max(mesh_reference_values)) if mesh_re
        "polarization_internal": pol,
        "polarization_user": _canonical_user_polarization_label(polarization),
        "polarization_aliases": [_canonical_user_polarization_label(polarization)
        "polarization_export": _canonical_user_polarization_label(polarization),
        "polarization_export_alias": _primary_alias_for_user_polarization(polariz
        "rcs_normalization_mode": rcs_norm_mode,
        "formulation": formulation_label,
        "solver_method": str(solver_method),
        "residual_norm_max": float(np.max(residual_values)) if residual_values el
        "residual_norm_mean": float(np.mean(residual_values)) if residual_values
        "constraint_residual_norm_max": float(np.max(constraint_residual_values))
        "constraint_residual_norm_mean": float(np.mean(constraint_residual_values
        "condition_est_max": float(np.max(cond_values)) if cond_values else float
        "condition_est_mean": float(np.mean(cond_values)) if cond_values else flo
        "warnings": list(materials.warnings),
        "warning_count": int(len(materials.warnings)),
        "math_backend_real_bessel": _BESSEL.backend_name,
        "math_backend_complex_hankel": _complex_hankel_backend_name(),
        "reused_matrix_solve_count": int(reused_matrix_solve_count),
        "parallel_elevation_solve_count": 0,
        "max_parallel_workers_used": int(max_parallel_workers_used),
        "mesh_reference_frequency_used": bool(mesh_ref_ghz is not None),
```

```
        "cfie_alpha": float(cfie_alpha),
        "solver_method": str(solver_method),
        "junction_nodes": int(junction_stats.get("junction_nodes", 0)),
        "junction_constraints": int(junction_stats.get("junction_constraints", 0)
        "junction_panels": int(junction_stats.get("junction_panels", 0)),
        "junction_trace_constraints": int(junction_stats.get("junction_trace_cons
        "junction_flux_constraints": int(junction_stats.get("junction_flux_constr
        "junction_orientation_conflict_nodes": int(junction_stats.get("junction_o
        "linear_node_count": int(junction_stats.get("linear_node_count", 0)),
        "linear_element_count": int(junction_stats.get("linear_element_count", 0)
        "shared_node_count": int(junction_stats.get("shared_node_count", 0)),
        "split_node_count": int(junction_stats.get("split_node_count", 0)),
        "split_boundary_primitive_count": int(junction_stats.get("split_boundary_
        "multi_signature_node_count": int(junction_stats.get("multi_signature_nod
        "preflight": dict(preflight_report),
    }

    quality_gate = evaluate_quality_gate(metadata, thresholds=quality_thresholds)
    metadata["quality_gate"] = quality_gate
    if strict_quality_gate and not bool(quality_gate.get("passed", False)):
        reason = str(quality_gate.get("reason", "quality gate failed"))
        raise ValueError(f"Quality gate failed: {reason}")

    return {
        "solver": "2d_bie_mom_rcs",
        "scattering_mode": "monostatic",
        "polarization": _canonical_user_polarization_label(polarization),
        "polarization_export": _canonical_user_polarization_label(polarization),
        "samples": samples,
        "metadata": metadata,
    }


def _farfield_at_angles_coupled(
    mesh: LinearMesh,
    infos: List[PanelCoupledInfo],
    u_trace: np.ndarray,
    q_minus: np.ndarray,
    k_air: float,
    obs_angles_deg: np.ndarray,
    order: int = 8,
) -> Tuple[np.ndarray, np.ndarray]:
    """
    Evaluate coupled-formulation far-field at arbitrary observation angles.

    Unlike the monostatic projector, this takes a single solution vector
    (one incidence angle) and projects it at multiple observation angles.
```

```
    """

    obs = np.asarray(obs_angles_deg, dtype=float).reshape(-1)
    phi = np.deg2rad(obs)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, int(order)))
    amp = np.zeros(obs.size, dtype=np.complex128)

    for elem, info in zip(mesh.elements, infos):
        ids = np.asarray(elem.node_ids, dtype=int)
        beta = complex(info.q_plus_beta)
        gamma = complex(info.q_plus_gamma)
        u_local = u_trace[ids]
        q_local = q_minus[ids]
        q_plus_local = beta * q_local + gamma * u_local
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k_air * (dirs @ rp))
            dot_n = dirs @ elem.normal
            u_t = shape[0] * u_local[0] + shape[1] * u_local[1]
            q_t = shape[0] * q_local[0] + shape[1] * q_local[1]
            qp_t = shape[0] * q_plus_local[0] + shape[1] * q_plus_local[1]
            if info.minus_has_incident:
                amp += float(w) * float(elem.length) * phase * (
                    -q_t + 1j * k_air * dot_n * u_t
                )
            if info.plus_has_incident:
                amp += float(w) * float(elem.length) * phase * (
                    qp_t - 1j * k_air * dot_n * u_t
                )

    rcs_lin = _rcs_sigma_from_amp(amp, k_air)
    return rcs_lin, amp


def _farfield_at_angles_slp(
    mesh: LinearMesh,
    density: np.ndarray,
    k_air: float,
    obs_angles_deg: np.ndarray,
    order: int = 8,
) -> Tuple[np.ndarray, np.ndarray]:
    """SLP far-field projector at arbitrary observation angles (for TM PEC MFIE).

    obs = np.asarray(obs_angles_deg, dtype=float).reshape(-1)
    phi = np.deg2rad(obs)
```

```
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, int(order)))
    amp = np.zeros(obs.size, dtype=np.complex128)

    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        sigma_local = density[ids]
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k_air * (dirs @ rp))
            sigma_t = shape[0] * sigma_local[0] + shape[1] * sigma_local[1]
            amp += float(w) * float(elem.length) * phase * sigma_t

    rcs_lin = _rcs_sigma_from_amp(amp, k_air)
    return rcs_lin, amp


def _farfield_at_angles_dlp(
    mesh: LinearMesh,
    density: np.ndarray,
    k_air: float,
    obs_angles_deg: np.ndarray,
    order: int = 8,
) -> Tuple[np.ndarray, np.ndarray]:
    """DLP far-field projector at arbitrary observation angles (for dielectric in

    obs = np.asarray(obs_angles_deg, dtype=float).reshape(-1)
    phi = np.deg2rad(obs)
    dirs = np.stack([np.cos(phi), np.sin(phi)], axis=1)
    qt, qw = _get_quadrature(max(2, int(order)))
    amp = np.zeros(obs.size, dtype=np.complex128)

    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        mu_local = density[ids]
        for t, w in zip(qt, qw):
            shape = _linear_shape_values(float(t))
            rp = elem.p0 + float(t) * (elem.p1 - elem.p0)
            phase = np.exp(1j * k_air * (dirs @ rp))
            dot_n = dirs @ elem.normal
            mu_t = shape[0] * mu_local[0] + shape[1] * mu_local[1]
            amp += float(w) * float(elem.length) * phase * 1j * k_air * dot_n * m

    rcs_lin = _rcs_sigma_from_amp(amp, k_air)
    return rcs_lin, amp
```

```
def solve_bistatic_rcs_2d(
    geometry_snapshot: Dict[str, Any],
    frequencies_ghz: List[float],
    incidence_angles_deg: List[float],
    observation_angles_deg: List[float],
    polarization: str,
    geometry_units: str = "inches",
    material_base_dir: str | None = None,
    progress_callback: Callable[[int, int, str], None] | None = None,
    max_panels: int = MAX_PANELS_DEFAULT,
    mesh_reference_ghz: float | None = None,
    cfie_alpha: float = CFIE_ALPHA_DEFAULT,
    abort_event: threading.Event | None = None,
    solver_method: str = "auto",
) -> Dict[str, Any]:
    """
    Bistatic 2D RCS solver.

    For each frequency and incidence angle, solves the boundary integral equation
    and evaluates the far-field RCS at all requested observation angles.

    Returns samples with ``theta_inc_deg != theta_scat_deg`` in general.
    Compatible with ``export_result_to_grim`` which splits by incidence angle.
    """

    if not frequencies_ghz:
        raise ValueError("At least one frequency is required.")
    if not incidence_angles_deg:
        raise ValueError("At least one incidence angle is required.")
    if not observation_angles_deg:
        raise ValueError("At least one observation angle is required.")

    frequencies = [float(f) for f in frequencies_ghz]
    inc_angles = [float(a) for a in incidence_angles_deg]
    obs_angles = [float(a) for a in observation_angles_deg]
    if any(f <= 0.0 for f in frequencies):
        raise ValueError("Frequencies must be positive GHz values.")

    _raise_if_untrusted_math_backends()
    pol = _normalize_polarization(polarization)
    unit_scale = _unit_scale_to_meters(geometry_units)
    base_dir = material_base_dir or os.getcwd()

    mesh_ref_ghz = float(mesh_reference_ghz) if mesh_reference_ghz is not None el

    preflight_report = validate_geometry_snapshot_for_solver(geometry_snapshot, b
```

```
materials = MaterialLibrary.from_entries(
    geometry_snapshot.get("ibcs", []) or [],
    geometry_snapshot.get("dielectrics", []) or [],
    base_dir=base_dir,
)

samples: List[Dict[str, Any]] = []
total_steps = len(frequencies) * len(inc_angles)
done_steps = 0
obs_arr = np.asarray(obs_angles, dtype=float)

def check_abort() -> None:
    if abort_event is not None and abort_event.is_set():
        raise InterruptedError("Solve cancelled by user.")

def emit_progress(msg: str) -> None:
    if progress_callback is not None:
        try:
            progress_callback(done_steps, total_steps, msg)
        except Exception:
            pass

for freq_ghz in frequencies:
    check_abort()
    freq_hz = freq_ghz * 1e9
    k0 = 2.0 * math.pi * freq_hz / C0
    mesh_freq_ghz = mesh_ref_ghz if mesh_ref_ghz is not None else float(freq_
    lambda_min = C0 / (mesh_freq_ghz * 1e9)

    panels = _build_panels(geometry_snapshot, unit_scale, lambda_min, max_pan
    preview_infos = _build_coupled_panel_info(panels, materials, freq_ghz, po
    mesh, _ = _build_linear_mesh_interface_aware(panels, preview_infos)
    coupled_infos = _build_linear_coupled_infos(mesh, materials, freq_ghz, po
    nnodes = len(mesh.nodes)

    use_tm_robin_mfie = (pol == 'TM' and _is_all_robin(coupled_infos))
    use_diel_indirect = _is_single_dielectric_body(coupled_infos)
    use_mixed = _is_mixed_pec_dielectric(coupled_infos)

    # Pre-assemble system matrices (reused across incidence angles).

    # --- Mixed PEC+dielectric pre-assembly ---
    mixed_sys = None
    mixed_diel_nodes = None
    mixed_node_is_diel = None
    mixed_node_is_pec = None
    mixed_node_alpha = None
```

```
if use_mixed:
    elements = list(mesh.elements)
    nelems = len(elements)
    mixed_node_is_diel = np.zeros(nnodes, dtype=bool)
    mixed_elem_is_diel = np.zeros(nelems, dtype=bool)
    mixed_node_alpha = np.zeros(nnodes, dtype=np.complex128)
    mixed_node_is_pec = np.zeros(nnodes, dtype=bool)
    for eidx, (elem, info) in enumerate(zip(elements, coupled_infos)):
        if info.bc_kind == 'transmission':
            mixed_node_is_diel[elem.node_ids[0]] = True
            mixed_node_is_diel[elem.node_ids[1]] = True
            mixed_elem_is_diel[eidx] = True
        else:
            for nid in elem.node_ids:
                z_s = complex(info.robin_impedance)
                if abs(z_s) <= EPS:
                    mixed_node_is_pec[nid] = True
                else:
                    eps_m = info.eps_minus if info.minus_region >= 0 else
                    mu_m = info.mu_minus if info.minus_region >= 0 else i
                    k_m = info.k_minus if info.minus_region >= 0 else inf
                    mixed_node_alpha[nid] = _surface_robin_alpha(pol, eps
    mixed_diel_nodes = np.flatnonzero(mixed_node_is_diel)
    n_diel = int(mixed_diel_nodes.size)
    n_sys = nnodes + n_diel
    diel_info = next(info for info in coupled_infos if info.bc_kind == 't
    k_int_vals = {complex(i.k_plus) for i in coupled_infos if i.bc_kind =
    if not k_int_vals:
        k_int_vals = {complex(i.k_minus) for i in coupled_infos if i.bc_k
    k_int = k_int_vals.pop() if k_int_vals else complex(k0)
    if pol == 'TE':
        beta = complex(diel_info.mu_plus / diel_info.mu_minus) if diel_in
    else:
        beta = complex(diel_info.eps_plus / diel_info.eps_minus) if diel_
    if abs(beta) <= EPS:
        beta = 1.0 + 0.0j
    inv_beta = 1.0 / beta
    S_0, _ = _assemble_linear_operator_matrices(mesh, k0, False)
    _, Kp_0 = _assemble_linear_operator_matrices(mesh, k0, True)
    S_k, _ = _assemble_linear_operator_matrices(mesh, k_int, False, sourc
    _, Kp_k = _assemble_linear_operator_matrices(mesh, k_int, True, sourc
    M = _assemble_linear_mass_matrix(mesh)
    mixed_sys = np.zeros((n_sys, n_sys), dtype=np.complex128)
    for nid in np.flatnonzero(~mixed_node_is_diel):
        row = int(nid)
        is_pec = bool(mixed_node_is_pec[nid])
        alpha = complex(mixed_node_alpha[nid])
```

```
        if pol == 'TE' and is_pec:
            mixed_sys[row, :nnodes] = S_0[nid, :]
        elif pol == 'TM' and is_pec:
            mixed_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :]
        else:
            mixed_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :] + a
    for di, nid in enumerate(mixed_diel_nodes):
        row = int(nid)
        mixed_sys[row, :nnodes] = -0.5 * M[nid, :] + Kp_0[nid, :]
        for dj, nid_j in enumerate(mixed_diel_nodes):
            mixed_sys[row, nnodes + dj] = -inv_beta * (0.5 * M[nid, nid_j
    for di, nid in enumerate(mixed_diel_nodes):
        row = nnodes + di
        mixed_sys[row, :nnodes] = S_0[nid, :]
        for dj, nid_j in enumerate(mixed_diel_nodes):
            mixed_sys[row, nnodes + dj] = -S_k[nid, nid_j]

# --- TM Robin MFIE pre-assembly ---
mfie_sys = None
mfie_alpha_nodes = None
if use_tm_robin_mfie:
    s_std, _ = _assemble_linear_operator_matrices(mesh, k0, obs_normal_de
    _, Kp = _assemble_linear_operator_matrices(mesh, k0, obs_normal_deriv
    M_mat = _assemble_linear_mass_matrix(mesh)
    mfie_sys = -0.5 * M_mat + Kp
    # Per-node Robin alpha for TM IBC.
    mfie_alpha_nodes = np.zeros(nnodes, dtype=np.complex128)
    inc_map: Dict[int, List[int]] = {}
    for eidx, elem in enumerate(mesh.elements):
        for nid in elem.node_ids:
            inc_map.setdefault(int(nid), []).append(int(eidx))
    for nid in range(nnodes):
        eids = inc_map.get(int(nid), [])
        if eids:
            info = coupled_infos[int(eids[0])]
            z_s = complex(info.robin_impedance)
            if abs(z_s) > EPS:
                eps_m = info.eps_minus if info.minus_region >= 0 else inf
                mu_m = info.mu_minus if info.minus_region >= 0 else info.
                k_m = info.k_minus if info.minus_region >= 0 else info.k_
                mfie_alpha_nodes[nid] = _surface_robin_alpha(pol, eps_m,
    if np.any(np.abs(mfie_alpha_nodes) > EPS):
        mfie_sys = mfie_sys + np.diag(mfie_alpha_nodes) @ s_std

for inc_deg in inc_angles:
    check_abort()
    inc_arr = np.asarray([inc_deg], dtype=float)
```

```
if use_mixed and mixed_sys is not None:
    # Mixed PEC+dielectric solve for this incidence angle.
    n_diel_local = int(mixed_diel_nodes.size)
    n_sys_local = nnodes + n_diel_local
    rhs_mixed = np.zeros(n_sys_local, dtype=np.complex128)
    for elem in mesh.elements:
        ids = list(elem.node_ids)
        load_u = _linear_element_incident_load_many(elem, k_air=k0, e
        load_dn = _linear_element_incident_dn_load_many(elem, k_air=k
        for li, nid in enumerate(ids):
            if mixed_node_is_diel[nid]:
                di = int(np.flatnonzero(mixed_diel_nodes == nid)[0])
                rhs_mixed[nid] -= float(load_dn[li, 0].real) + 1j * f
                rhs_mixed[nnodes + di] -= float(load_u[li, 0].real) +
            else:
                is_pec = bool(mixed_node_is_pec[nid])
                alpha = complex(mixed_node_alpha[nid])
                if pol == 'TE' and is_pec:
                    rhs_mixed[nid] -= load_u[li, 0]
                elif pol == 'TM' and is_pec:
                    rhs_mixed[nid] -= load_dn[li, 0]
                else:
                    rhs_mixed[nid] -= (load_dn[li, 0] + alpha * load_
    sol_mixed = np.linalg.solve(mixed_sys, rhs_mixed)
    sigma = sol_mixed[:nnodes]
    rcs_lin, amp = _farfield_at_angles_slp(mesh, sigma, k0, obs_arr)

elif use_tm_robin_mfie:
    # Generalized MFIE solve for this incidence angle.
    rhs = np.zeros(nnodes, dtype=np.complex128)
    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        rhs[ids] -= _linear_element_incident_dn_load_many(
            elem, k_air=k0, elevations_deg=inc_arr,
        )[:, 0]
        if mfie_alpha_nodes is not None and np.any(np.abs(mfie_alpha_
            rhs[ids] -= mfie_alpha_nodes[ids] * _linear_element_incid
                elem, k_air=k0, elevations_deg=inc_arr,
            )[:, 0]
    sigma = np.linalg.solve(mfie_sys, rhs)
    rcs_lin, amp = _farfield_at_angles_slp(mesh, sigma, k0, obs_arr)

elif use_diel_indirect:
    # Indirect dielectric solve for this incidence angle.
    info0 = coupled_infos[0]
    k1_vals = {complex(i.k_plus) for i in coupled_infos if i.plus_reg
```

```
        k1 = k1_vals.pop() if k1_vals else k0
        factor = complex(info0.mu_minus / info0.mu_plus) if pol == 'TE' e

        S0, K0 = _assemble_linear_operator_matrices(mesh, k0, False)
        _, Kp1 = _assemble_linear_operator_matrices(mesh, k1, True)
        S1, _ = _assemble_linear_operator_matrices(mesh, k1, False)
        D0 = _assemble_linear_hypersingular_matrix(mesh, k0)
        M = _assemble_linear_mass_matrix(mesh)

        a_sys = np.zeros((2 * nnodes, 2 * nnodes), dtype=np.complex128)
        a_sys[:nnodes, :nnodes] = 0.5 * M + K0
        a_sys[:nnodes, nnodes:] = -S1
        a_sys[nnodes:, :nnodes] = D0
        a_sys[nnodes:, nnodes:] = factor * (0.5 * M + Kp1)

        rhs_sys = np.zeros(2 * nnodes, dtype=np.complex128)
        for elem in mesh.elements:
            ids = np.asarray(elem.node_ids, dtype=int)
            rhs_sys[ids] += _linear_element_incident_load_many(elem, k0,
            rhs_sys[nnodes + ids] -= _linear_element_incident_dn_load_man

        sol = np.linalg.solve(a_sys, rhs_sys)
        mu = sol[:nnodes]
        rcs_lin, amp = _farfield_at_angles_dlp(mesh, mu, k0, obs_arr)

    else:
        # Coupled formulation (TE PEC, IBC, mixed).
        jc, _ = _build_linear_junction_constraints(mesh, coupled_infos)
        a_mat = _build_coupled_matrix_linear(
            mesh=mesh, infos=coupled_infos, pol=pol,
            cfie_alpha=float(cfie_alpha), k_air=float(k0),
        )
        rhs = _build_coupled_rhs_many_linear(
            mesh=mesh, infos=coupled_infos, k_air=k0,
            elevations_deg=inc_arr, cfie_alpha=float(cfie_alpha),
        )
        cmat = jc if jc.size > 0 else None
        prepared = _prepare_linear_solver(a_mat, constraint_mat=cmat, sol
        sol = _solve_with_prepared_solver(prepared, rhs)
        if sol.ndim == 2:
            sol = sol[:, 0]
        u = sol[:nnodes]
        q = sol[nnodes:2 * nnodes]
        rcs_lin, amp = _farfield_at_angles_coupled(
            mesh, coupled_infos, u, q, k0, obs_arr,
        )
```

```
            rcs_db = 10.0 * np.log10(rcs_lin)
            for idx, obs_deg in enumerate(obs_angles):
                amp_val = complex(amp[idx])
                samples.append({
                    "frequency_ghz": float(freq_ghz),
                    "theta_inc_deg": float(inc_deg),
                    "theta_scat_deg": float(obs_deg),
                    "rcs_linear": float(rcs_lin[idx]),
                    "rcs_db": float(rcs_db[idx]),
                    "rcs_amp_real": float(np.real(amp_val)),
                    "rcs_amp_imag": float(np.imag(amp_val)),
                    "rcs_amp_phase_deg": float(math.degrees(cmath.phase(amp_val))
                    "linear_residual": 0.0,
                })

            done_steps += 1
            emit_progress(f"Bistatic {freq_ghz:g} GHz inc={inc_deg:g} deg")

    return {
        "solver": "2d_bie_mom_rcs",
        "scattering_mode": "bistatic",
        "polarization": _canonical_user_polarization_label(polarization),
        "polarization_export": _canonical_user_polarization_label(polarization),
        "samples": samples,
        "metadata": {
            "formulation": "bistatic 2D BIE/MoM",
            "cfie_alpha": float(cfie_alpha),
            "solver_method": str(solver_method),
        },
    }

def compute_surface_currents(
    geometry_snapshot: Dict[str, Any],
    frequency_ghz: float,
    elevation_deg: float,
    polarization: str,
    geometry_units: str = "inches",
    material_base_dir: str | None = None,
    cfie_alpha: float = CFIE_ALPHA_DEFAULT,
    max_panels: int = MAX_PANELS_DEFAULT,
) -> Dict[str, Any]:
    """
    Compute and return boundary unknowns (surface currents) for visualization.

    Returns element-center positions, the boundary density (current), panel norma
    and the formulation used.  This is a single-frequency, single-angle solve wit
    full boundary solution output for debugging.
```

```
    """

    pol = _normalize_polarization(polarization)
    unit_scale = _unit_scale_to_meters(geometry_units)
    base_dir = material_base_dir or os.getcwd()
    freq_hz = frequency_ghz * 1e9
    k0 = 2.0 * math.pi * freq_hz / C0
    lambda_min = C0 / freq_hz

    preflight = validate_geometry_snapshot_for_solver(geometry_snapshot, base_dir
    panels = _build_panels(geometry_snapshot, unit_scale, lambda_min, max_panels=
    materials = MaterialLibrary.from_entries(
        geometry_snapshot.get("ibcs", []) or [],
        geometry_snapshot.get("dielectrics", []) or [],
        base_dir=base_dir,
    )
    preview_infos = _build_coupled_panel_info(panels, materials, frequency_ghz, p
    mesh, _ = _build_linear_mesh_interface_aware(panels, preview_infos)
    coupled_infos = _build_linear_coupled_infos(mesh, materials, frequency_ghz, p
    nnodes = len(mesh.nodes)
    elev_arr = np.asarray([elevation_deg], dtype=float)

    centers = np.asarray([e.center for e in mesh.elements], dtype=float)
    normals = np.asarray([e.normal for e in mesh.elements], dtype=float)
    lengths = np.asarray([e.length for e in mesh.elements], dtype=float)

    use_tm_robin = pol == 'TM' and _is_all_robin(coupled_infos)
    use_diel = _is_single_dielectric_body(coupled_infos)
    use_ibc = _is_all_ibc(coupled_infos)

    if use_tm_robin:
        # MFIE: solve for SLP density sigma
        _, kp = _assemble_linear_operator_matrices(mesh, k0, obs_normal_deriv=Tru
        s_std, _ = _assemble_linear_operator_matrices(mesh, k0, obs_normal_deriv=
        m_mat = _assemble_linear_mass_matrix(mesh)
        alpha_nodes = np.zeros(nnodes, dtype=np.complex128)
        inc_map: Dict[int, List[int]] = {}
        for eidx, elem in enumerate(mesh.elements):
            for nid in elem.node_ids:
                inc_map.setdefault(int(nid), []).append(int(eidx))
        for nid in range(nnodes):
            eids = inc_map.get(int(nid), [])
            if eids:
                info = coupled_infos[int(eids[0])]
                z_s = complex(info.robin_impedance)
                if abs(z_s) > EPS:
                    eps_m = info.eps_minus if info.minus_region >= 0 else info.ep
```

```
                mu_m = info.mu_minus if info.minus_region >= 0 else info.mu_p
                k_m = info.k_minus if info.minus_region >= 0 else info.k_plus
                alpha_nodes[nid] = _surface_robin_alpha(pol, eps_m, mu_m, k_m
    a_sys = -0.5 * m_mat + kp
    if np.any(np.abs(alpha_nodes) > EPS):
        a_sys += np.diag(alpha_nodes) @ s_std
    rhs = np.zeros(nnodes, dtype=np.complex128)
    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        rhs[ids] -= _linear_element_incident_dn_load_many(elem, k0, elev_arr)
        if np.any(np.abs(alpha_nodes[ids]) > EPS):
            rhs[ids] -= alpha_nodes[ids] * _linear_element_incident_load_many
    sigma_nodes = np.linalg.solve(a_sys, rhs)
    # Interpolate to element centers
    density = np.asarray([
        0.5 * (sigma_nodes[e.node_ids[0]] + sigma_nodes[e.node_ids[1]])
        for e in mesh.elements
    ], dtype=np.complex128)
    formulation = "TM Robin MFIE (SLP density)"

elif use_diel:
    rcs_lin, amp, _ = _solve_dielectric_indirect(
        mesh, coupled_infos, pol, k0, elev_arr)
    # Re-solve to extract density (simplified: re-do the solve)
    info0 = coupled_infos[0]
    k1_vals = {complex(i.k_plus) for i in coupled_infos if i.plus_region > 0}
    k1 = k1_vals.pop() if k1_vals else k0
    factor = complex(info0.mu_minus / info0.mu_plus) if pol == 'TE' else comp
    S0, K0 = _assemble_linear_operator_matrices(mesh, k0, False)
    _, Kp1 = _assemble_linear_operator_matrices(mesh, k1, True)
    S1, _ = _assemble_linear_operator_matrices(mesh, k1, False)
    D0 = _assemble_linear_hypersingular_matrix(mesh, k0)
    M = _assemble_linear_mass_matrix(mesh)
    a = np.zeros((2*nnodes, 2*nnodes), dtype=np.complex128)
    a[:nnodes,:nnodes] = 0.5*M+K0; a[:nnodes,nnodes:] = -S1
    a[nnodes:,:nnodes] = D0; a[nnodes:,nnodes:] = factor*(0.5*M+Kp1)
    rhs = np.zeros(2*nnodes, dtype=np.complex128)
    for elem in mesh.elements:
        ids = np.asarray(elem.node_ids, dtype=int)
        rhs[ids] += _linear_element_incident_load_many(elem, k0, elev_arr)[:,
        rhs[nnodes+ids] -= _linear_element_incident_dn_load_many(elem, k0, el
    sol = np.linalg.solve(a, rhs)
    mu_nodes = sol[:nnodes]
    density = np.asarray([
        0.5*(mu_nodes[e.node_ids[0]]+mu_nodes[e.node_ids[1]])
        for e in mesh.elements
    ], dtype=np.complex128)
```

```
        formulation = "Indirect dielectric (DLP density)"

    elif use_ibc:
        info0 = coupled_infos[0]
        alpha = _surface_robin_alpha(pol, info0.eps_minus, info0.mu_minus, comple
        S, _ = _assemble_linear_operator_matrices(mesh, k0, False)
        _, Kp = _assemble_linear_operator_matrices(mesh, k0, True)
        M = _assemble_linear_mass_matrix(mesh)
        a = -0.5*M + Kp + alpha*S
        rhs = np.zeros(nnodes, dtype=np.complex128)
        for elem in mesh.elements:
            ids = np.asarray(elem.node_ids, dtype=int)
            lu = _linear_element_incident_load_many(elem, k0, elev_arr)[:,0]
            ldn = _linear_element_incident_dn_load_many(elem, k0, elev_arr)[:,0]
            rhs[ids] -= (ldn + alpha*lu)
        sigma_nodes = np.linalg.solve(a, rhs)
        density = np.asarray([
            0.5*(sigma_nodes[e.node_ids[0]]+sigma_nodes[e.node_ids[1]])
            for e in mesh.elements
        ], dtype=np.complex128)
        formulation = "Robin BIE (SLP density)"

    else:
        # TE PEC fallback: use EFIE  S*sigma = -u_inc  to get the physical surfac
        # The coupled trace formulation gives correct RCS but q=0 (wrong boundary
        # because its BC row enforces q=0 instead of u=0 for TE PEC.
        s_mat, _ = _assemble_linear_operator_matrices(mesh, k0, obs_normal_deriv=
        rhs_efie = np.zeros(nnodes, dtype=np.complex128)
        for elem in mesh.elements:
            ids = np.asarray(elem.node_ids, dtype=int)
            rhs_efie[ids] -= _linear_element_incident_load_many(elem, k_air=k0, e
        sigma_nodes = np.linalg.solve(s_mat, rhs_efie)
        density = np.asarray([
            0.5 * (sigma_nodes[e.node_ids[0]] + sigma_nodes[e.node_ids[1]])
            for e in mesh.elements
        ], dtype=np.complex128)
        formulation = "EFIE (SLP density, TE PEC)"

    return {
        "formulation": formulation,
        "frequency_ghz": float(frequency_ghz),
        "elevation_deg": float(elevation_deg),
        "polarization": pol,
        "element_count": int(len(mesh.elements)),
        "node_count": int(nnodes),
        "centers_x": centers[:, 0].tolist(),
        "centers_y": centers[:, 1].tolist(),
```

```
        "normals_x": normals[:, 0].tolist(),
        "normals_y": normals[:, 1].tolist(),
        "lengths": lengths.tolist(),
        "density_real": np.real(density).tolist(),
        "density_imag": np.imag(density).tolist(),
        "density_abs": np.abs(density).tolist(),
        "density_phase_deg": np.degrees(np.angle(density)).tolist(),
    }


def solve_adaptive_frequency_sweep(
    geometry_snapshot: Dict[str, Any],
    freq_start_ghz: float,
    freq_stop_ghz: float,
    elevations_deg: List[float],
    polarization: str,
    geometry_units: str = "inches",
    material_base_dir: str | None = None,
    progress_callback: Callable[[int, int, str], None] | None = None,
    max_panels: int = MAX_PANELS_DEFAULT,
    cfie_alpha: float = CFIE_ALPHA_DEFAULT,
    abort_event: threading.Event | None = None,
    solver_method: str = "auto",
    initial_points: int = 11,
    max_refinements: int = 3,
    db_threshold: float = 1.0,
    max_total_points: int = 201,
) -> Dict[str, Any]:
    """
    Adaptive broadband frequency sweep with automatic refinement.

    Starts with ``initial_points`` uniformly spaced frequencies, then inserts
    midpoints in intervals where adjacent samples differ by more than
    ``db_threshold`` dB.  Repeats up to ``max_refinements`` times or until
    ``max_total_points`` is reached.

    Parameters
    ----------
    freq_start_ghz, freq_stop_ghz : float
        Frequency range in GHz.
    initial_points : int
        Number of uniformly spaced initial samples (default 11).
    max_refinements : int
        Maximum number of adaptive refinement passes (default 3).
    db_threshold : float
        Insert midpoints where adjacent samples differ by more than this (default
    max_total_points : int
```

```
    Hard cap on total frequency points (default 201).

Returns
-------
dict
    Same format as solve_monostatic_rcs_2d with additional metadata about
    the adaptive process (refinement_count, final_point_count).
"""

if freq_start_ghz <= 0 or freq_stop_ghz <= 0:
    raise ValueError("Frequencies must be positive.")
if freq_start_ghz >= freq_stop_ghz:
    raise ValueError("freq_start_ghz must be less than freq_stop_ghz.")
if initial_points < 3:
    initial_points = 3

freqs = sorted(set(np.linspace(freq_start_ghz, freq_stop_ghz, initial_points)
all_samples: List[Dict[str, Any]] = []
freq_to_samples: Dict[float, List[Dict[str, Any]]] = {}

def run_freqs(freq_list: List[float]) -> None:
    if not freq_list:
        return
    result = solve_monostatic_rcs_2d(
        geometry_snapshot=geometry_snapshot,
        frequencies_ghz=freq_list,
        elevations_deg=elevations_deg,
        polarization=polarization,
        geometry_units=geometry_units,
        material_base_dir=material_base_dir,
        progress_callback=progress_callback,
        max_panels=max_panels,
        cfie_alpha=cfie_alpha,
        abort_event=abort_event,
        solver_method=solver_method,
    )
    for s in result.get("samples", []):
        f = round(float(s["frequency_ghz"]), 12)
        freq_to_samples.setdefault(f, []).append(s)
        all_samples.append(s)

run_freqs(freqs)
refinement_count = 0

for _ in range(max_refinements):
    if abort_event is not None and abort_event.is_set():
        break
```

```
    if len(freqs) >= max_total_points:
        break

    # For each elevation, find intervals needing refinement.
    new_freqs: set = set()
    sorted_freqs = sorted(freqs)
    for elev in elevations_deg:
        db_at_freq = {}
        for f in sorted_freqs:
            for s in freq_to_samples.get(round(f, 12), []):
                if abs(s["theta_inc_deg"] - elev) < 0.01:
                    db_at_freq[f] = s["rcs_db"]
                    break

        for i in range(len(sorted_freqs) - 1):
            f0, f1 = sorted_freqs[i], sorted_freqs[i + 1]
            db0 = db_at_freq.get(f0)
            db1 = db_at_freq.get(f1)
            if db0 is not None and db1 is not None:
                if abs(db1 - db0) > db_threshold:
                    mid = 0.5 * (f0 + f1)
                    if mid not in freq_to_samples:
                        new_freqs.add(round(mid, 12))

    if not new_freqs:
        break

    remaining = max_total_points - len(freqs)
    if remaining <= 0:
        break
    new_list = sorted(new_freqs)[:remaining]
    run_freqs(new_list)
    freqs = sorted(set(freqs) | set(new_list))
    refinement_count += 1

return {
    "solver": "2d_bie_mom_rcs",
    "scattering_mode": "monostatic_adaptive",
    "polarization": _canonical_user_polarization_label(polarization),
    "samples": sorted(all_samples, key=lambda s: (s["frequency_ghz"], s["thet
    "metadata": {
        "formulation": "adaptive frequency sweep",
        "initial_points": initial_points,
        "final_point_count": len(freqs),
        "refinement_count": refinement_count,
        "db_threshold": db_threshold,
        "freq_start_ghz": freq_start_ghz,
```

```
        "freq_stop_ghz": freq_stop_ghz,
    },
}
```